PUSHDOWN NORMAL-FORM BISIMULATION: FULL ABSTRACTION AND DECIDABILITY FOR BOUNDED CONTEXT-FREE EXPRESSIONS

VASILEIOS KOUTAVAS, YU-YANG LIN, NIKOS TZEVELEKOS

Editorial scope. One PDNF full-abstraction/decidability main group. Active reviewer appendix preserved, excluded drafts remain excluded; benchmark/up-to support is not counted separately.

Allowed standard prerequisites (editorial).

- Nominal operational semantics and normal-form bisimulation. Original language/typed stacked LTS, nominal equivariance and standard earlier NF-contextual-equivalence facts retained. Complete local stackless/stacked soundness and completeness plus bounded-context-free decidability proofs retained, not outsourced.

Abstract. We propose Pushdown Normal Form (PDNF) Bisimulation to verify contextual equivalence in higher-order functional programming languages with local state. Similar to previous work on Normal Form (NF) bisimulation, PDNF Bisimulation is sound and complete with respect to contextual equivalence. However, unlike traditional NF Bisimulation, PDNF Bisimulation is also decidable for a class of program terms that reach bounded configurations but can potentially have unbounded call stacks and input an unbounded number of unknown functions from their context. Our approach relies on the principle that, in model-checking for reachability, pushdown systems can be simulated by finite-state automata designed to accept their initial/final stack content. We embody this in a stackless Labelled Transition System (LTS), together with an on-the-fly saturation procedure for call stacks, upon which bisimulation is defined. To enhance the effectiveness of our bisimulation, we develop up-to techniques and confirm their soundness for PDNF Bisimulation. We develop a prototype implementation of our technique which is able to verify equivalence in examples from practice and the literature that were out of reach for previous work.

1 Introduction

The problem of contextual equivalence for programming languages aims at determining whether two program terms exhibit the same operational behaviour within any given program context [52]. Although an undecidable problem, relatively recent work is pushing the frontier of decidable equivalence verification in languages incorporating functional, higher-order paradigms, where the behaviour of a term can depend on external unknown code provided by the context as an argument [32, 53, 37, 41, 42].

Normal-Form (NF) bisimulation is a technique that treats unknown code (provided as higher-order arguments) symbolically. The technique was originally defined for characterising Lévy-Longo tree equivalence for the lazy lambda calculus [59] and adapted to languages with call-by-name [46], call-by-value [47], nondeterminism [48], aspects [39], recursive types [49], polymorphism [50], control with state [62], state-only [12], and control-only [11]. More recently, it was used to create equivalence verification techniques for call-by-value functional languages with and without state [41, 42].

However, even NF bisimulations are prone to unbounded behaviour that needs to be explored to verify equivalence. A main source of such behaviour is the potential repeated nested calls between term and context which lead to unbounded stacks of term continuations being created by the bisimulation exploration. Such behaviour is common when programming with callback functions, as is the case in instances of the Observer Pattern [25], shown in the following ML example which models event listeners inspired by JavaScript. Similar examples have been showcased in the literature of program equivalence [18].

Date: arxiv:2311.01325v1; CC BY 4.0.

[0]This publication has emanated from research supported in part by a grant from Science Foundation Ireland under Grant number 13/RC/2094_2; and the Cisco University Research Program Fund, a corporate advised fund of Silicon Valley Community Foundation.

Example 1. $M, N : (\mathsf{unit} \to \mathsf{unit}) * (\mathsf{unit} \to \mathsf{unit}) \to \mathsf{unit} \to \mathsf{unit}$

```
M = let createElement (onstart,onend) =
      ref flag = false in
      let event () =
        flag := true; onstart ();
        flag := false; onend (); !flag
      in event
    in createElement
```

```
N = let createElement
        (onstart,onend) =
      let event () =
        onstart ();
        onend (); 0
      in event
    in createElement
```

In a proof of equivalence of M and N, we can get an unbounded sequence of nested calls to `event`, caused by the unknown functions `onstart` and `onend`. This makes the equivalence non-trivial as `flag` may change values an arbitrary amount of times. However, each assignment of `flag` to true is matched by one setting it back to false because each call of `onstart` is matched by a return of this function. In other words, calls and returns of `onstart` are well-bracketed and, hence, updates of `flag` to true and then false are also well-bracketed.

Reasoning with such examples, for instance using Normal-Form bisimulation, requires the creation of an infinite candidate relation (due to the unbounded stack of nested calls) and then prove it a bisimulation [12]. Although effective for hand-crafted proofs, such an approach would not work for a verification tool of equivalence, which would need to explore all tuples in the candidate relation. In previous equivalence verification techniques such as [41], stacks of effectively pure functions were bounded with up-to techniques which however were unable to finitise the exploration — and thus prove equivalence — of stateful examples such as the one above.

In this work we propose *Pushdown Normal Form (PDNF) Bisimulation* to finitise the exploration of such examples. This is an alternative NF bisimulation for a higher-order functional programming language with local state (Sec. 3) that abstracts away stacks without losing precision, by relying on the fact that traces of such interactions form a *context-free language* and, when model-checking for reachability, they can be simulated precisely by finite-state automata designed to accept their initial/final stack content [15, 23].

We develop PDNF bisimulation on a behavioural LTS of a core-ML language. Contrary to the LTS in [41] (reviewed in Sec. 4), the LTS we design here (Sec. 5.1) is *stackless* and the definition of PDNF bisimulation incorporates a so-called *saturation procedure* [15, 23], albeit performed on the fly as the bisimulation exploration evolves (Sec. 5.2). Our approach follows exact-stack analyses used in Control-Flow Analysis ([65, 19, 33] and in particular [28]), which similarly remove the need for an explicit continuation stack without losing precision.

This approach allows us to adapt a decidability result from nominal pushdown automata [16, 54] to program equivalence of higher-order stateful languages. PDNF bisimulation equivalence is decidable between program terms that reach bounded stackless configurations, even though they may input an unbounded number of unknown functions from their context and their corresponding suppressed stacks may be unbounded (Sec. 5.3). This result is further amenable to up-to techniques, for example considering configurations up to garbage collection.

We establish that PDNF bisimulation is fully abstract for contextual equivalence by relating it to the NF bisimulation of [41] (Sec.(s) 5.4 and 5.5). Furthermore, we increase the strength of our tool in proving equivalences, and similarly to [12, 41], we develop a number of bisimulation up-to techniques and prove their soundness for PDNF bisimulation (Sec. 6). These are powerful rules that allow us to reduce the size of the relation that we examine for bisimulation. In particular, apart from simple techniques such as up to garbage collection, name permutation, and beta reductions, we develop up to separation and name reuse. These two techniques, besides being sound, are also complete in the sense that if after applying them an inequivalence is found, this is a real inequivalence and no backtracking is needed by the bisimulation verification procedure.

We modified the Hobbit tool of [41] to implement a bounded equivalence checker called PDNF-Bisim (Sec. 7), which remains bounded complete, i.e. it finds all inequivalences given sufficiently large bounds and divergence detection. Our tool and this work have the advantage of being able to finitise, and therefore prove, the otherwise infinite NF bisimulation exploration of equivalences that are beyond the reach of Hobbit (Sec. 2). Of course, not all cases can be finitised this way, as contextual equivalence in a Turing complete language is undecidable. Finally, we discuss related and future work (Sec. 8).

2 Motivating Examples

We presented Thm. 1 as a motivating instance of equivalence that can be resolved via PDNF bisimulation. To give an intuitive understanding of the method, we next look at a simplified version

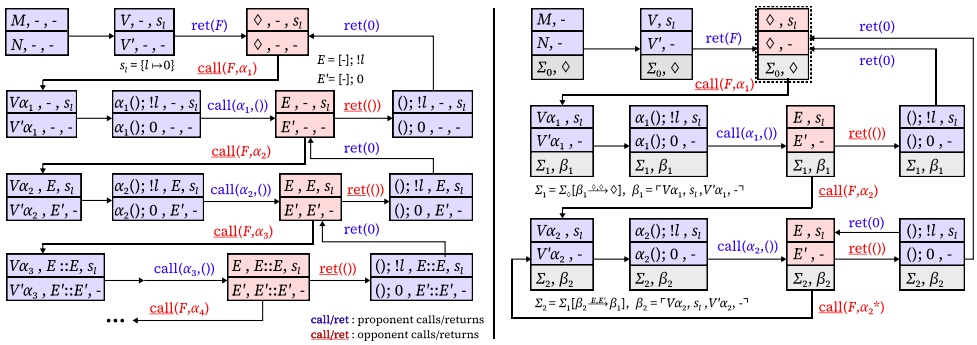


Figure 1. NF bisimulations for terms M and N (Thm. 2); standard/stacked (left) and pushdown/stackless (right). We use "$\diamond$" to denote the top-level continuation and entry point, and set Σ_0 to be the empty continuation graph. We write "-" for the empty store and continuation stack.

of Thm. 1 and how its NF bisimulation game [12, 41] becomes infinite because of nested context calls. For the next example, and to prepare for the developments in the main body of the paper, we shall follow more closely the style of presentation in [41] and describe the interactions between a term and its context, and the ensuing LTS, using terminology taken from game semantics [9, 35, 56].[1] In particular, we shall refer to the examined term as the *Proponent*, whereas its syntactic context will be the *Opponent*. The two parties, i.e. proponent and opponent, can interact by issuing *moves*, which are simply calls to functions provided by the opposite party and their corresponding returns. The bisimulation game is based on the matching of these moves.

Example 2. Consider the following equivalent terms, the NF bisimulation game of which is depicted in Fig. 1 (left).

$M =$ `let x = ref 0 in fun f -> f(); !x`

$N =$ `fun f -> f(); 0`

$V =$ `fun f -> f(); !l (for location l)`

$V' =$ `fun f -> f(); 0`

The game involves pairs of configurations[2] of the form (Φ, K, s), where Φ is either a term of the language (in configurations where proponent plays next) or a continuation (when opponent plays next), K is a continuation stack, and s is a local store. Initially, proponent returns an abstract function F representing respectively the functions V and V' (move $\mathsf{ret}(F)$). Next, the bisimulation game can engage in a series of moves as on the left below, where opponent repeatedly calls F with (fresh) arguments $\alpha_1, \alpha_2, \ldots$,

$$\underline{\mathsf{call}}(F, \alpha_1)\ \mathsf{call}(\alpha_1, ())\ \underline{\mathsf{call}}(F, \alpha_2)\ \mathsf{call}(\alpha_2, ())\ \ldots \qquad K = E::E::\ldots,\ K' = E'::E'::\ldots$$

thus leading to unbounded continuation stacks K and K' respectively as on the right.

In Fig. 1 (right) we can see the PDNF bisimulation game.[3] We observe that now configurations are *stackless* pairs (Φ, s), and that we have incorporated an additional *environment* component (Σ, β). The latter is an over-approximation of the (combined) stack structure which records:

- The opponent call that is currently being evaluated, by means of an *entry point* β: this is simply the pair of configurations (C_1, C_2) that the call led to and, in this case, there is only one such pair with terms $V\alpha$ and $V'\alpha$ (and corresponding stores).
- The possible sequencings of entry points β, using a *continuation graph* Σ. Edges in Σ are of the form $\beta' \xrightarrow{\mathcal{E}_1, \mathcal{E}_2} \beta$ which denote that, starting from β, we are led to an opponent call with continuations $\mathcal{E}_1, \mathcal{E}_2$ and resulting entry point β'.

[1] The terms "proponent", "opponent" and "move" will be all the terminology we use from game semantics in this paper; the term "game" will almost exclusively refer to the bisimulation game, which is traditionally between *Challenger* and *Defender*.

[2] For expository reasons, the notation used here for configurations, their components and the LTS is a simplified version of the one used later on when these notions are formally defined.

[3] The loop transition at the bottom right, labelled $\underline{\mathsf{call}}(F, \alpha_2^*)$, represents a transition *for each fresh* α_2. This representation is informal and used here for economy to demonstrate finiteness. In fact, the pushdown NF bisimulation would not be finite but, instead, *orbit-finite* (cf. Thm. 25).

$$
\begin{array}{rrcl}
\mathsf{Loc}: & l,k & \quad \mathsf{Var}: \ x,y,z \quad \mathsf{Const}: \ c \quad \mathsf{ANam}: \ \alpha & \\
\mathsf{Type}: & T & ::= & \mathsf{bool} \mid \mathsf{int} \mid \mathsf{unit} \mid T \to T \mid T_1 * \ldots * T_n \\
\mathsf{Exp}: & e,M & ::= & v \mid (\vec{e}) \mid op(\vec{e}) \mid e\,e \mid \mathsf{if}\ e\ \mathsf{then}\ e\ \mathsf{else}\ e \mid \mathsf{ref}\ l = v\ \mathsf{in}\ e \mid\ !l \mid l := e \mid \mathsf{let}(\vec{x}) = e\ \mathsf{in}\ e \\
\mathsf{Val}: & u,v & ::= & c \mid x \mid \alpha_{T\to T} \mid \mathsf{fix} f_{T\to T}(x).e \mid (\vec{v}) \\
\mathsf{ECxt}: & E & ::= & [\cdot]_T \mid (\vec{v},E,\vec{e}) \mid op(\vec{v},E,\vec{e}) \mid E\,e \mid v\,E \mid l := E \mid \mathsf{if}\ E\ \mathsf{then}\ e\ \mathsf{else}\ e \mid \mathsf{let}(\vec{x}) = E\ \mathsf{in}\ e \\
\mathsf{Cxt}: & D & ::= & [\cdot]_{i,T} \mid e \mid (\vec{D}) \mid op(\vec{D}) \mid D\,D \mid l := D \mid \mathsf{if}\ D\ \mathsf{then}\ D\ \mathsf{else}\ D \mid \mathsf{fix} f_{T\to T}(x).D \\
& & & \mid \mathsf{ref}\ l = D\ \mathsf{in}\ D \mid \mathsf{let}(\vec{x}) = D\ \mathsf{in}\ D
\end{array}
$$

$$
\begin{array}{lll}
\langle s\,;op(\vec{c})\rangle & \hookrightarrow \langle s\,;w\rangle & \text{if } op^{\mathsf{arith}}(\vec{c}) = w \\
\langle s\,;(\mathsf{fix} f(x).e)\,v\rangle & \hookrightarrow \langle s\,;e[v/x][\mathsf{fix} f(x).e/f]\rangle & \\
\langle s\,;\mathsf{let}(\vec{x}) = (\vec{v})\ \mathsf{in}\ e\rangle & \hookrightarrow \langle s\,;e[\vec{v}/\vec{x}]\rangle & \\
\langle s\,;\mathsf{ref}\ l = v\ \mathsf{in}\ e\rangle & \hookrightarrow \langle s[l\mapsto v]\,;e\rangle & \text{if } l \notin \mathsf{dom}(s) \\
\langle s\,;!l\rangle & \hookrightarrow \langle s\,;v\rangle & \text{if } s(l) = v \\
\langle s\,;l := v\rangle & \hookrightarrow \langle s[l\mapsto v]\,;()\rangle & \\
\langle s\,;\mathsf{if}\ c\ \mathsf{then}\ e_1\ \mathsf{else}\ e_2\rangle & \hookrightarrow \langle s\,;e_i\rangle & \text{if } (c,i) \in \{(\mathsf{tt},1),(\mathsf{ff},2)\} \\
\langle s\,;E[e]\rangle & \to \langle s'\,;E[e']\rangle & \text{if } \langle s\,;e\rangle \hookrightarrow \langle s'\,;e'\rangle
\end{array}
$$

$\langle s\,;e\rangle \in \mathsf{Exp} \times \mathsf{St}$
$\mathsf{St} = \mathsf{Loc} \xrightarrow{\mathsf{fin}} \mathsf{Val}$

Figure 2. Syntax and reduction semantics of the language λ^{imp}.

We assume by convention that there is a top-level opponent call with $\beta = \diamond$ which starts the bisimulation game. We can see in Fig. 1 that every path in the graph on the left has a corresponding path in the graph on the right. In other words, PDNF bisimulation is sound. On the other hand, the graph on the right has infeasible paths. For example we can form a path from the initial vertex to the highlighted top-level one with trace:

$$\mathsf{ret}(F)\ \underline{\mathsf{call}}(F,\alpha_1)\ \mathsf{call}(\alpha_1,())\ \underline{\mathsf{call}}(F,\alpha_2)\ \mathsf{call}(\alpha_2,())\ \underline{\mathsf{ret}}(())\ \mathsf{ret}(0)$$

which breaks the stack discipline (we reach the top level while doing more pushes than pops). However, such spurious paths are harmless and do not affect completeness of our method: all pairs of configurations that are spuriously reached can also be reached by real paths. In fact, the (highlighted) vertex reached by the path above was already reached after the first move $\mathsf{ret}(F)$.

3 Language and Semantics

We work with λ^{imp}, a simply-typed call-by-value lambda calculus with local state [41]. The syntax and operational semantics are shown in Fig. 2. Expressions (Exp) include the standard lambda expressions with recursive functions ($\mathsf{fix} f(x).e$), together with location creation ($\mathsf{ref}\, l = v\, \mathsf{in}\, e$), dereferencing ($!l$), and assignment ($l := e$), as well as standard base type constants (c) and operations ($op(\vec{e})$). Locations are mapped to values, including function values, in a store (St). We write $\cdot$ for the empty store and let $\mathsf{fl}(X)$ denote the set of free locations in syntactic or semantic object X. Values consist of boolean, integer, and unit constants, functions and arbitrary length tuples of values. Functions consist of standard functions ($\mathsf{fix} f(x).e$) as well as *abstract* ones (α) sourced from a typed-indexed set of countably infinite sets of *abstract names* $\mathsf{ANam} = \biguplus_{T,T'} \mathsf{ANam}_{T\to T'}$. These correspond to environment (unknown) functions and are used in the open-term LTS used in NF bisimulation. Given an object X, we write $\mathsf{an}(X)$ for the set of abstract names appearing in X.

The language λ^{imp} is simply-typed with typing judgements of the form $\Delta;\Lambda \vdash e : T$, where Δ is a type environment (omitted when empty), Λ a store typing and T a type (Type). Abstract functions are explicitly typed in terms, and we assume that said typing is consistent within terms. The rules of the type system are standard and omitted here. We call an expression e *closed* when $\Lambda \vdash e : T$.

The reduction semantics is by small-step transitions between configurations containing a store and an expression, $\langle s\,;e\rangle \to \langle s'\,;e'\rangle$, defined using single-hole evaluation contexts (ECxt) over a base relation $\hookrightarrow$. Holes $[\cdot]_T$ are annotated with the type T of closed values they accept, which we may omit to lighten notation. Stores map locations to closed values; the latter are uniquely typed and, thus, each store s yields a store typing Λ_s. Beta substitution of x with v in e is written as $e[v/x]$. We write $\langle s\,;e\rangle \Downarrow$ to denote $\langle s\,;e\rangle \to^* \langle t\,;v\rangle$ for some t, v. We write $\vec{X}$ to mean a syntactic sequence, and assume standard syntactic sugar from the lambda calculus. In our examples we assume an ML-like syntax and implementation of the type system, which is also the concrete syntax of our tool (same syntax as that used in Hobbit [41]). We write $\bot$ for a diverging computation.

Contexts D contain multiple, non-uniquely indexed holes $[\cdot]_{i,T}$, where T is the type of value that can replace the hole (each index can have one associated type). A context is called *canonical* if its holes are indexed $1,\ldots,n$, for some n. Given a canonical context D and a sequence of typed expressions

$\Lambda \vdash \vec{e} : \vec{T}$, notation $D[\vec{e}]$ denotes the context D with each hole $[\cdot]_{i,T_i}$ replaced by e_i. We omit hole types and indices where possible. We assume the Barendregt convention for locations, thus replacing context holes avoids location capture (note ref is a binder). Standard contextual equivalence [52] follows.

Definition 3 (Contextual Equivalence)**.** Expressions $\vdash e_1 : T$ and $\vdash e_2 : T$ with $\mathsf{an}(e_1) = \mathsf{an}(e_2) = \emptyset$ are *contextually equivalent*, written as $e_1 \equiv e_2$, when for all contexts D such that $\vdash D[e_1] : \mathsf{unit}$ and $\vdash D[e_2] : \mathsf{unit}$ we have $\langle \cdot \,;\, D[e_1] \rangle \Downarrow$ iff $\langle \cdot \,;\, D[e_2] \rangle \Downarrow$.

We finally consider environments $\Gamma \in \mathbb{N} \xrightarrow{\mathsf{fin}} \mathsf{Val}$ which map natural numbers to closed values. The concatenation of two such environments Γ_1 and Γ_2, written Γ_1, Γ_2 is defined when $\mathsf{dom}(\Gamma_1) \cap \mathsf{dom}(\Gamma_2) = \emptyset$. We write $({}^{i_1}v_1, \ldots, {}^{i_n}v_n)$ for a concrete environment mapping $i_1, \ldots, i_n$ to $v_1, \ldots, v_n$, respectively. Environment Γ can be used to fill in holes of context D with matching indices; we refer to the result as $D[\Gamma]$. When indices are unimportant we omit them and treat Γ environments as lists.

Names and permutations. It is useful to introduce notation that allows us to easily reason on locations, abstract names and environment indices, which we collectively refer to as *names*:

$$\mathsf{Names} = \mathsf{Loc} \cup \mathsf{ANam} \cup \mathbb{N}$$

These appear in the syntax and semantics of our language in a *nominal* way: the identity of a given name is immaterial – what is relevant is how the name compares to other names in its environment. Technically speaking, our constructions are founded on *nominal sets* [24]. Below, we refer to elements in our syntax and semantics as *objects*.

Definition 4 (Permutations)**.** We consider permutations of store locations (π_l), abstract names (π_α, which are type-reserving) and environment indices (π_i), respectively. We combine these in permutations π of the form $\pi_l \uplus \pi_\alpha \uplus \pi_i$, which we compose as functions (e.g. we may write $\pi \circ \pi'$). We restrict our attention to *finitary* permutations π, i.e. such that the set $\mathsf{supp}(\pi) = \{x \in \mathsf{Names} \mid \pi(x) \neq x\}$ be finite. We let Perm be the set of all finitary permutations. Given names x, x' we write $(x\ x')$ for the permutation that swaps x with x' (and fixes all other names).

Given an object X and (finitary) permutation π, we write $\pi \cdot X$ for the result of *applying* π *on* X. The result of applying a permutation on an object X is done as expected, e.g. applying a permutation π_l to a store s, the former acts on both the domain and range of the latter. When applying a permutation π_i, we treat environment index i differently than other instances of the natural number i.

Definition 5 (Nominal set and orbit-finiteness)**.** Given an object X, its *support* $\mathsf{supp}(X)$ is the least $S \subseteq \mathsf{Names}$ such that permutations fixing all $x \in S$ also fix X:

$$\forall \pi \in \mathsf{Perm}.\ (\forall x \in S.\, \pi(x) = x) \implies \pi \cdot X = X.$$

We henceforth assume that all objects have finite support. X is called *equivariant* if $\mathsf{supp}(X) = \emptyset$, in which case $\pi \cdot X = X$ for all π. A set $\mathcal{X}$ of objects is called a *nominal set* if it is closed under permutation, i.e. $\pi \cdot X \in \mathcal{X}$ for all $\pi \in \mathsf{Perm}$ and $X \in \mathcal{X}$. Given $x \in \mathsf{Names}$, we say that x *is fresh for* X, and write $x \# X$, when $x \notin \mathsf{supp}(X)$.
Given object X, its *orbit* is defined by: $\mathsf{orb}(X) = \{\pi \cdot X \mid \pi \in \mathsf{Perm}\}$. Nominal set $\mathcal{X}$ is *orbit-finite* if its set of orbits $\{\mathsf{orb}(X) \mid X \in \mathcal{X}\}$ is finite.

Note that in finite objects (e.g. terms of λ^{imp}), the support of an object typically coincides with the set of free names featuring in it. In such a case, writing e.g. $\vec{\alpha} \# \vec{X}$ will stand for $\forall i, j.\, \alpha_i \notin \mathsf{an}(X_j)$. Orbit-finiteness is central in computability with nominal sets [14, 13] and can be seen as the analogue of finiteness in nominal sets.

4 Stacked LTS and NF Bisimulation

We next recall the LTS and NF bisimulation presented in [41]. As mentioned in Sec. 2, the LTS is based on game semantics and uses opponent and proponent call and return transitions: proponent transitions are the moves of an expression interacting with its context; opponent transitions are the moves of the context surrounding the expression. These transitions are over *proponent, opponent and divergence configurations*, respectively:

$$\langle \Gamma \,;\, K \,;\, s \,;\, e \rangle, \quad \langle \Gamma \,;\, K \,;\, s \,;\, \mathcal{E} \rangle \quad \text{and} \quad \langle \bot \rangle.$$

$$
\begin{array}{lll}
\text{PropCall} : \langle \Gamma\,;K\,;s\,;E[\alpha\,v]\rangle \xrightarrow{\mathsf{call}(\alpha,D)} \langle \Gamma,\Gamma'\,;K\,;s\,;E\rangle & \text{if} & (D,\Gamma') \in \mathsf{ulpatt}(v)\\
\text{PropRet} : \langle \Gamma\,;\mathcal{E},K\,;s\,;v\rangle \xrightarrow{\underline{\mathsf{ret}}(D)} \langle \Gamma,\Gamma'\,;K\,;s\,;\mathcal{E}\rangle & \text{if} & (D,\Gamma') \in \mathsf{ulpatt}(v)\\
\text{OpCall} : \langle \Gamma\,;K\,;s\,;\mathcal{E}\rangle \xrightarrow{\underline{\mathsf{call}}(i,D[\vec{\alpha}])} \langle \Gamma\,;\mathcal{E},K\,;s\,;e\rangle & \text{if} & \vec{\alpha}\,\#\,\Gamma,K,s,\mathcal{E} \wedge (D,\vec{\alpha}) \in \mathsf{ulpatt}(T)\\
 & & \quad \wedge\, \Lambda_s \vdash \Gamma(i) : T \to T' \wedge \Gamma(i)\,D[\vec{\alpha}] \succ e\\
\text{OpRet} : \langle \Gamma\,;K\,;s\,;E[\cdot]_T\rangle \xrightarrow{\underline{\mathsf{ret}}(D[\vec{\alpha}])} \langle \Gamma\,;K\,;s\,;E[D[\vec{\alpha}]]\rangle & \text{if} & \vec{\alpha}\,\#\,\Gamma,K,s,E \wedge (D,\vec{\alpha}) \in \mathsf{ulpatt}(T)\\
\text{Tau} : \langle \Gamma\,;K\,;s\,;e\rangle \xrightarrow{\tau} \langle \Gamma\,;K\,;s'\,;e'\rangle & \text{if} & \langle s\,;e\rangle \to \langle s'\,;e'\rangle\\
\text{Response} : C \xrightarrow{\eta} \langle \bot\rangle & \text{if} & \eta \neq \tau \text{ and } C \not\xrightarrow{\eta} \text{ from other rules}
\end{array}
$$

Figure 3. The stacked Labelled Transition System (following [41]). We let $vu \succ e$ mean $e = \alpha u$ when $v = \alpha$; and $e = e'[u/x][\mathsf{fix}f(x).e'/f]$ when $v = \mathsf{fix}f(x).e'$.

$\langle \bot \rangle$ is a special configuration which is used in order to represent expressions that cannot perform given transitions (cf. Remark 9). In other configurations:

- Γ is an environment indexing proponent functions known to opponent;
- K is a stack of *continuations* $\mathcal{E}$, created by opponent calls; a continuation is either an evaluation context E or the constant $\diamond$ (for the *top-level*, empty continuation);
- s is the store containing proponent locations;
- $\mathcal{E}$ is a continuation, and is either the most recent evaluation context E or $\diamond$;
- e is the expression reduced in proponent configurations.

Given a configuration C, we write $C.\Gamma$ ($C.K$, etc.) for the first (second, etc.) component of C; if $C = \langle \bot \rangle$ then by convention $C._ = \bot$. We shall use Φ to range over $\mathcal{E}$ and e.

Compared to [41], we have made minor technical modifications in the structure of configurations to ensure uniformity between this and the stackless LTS of the next section. In particular we (1) drop sets of abstract names from configurations, replacing them with a mutual freshness condition for new abstract names in the bisimulation (Thm. 8); (2) separate the most recent evaluation context from those stacked in K in opponent configurations. To ensure that $\mathcal{E} = \diamond$ corresponds to a top-level configuration we require that opponent configurations $\langle \Gamma\,;K\,;s\,;\mathcal{E}\rangle$ satisfy the condition on the left below, while the push operation on stacks K is defined as on the right.

$$
\mathcal{E} = \diamond \iff K = \cdot \qquad\Big|\qquad \mathcal{E},K = \begin{cases} E,K & \text{if } \mathcal{E} = E\\ \cdot & \text{if } \mathcal{E} = \diamond \text{ and } K = \cdot\\ \text{undefined} & \text{otherwise}\end{cases}
$$

The LTS uses *moves* of the forms: $\eta ::= \underline{\mathsf{call}}(i, D[\vec{\alpha}]) \mid \underline{\mathsf{ret}}(D[\vec{\alpha}]) \mid \mathsf{call}(\alpha, D) \mid \mathsf{ret}(D)$. Underlined moves are *opponent moves*, and the rest are *proponent moves*. Contexts D are picked from the following restricted grammar (of values with higher-order holes):

$$D^\bullet ::= c \mid [\cdot]_{i,T\to T} \mid (\vec{D_\bullet})$$

Given such a context $D^\bullet$, we can derive its *hole signature* $\mathsf{sig}(D^\bullet) \in \mathbb{N}^*$ setting:

$$\mathsf{sig}(c) = \varepsilon, \quad \mathsf{sig}([\cdot]_{i,T}) = i, \quad \mathsf{sig}((D_1^\bullet,\ldots,D_n^\bullet)) = \mathsf{sig}(D_1^\bullet),\ldots,\mathsf{sig}(D_n^\bullet).$$

We stipulate that $\mathsf{sig}(D^\bullet)$ must be a non-repeating sequence (i.e. every hole appears exactly once). Given a value v, we can extract its *ultimate pattern* [49], which is a pair $(D^\bullet,\Gamma)$, and extend ulpatt to types through the use of abstract function names:

$$
\begin{array}{rcl}
(D^\bullet,\Gamma) \in \mathsf{ulpatt}(v) & \iff & v = D^\bullet[\Gamma] \wedge \mathsf{dom}(\Gamma) = \{i \mid i \in \mathsf{sig}(D^\bullet)\}\\
(D^\bullet,\Gamma) \in \mathsf{ulpatt}(T) & \iff & \vdash D^\bullet[\Gamma] : T \wedge \Gamma : \{i \mid i \in \mathsf{sig}(D^\bullet)\} \to \mathsf{ANam}
\end{array}
$$

In the latter case, we write $(D^\bullet,\Gamma)$ simply as $(D^\bullet,\vec{\alpha})$, where $\vec{\alpha} = \Gamma(i_1),\ldots,\Gamma(i_k)$ and $i_1,\ldots,i_k = \mathsf{sig}(D^\bullet)$. For economy, we will henceforth denote these contexts by D.

Definition 6 (Stacked LTS)**.** The LTS is defined by the rules in Fig. 3. We write $C \overset{\eta}{\mapsto} C'$ if $C \xrightarrow{\eta} C'$ without using the Response rule. We write $C \downarrow$ if $C = \langle \Gamma\,;\cdot\,;s\,;\diamond\rangle$.

We next introduce a notion of boundedness on terms that examines the sizes of all their possible descendant configurations in their LTS, ignoring stacks K.

Definition 7. Define a size function $\|\cdot\|$ for expressions inductively as:

$$\|c_{\mathsf{int}}\| = |c| + 1 \qquad \|x\| = 1 \qquad \|\alpha_{T\to T}\| = 1 \qquad \|\mathsf{fix} f_{T\to T}(x).e\| = 1 + \|e\|$$
$$\|c_T\| = 1\,(T \neq \mathsf{int}) \qquad \|!l\| = 1 \qquad \|l := e\| = 1 + \|e\| \qquad \|\mathsf{ref}\, l = v\, \mathsf{in}\, e\| = 1 + \|v\| + \|e\|$$

and $\|\kappa(e_1, \ldots, e_n)\| = 1 + \|e_1\| + \cdots + \|e_n\|$ for all other n-ary syntactic constructs κ. Extend this to continuations by $\|E\| = \|E[x]\|$ and $\|\diamond\| = 1$, and to configurations by:

$$\|\langle\bot\rangle\| = 1 \qquad \|\langle\Gamma\,;K\,;s\,;\Phi\rangle\| = \max\left(\sum\nolimits_{i\in\mathsf{dom}(\Gamma)} \|\Gamma(i)\|, \sum\nolimits_{l\in\mathsf{dom}(s)} \|s(l)\|, \|\Phi\|\right)$$

Call expression $\vdash e : T$ *context-free with bound* k if the set $\{\|C\| \mid \exists t.\ \langle\cdot\,;\cdot\,;\cdot\,;e\rangle \xrightarrow{t}{}^{*} C\}$ is upper-bounded by $k \in \mathbb{N}$. Let e be *context-free* if it is context-free with some bound k.

Thus, an expression is context-free when its stacked LTS can be represented as a nominal pushdown system [16, 54]. We shall show equivalence is decidable for these expressions, using the PDNF bisimulation in the following section.

We next present NF bisimulation. We write that *move* η *introduces* $\vec{\alpha}$ if η is a proponent move and $\vec{\alpha}$ is empty, or $\eta \in \{\underline{\mathsf{call}}(i, D[\vec{\alpha}]), \underline{\mathsf{ret}}(D[\vec{\alpha}])\}$ for some i, D. Moreover, $\overset{\eta}{\Longrightarrow}$ means $\xrightarrow{\tau}{}^{*}$, when $\eta = \tau$; and $\overset{\tau}{\Longrightarrow}\xrightarrow{\eta}\overset{\tau}{\Longrightarrow}$ otherwise.

Definition 8 (NF Bisimulation)**.** Configurations C_1, C_2 are called *compatible* whenever $\langle\bot\rangle \in \{C_1, C_2\}$, or C_1, C_2 have same polarity and $\mathsf{dom}(C_1.\Gamma) = \mathsf{dom}(C_2.\Gamma)$. Relation $\mathcal{R}$ between compatible configurations is a *weak simulation* when for all $C_1\ \mathcal{R}\ C_2$:

- if $C_1 \downarrow$ then $C_2 \downarrow$,
- if $C_1 \overset{\eta}{\mapsto} C_1'$ with η introducing $\vec{\alpha} \# C_2$ then $C_2 \overset{\eta}{\Longrightarrow} C_2'$ and $C_1'\ \mathcal{R}\ C_2'$.

If $\mathcal{R}$, $\mathcal{R}^{-1}$ are weak simulations then $\mathcal{R}$ is a *weak bisimulation*. Similarity ($\lessapprox$) and bisimilarity ($\approx$) are the largest weak simulation and bisimulation, respectively.

Remark 9. Following [41], any proponent configuration that cannot match a standard bisimulation transition challenge can trivially respond to the challenge by transitioning into $\langle\bot\rangle$ by the Response rule in Fig. 3. By the same rule, this configuration can trivially perform all non-τ transitions. While in *loc. cit.* there is an explicit termination transition from top-level, non-$\langle\bot\rangle$ configurations, here we choose instead to use the termination predicate $\downarrow$ to signify the end of a complete trace. To obtain determinacy, we also impose trivial transitions to $\langle\bot\rangle$ to take place only if same-labelled transitions are not possible by the other rules. The differences made in this section are inessential leaving the full abstraction result of [41] unaffected.

Definition 10 (NF Bisimilar Expressions)**.** Expressions $\vdash e_1 : T$ and $\vdash e_2 : T$ with $\mathsf{an}(e_1) = \mathsf{an}(e_2) = \emptyset$ are *NF bisimilar*, written $e_1 \approx e_2$, when $\langle\cdot\,;\cdot\,;\cdot\,;e_1\rangle \approx \langle\cdot\,;\cdot\,;\cdot\,;e_2\rangle$.

Theorem 11 (Full abstraction [41])**.** *$e_1 \approx e_2$ iff $e_1 \equiv e_2$.* □

Finally, similarity is a nominal set and closed under τ-transitions (cf. [41]).

Lemma 12. *Given $C_1 \lessapprox C_2$, if $C_i \overset{\tau}{\Longrightarrow} C_i'$ or $C_i' \overset{\tau}{\Longrightarrow} C_i$ (for $i = 1, 2$) then $C_1' \lessapprox C_2'$. Moreover, for all $\pi \in \mathsf{Perm}$, $\pi \cdot C_1 \lessapprox \pi \cdot C_2$.* □

5 Stackless LTS and Pushdown NF Bisimulation

5.1 The Stackless LTS In the LTS of Fig. 3 the stack is manipulated solely by rules OpCall (push $\mathcal{E}$ on the stack) and PropRet (pop last $\mathcal{E}$). Thus, in a bisimulation game where both component configurations are not blocked (i.e. not $\langle\bot\rangle$), the stack operations of the two components are synchronised. For instance, if we currently are in configuration pair (C_1, C_2) and a challenge is made in C_1 that pushes some $\mathcal{E}_1$ on $C_1.K$, then the response in C_2 must push some $\mathcal{E}_2$ on $C_2.K$. The bisimulation game can thus be seen as using a single stack of pairs $(\mathcal{E}_1, \mathcal{E}_2)$. This in turn allows us to remove the stack component from configurations and attach it to bisimulations as an *environment* component (which we can then apply abstractions on). This is the intuition behind the stackless LTS that we present next.

We start off with stackless configurations, which will be triples of the forms:

$$\text{Proponent: } (\!|\Gamma\,;s\,;e|\!)\,; \quad \text{Opponent: } (\!|\Gamma\,;s\,;\mathcal{E}|\!) \text{ or } (\!|\Gamma\,;s\,;\chi|\!)\,; \quad \text{Divergence: } (\!|\bot|\!)\,.$$

Here the constant χ stands for an unspecified continuation and it is used merely as a placeholder so that we can later apply a substitution of the form $[\mathcal{E}/\chi]$. This will become clearer in Thm. 21; for now we can think that a configuration of the form $(\!|\Gamma\,;s\,;\chi|\!)$ may silently reduce to $(\!|\Gamma\,;s\,;\mathcal{E}|\!)$ for selected previously encountered continuations $\mathcal{E}$.

Definition 13 (Stackless LTS)**.** The stackless LTS has the exact same rules as those in Fig. 3, with the exception that configurations are now stackless, K is dropped from the side conditions, and rules OpCall and PropRet have the following transitions while maintaining the same side-conditions (see Sec. A):

$$(\!|\Gamma\,;s\,;\mathcal{E}|\!) \xrightarrow{\mathsf{call}(i,D[\vec{\alpha}])} (\!|\Gamma\,;s\,;e|\!) \qquad\qquad (\!|\Gamma\,;s\,;v|\!) \xrightarrow{\mathsf{ret}(D)} (\!|\Gamma,\Gamma'\,;s\,;\chi|\!)$$

We write $C \overset{\eta}{\mapsto} C'$ if $C \xrightarrow{\eta} C'$ without using the Response rule, and $C{\downarrow}$ if $C = (\!|\Gamma\,;s\,;\diamond|\!)$.

Example 14. Recall below the equivalent terms M and N from our introductory Thm. 2:

$M =$ `let x = ref 0 in` `fun f -> f(); !x`	$V_l =$ `fun f -> f(); !l` `(for location l)`	$e_{l,\alpha} =$ `α(); !l` `(for abstract name α)`
$N =$ `fun f -> f(); 0`	$V' =$ `fun f -> f(); 0`	$e'_\alpha =$ `α(); 0`

Their stackless LTS's include transitions:

$$(\!|\cdot\,;\cdot\,;M|\!) \xrightarrow{\tau}{}^{*} (\!|\cdot\,;s_l\,;V_l|\!) \xrightarrow{\mathsf{ret}([\cdot])} (\!|\Gamma_l\,;s_l\,;\chi|\!) = C_\chi \text{ with } \Gamma_l = {}^1V_l,\ s_l = \{l \mapsto 0\}$$

$$C_\chi[\diamond/\chi] = (\!|\Gamma_l\,;s_l\,;\diamond|\!) \xrightarrow{\mathsf{call}(1,\alpha_1)} (\!|\Gamma_l\,;s_l\,;e_{l,\alpha_1}|\!) \xrightarrow{\mathsf{call}(\alpha_1,())} (\!|\Gamma_l\,;s_l\,;E_l|\!) \text{ with } E_l = [\cdot];!l$$

$$(\!|\Gamma_l\,;s_l\,;E_l|\!) \begin{cases} \xrightarrow{\mathsf{ret}(())} (\!|\Gamma_l\,;s_l\,;E_l[()]|\!) \xrightarrow{\tau}{}^{*} \xrightarrow{\mathsf{ret}(0)} (\!|\Gamma_l\,;s_l\,;\chi|\!) \\ \xrightarrow{\mathsf{call}(1,\alpha_2)} (\!|\Gamma_l\,;s_l\,;e_{l,\alpha_2}|\!) \xrightarrow{\mathsf{call}(\alpha_2,())} (\!|\Gamma_l\,;s_l\,;E_l|\!) \end{cases}$$

$$(\!|\cdot\,;\cdot\,;N|\!) \xrightarrow{\tau}{}^{*} (\!|\cdot\,;\cdot\,;V'|\!) \xrightarrow{\mathsf{ret}([\cdot])} (\!|\Gamma'\,;\cdot\,;\chi|\!) = C'_\chi \text{ with } \Gamma' = {}^1V'$$

$$C'_\chi[\diamond/\chi] = (\!|\Gamma'\,;\cdot\,;\diamond|\!) \xrightarrow{\mathsf{call}(1,\alpha_1)} (\!|\Gamma'\,;\cdot\,;e'_{\alpha_1}|\!) \xrightarrow{\mathsf{call}(\alpha_1,())} (\!|\Gamma'\,;\cdot\,;E'|\!) \text{ with } E' = [\cdot];0$$

$$(\!|\Gamma'\,;\cdot\,;E'|\!) \begin{cases} \xrightarrow{\mathsf{ret}(())} (\!|\Gamma'\,;\cdot\,;E'[()]|\!) \xrightarrow{\tau}{}^{*} \xrightarrow{\mathsf{ret}(0)} (\!|\Gamma'\,;\cdot\,;\chi|\!) \\ \xrightarrow{\mathsf{call}(1,\alpha_2)} (\!|\Gamma'\,;\cdot\,;e'_{\alpha_2}|\!) \xrightarrow{\mathsf{call}(\alpha_2,())} (\!|\Gamma'\,;\cdot\,;E'|\!) \end{cases}$$

Note that the above are not complete descriptions of the LTS's as we have not explored all possible continuations that can instantiate the abstract continuation χ. As we shall see next, in fact, only a restricted set of relevant instantiations needs to be considered.

5.2 Pushdown Normal Form Bisimulation Having disengaged stacks from configurations, we need to engineer bisimulations to account for stack discipline during the bisimulation game. Following [28], we employ an abstraction which makes part of *saturation algorithms* [15, 23] that finitise pushdown systems with a finite number of control states. We abstract stacks by so-called *continuation graphs*, which consist of:

- **Vertices (β, etc.):** these represent pairs of Proponent function entry points, that is, Proponent configuration pairs (C_1, C_2) reached after an Opponent call (i.e. after a push). We shall write $\beta = \ulcorner C_1, C_2 \urcorner$.
- **Edges ($\beta' \xrightarrow{\mathcal{E}_1,\mathcal{E}_2} \beta$):** these are directed and labelled with pairs of continuations, and can be seen as procedure summaries. An edge $\ulcorner C'_1, C'_2 \urcorner \xrightarrow{(\mathcal{E}_1,\mathcal{E}_2)} \ulcorner C_1, C_2 \urcorner$ means that playing the bisimulation game from (C_1, C_2) we can reach a pair of Opponent configurations with evaluation contexts $\mathcal{E}_1, \mathcal{E}_2$ respectively from which, in turn, we can fire OpCall transitions and reach (C'_1, C'_2).

Our bisimulation game will now involve tuples of the form $(C_1, C_2, \Sigma, \beta)$ including a pair of stackless configurations along with a continuation graph Σ and an encoding β of the pair of entry points that is currently evaluated. At each OpCall step in the bisimulation game, containing transitions $C_i \xrightarrow{\mathsf{call}(i,D[\vec{\alpha}])} C'_i$ (for $i = 1, 2$), we shall extend Σ by adding a new edge (if not already present) and update the current β:

$$(C_1, C_2, \Sigma, \beta) \xrightarrow{(\mathsf{call}(i,D[\vec{\alpha}]))} (C'_1, C'_2, \Sigma[\beta' \xrightarrow{\mathcal{E}_1,\mathcal{E}_2} \beta], \beta')$$

where $\beta' = \ulcorner C_1', C_2' \urcorner$ and $\mathcal{E}_i = C_i.\mathcal{E}$ (for $i = 1, 2$).

The above scenario accounts for points in the bisimulation game where a push operation needs to be performed. On the other hand, for pop operations, we need to turn to PropRet steps. Given a current tuple $(\hat{C}_1, \hat{C}_2, \Sigma, \beta')$ with $\beta' = \ulcorner C_1', C_2' \urcorner$, and assuming that $\hat{C}_1, \hat{C}_2$ are about to perform $\hat{C}_i \xrightarrow{\mathsf{ret}(D)} C_i$ (for $i = 1, 2$), the set

$$\Sigma(\beta') = \{\, (\mathcal{E}_1, \mathcal{E}_2, \beta) \mid \beta' \xrightarrow{\mathcal{E}_1, \mathcal{E}_2} \beta \,\}$$

contains all the push operations that have led to the pair of entry points (C_1', C_2') that we are currently evaluating. Though only one of them is the operation that has led to the current pair $(\hat{C}_1, \hat{C}_2)$, it is sound for the bisimulation game to pop back *any of the operations* in $\Sigma(\beta')$. Thus, we can have:

$$(\hat{C}_1, \hat{C}_2, \Sigma, \beta') \xmapsto{(\mathsf{ret}(D))} (C_1[\mathcal{E}_1/\chi], C_2[\mathcal{E}_2/\chi], \Sigma, \beta)$$

for any $(\mathcal{E}_1, \mathcal{E}_2, \beta) \in \Sigma(\beta')$. We next make concrete this high-level presentation by formally introducing continuation graphs and defining the ensuing notion of bisimulation.

To simplify presentation, we will abuse notation and utilise X to stand for some object X or $\bot$ (for X an environment Γ, a store s, a continuation $\mathcal{E}$ or a stack K). The constant $\bot$ denotes a dummy component of a divergent configuration. For continuations and stacks in particular, we extend the push operation by setting:

$$\mathcal{E}, K = \begin{cases} \mathcal{E}, K & \text{if } \mathcal{E} \neq \bot \text{ and } K \neq \bot \text{ and } \mathcal{E}, K \text{ is defined according to Sec. 4} \\ \bot & \text{if } \mathcal{E} = \bot \\ \text{undefined} & \text{otherwise} \end{cases}$$

Below we write Kont for the set of continuations $\mathcal{E}$, i.e. $\mathsf{Kont} = \mathsf{ECxt} \uplus \{\diamond, \bot\}$.

Definition 15. Define *entry points* and *continuation graphs* as follows:

$$\mathsf{EPoint} \ni \beta ::= \diamond \mid \ulcorner C_1, C_2 \urcorner$$

$$\mathsf{CGrph} \ni \Sigma \subseteq^{\neq \emptyset}_{\text{fin.orb.}} \mathsf{EPoint} \times \mathsf{Kont} \times \mathsf{Kont} \times \mathsf{EPoint}$$

where C_1, C_2 are non-Opponent configurations, $\{C_1, C_2\} \neq \{(\!|\bot|\!)\}$, and each Σ must satisfy the conditions (note we write $\beta' \xrightarrow{\mathcal{E}_1, \mathcal{E}_2}_\Sigma \beta$ for $(\beta', \mathcal{E}_1, \mathcal{E}_2, \beta) \in \Sigma$ and let $\mathsf{dom}(\Sigma)$ contain all such β'):

- *Reachability.* For all $\beta \in \mathsf{dom}(\Sigma)$ there are evaluation stacks K_1, K_2 such that $\beta \xrightarrow{K_1, K_2}{}^*_\Sigma \diamond$, where $\to^*_\Sigma$ is the transitive closure of $\to_\Sigma$. In particular, $\cdot \to^*_\Sigma \diamond$ is defined inductively by:

$$\frac{}{\diamond \xrightarrow{\cdot,\cdot}{}^*_\Sigma \diamond} \qquad \frac{\beta \xrightarrow{K_1, K_2}{}^*_\Sigma \diamond \qquad \beta' \xrightarrow{\mathcal{E}_1, \mathcal{E}_2}_\Sigma \beta}{\beta' \xrightarrow{(\mathcal{E}_1, K_1), (\mathcal{E}_2, K_2)}{}^*_\Sigma \diamond}$$

- *Top and Divergence.* For all $\beta' \xrightarrow{\mathcal{E}_1, \mathcal{E}_2}_\Sigma \beta$ and $j \in \{1, 2\}$:

$$\diamond \in \mathsf{dom}(\Sigma) \wedge (\beta' = \diamond \implies \beta = \diamond) \wedge (\beta = \diamond \iff \mathcal{E}_j = \diamond)$$
$$\wedge\, (\beta'.j = \bot \iff \mathcal{E}_j = \bot) \wedge (\beta.j = \bot \implies \mathcal{E}_j = \bot)$$

  where we write $\beta.1 = \bot$ just if $\beta = \ulcorner (\!|\bot|\!), C \urcorner$ (and similarly for $\beta.2 = \bot$).
- *Nominal closure.* For all $\beta' \xrightarrow{\mathcal{E}_1, \mathcal{E}_2}_\Sigma \beta$ and permutations π, $\pi \cdot \beta' \xrightarrow{\pi \cdot \mathcal{E}_1, \pi \cdot \mathcal{E}_2}_\Sigma \pi \cdot \beta$.

Remark 16. Given any continuation graph Σ, the top condition along with the fact that Σ cannot be empty imply that Σ contains the loop:

$$\Sigma_\diamond = \quad \diamond \circlearrowleft \diamond, \diamond$$

which is itself a continuation graph. Note also that the divergence condition ensures that, for any edge $\cdot \xrightarrow{\mathcal{E}_1, \mathcal{E}_2}_\Sigma \cdot$, we cannot have $\mathcal{E}_1 = \mathcal{E}_2 = \bot$.

Remark 17. It is worth commenting on the nominal closure condition. The condition imposes that continuation graphs be closed under permutations so that e.g. extending a graph with an edge $\beta' \xrightarrow{\mathcal{E}_1, \mathcal{E}_2}_\Sigma \beta$ in fact extends it with the whole orbit $\mathsf{orb}(\beta' \xrightarrow{\mathcal{E}_1, \mathcal{E}_2}_\Sigma \beta)$. The addition of elements of the orbit is sound and complete as, the behaviour that led β to reach β' while pushing $(\mathcal{E}_1, \mathcal{E}_2)$, can also be used by $\pi \cdot \beta$ to reach the corresponding $\pi \cdot \beta'$ while pushing $(\pi \cdot \mathcal{E}_1, \pi \cdot \mathcal{E}_2)$, for any permutation π. The latter allows us to saturate continuation graphs in a finite amount of steps in examples like

the ones we saw in Thm.(s) 1 and 2. More foundationally, the saturation of Σ's under permutation amounts to treating the pushdown stack (of pairs $(\mathcal{E}_1, \mathcal{E}_2)$) and its abstraction Σ *nominally*, i.e. by means of representatives. In effect, we are working with pushdown nominal automata [16, 54], and that bring about decidability for context-free expressions (Thm. 24).

We next show that Σ can only produce valid, compatible stacks (cf. Sec. C).

Lemma 18. *For any* $\beta \xrightarrow{K_1,K_2}{}^*_\Sigma \diamond$, *we have that* K_1, K_2 *are defined and:*
- $\beta = \diamond$ *and* $K_1 = K_2 = \cdot$, *or*
- $\beta.1, \beta.2 \neq \bot$ *and* $|K_1| = |K_2|$, *or*
- $\exists j \in \{1,2\}.\ \beta.j = \bot \wedge K_j = \bot \neq K_{3-j}$

where $|K|$ *is the length of* K *(if* $K \neq \bot$*).* □

Continuation graphs will be updated in the bisimulation game using the following two operations. By definition, continuation graph updates that satisfy the *Top and Divergence* condition produce valid continuation graphs.

Definition 19. We can *extend* Σ with an edge $\beta' \xrightarrow{\mathcal{E}_1,\mathcal{E}_2} \beta$ or *restrict* it to its reachable subgraph starting from $\beta \in \mathsf{dom}(\Sigma)$ as follows (note $\pi \cdot (x \xrightarrow{\eta} y) \stackrel{\text{def}}{=} (\pi \cdot x) \xrightarrow{\pi\cdot\eta} (\pi \cdot y)$):

$$\Sigma[\beta' \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}_\Sigma \beta] = \Sigma \cup \{\pi \cdot (\beta' \xrightarrow{\mathcal{E}_1,\mathcal{E}_2} \beta) \mid \pi \in \mathsf{Perm}\}$$

$$\Sigma@\beta = \{\pi \cdot (\beta' \xrightarrow{\mathcal{E}_1,\mathcal{E}_2} \beta'') \mid \pi \in \mathsf{Perm} \wedge \exists K_1, K_2.\ \beta \xrightarrow{K_1,K_2}{}^*_\Sigma \beta'\}$$

Moreover, entry points and continuation graphs can be left-right inverted as follows:

$$\diamond^{-1} = \diamond, \quad \ulcorner C_1, C_2 \urcorner^{-1} = \ulcorner C_2, C_1 \urcorner, \quad \Sigma^{-1} = \{\beta'^{-1} \xrightarrow{\mathcal{E}_2,\mathcal{E}_1} \beta^{-1} \mid \beta' \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}_\Sigma \beta\}.$$

Bisimulations for stackless configurations will involve tuples of the form defined next.

Definition 20 (Compatible (bi)simulation tuples)**.** Let us call configurations C_1, C_2 *compatible* whenever $\{(\!|\bot|\!)\} \subsetneq \{C_1, C_2\}$, or C_1, C_2 have the same polarity, $\mathsf{dom}(C_1.\Gamma) = \mathsf{dom}(C_2.\Gamma)$ and $(C_1.\mathcal{E} = \diamond \iff C_2.\mathcal{E} = \diamond)$.
Tuple $(C_1, C_2, \Sigma, \beta)$ is compatible if C_1, C_2 compatible, $\Sigma@\beta = \Sigma$ and for $j \in \{1,2\}$:
- $(\bot)$ if $\beta.j = \bot$ then $C_j = (\!|\bot|\!)$;
- $(\diamond)$ if $\beta = \diamond$ then $C_j.\mathcal{E} = \diamond$ or $\mathsf{an}(C_j.e) = \emptyset$; and if $C_j.\mathcal{E} = \diamond$ then $\beta = \diamond$.

Condition $(\diamond)$ above says that a top-level β $(\diamond)$ is only allowed in opponent configurations with top-level continuation $(\diamond)$ and in initial proponent configurations evaluating the top-level term (e.g. $(\!|\cdot;\cdot;M|\!)$ in Thm. 14); dually, any top-level continuation $(\diamond)$ needs a top-level β $(\diamond)$. We now give the definition of (bi)simulation for the stackless LTS.

Definition 21 (PDNF Bisimulation)**.** A relation $\mathcal{R}$ with elements of the form $(C_1, C_2, \Sigma, \beta)$, and membership thereof denoted $C_1\ \mathcal{R}_{\Sigma,\beta}\ C_2$, is called *weak simulation* when for all $C_1\ \mathcal{R}_{\Sigma,\beta}\ C_2$ we have $(C_1, C_2, \Sigma, \beta)$ compatible and:

0. if $C_1 \downarrow$ then $C_2 \downarrow$
(1) if $C_1 \xrightarrow{\mathsf{ret}(D[\vec{\alpha}])} C_1'$ with $\vec{\alpha} \# C_2, \beta$ then $C_2 \xrightarrow{\mathsf{ret}(D[\vec{\alpha}])} C_2'$ such that $C_1'\ \mathcal{R}_{\Sigma,\beta}\ C_2'$
(2) if $C_1 \xmapsto{\eta} C_1'$ then $C_2 \xRightarrow{\eta} C_2'$ and $C_1'\ \mathcal{R}_{\Sigma,\beta}\ C_2'$, for $\eta \in \{\tau, \mathsf{call}(\alpha, D)\}$
(3) if $C_1 \xrightarrow{\mathsf{call}(i,D[\vec{\alpha}])} C_1'$ with $\vec{\alpha} \# C_2, \beta$ then $C_2 \xrightarrow{\mathsf{call}(i,D[\vec{\alpha}])} C_2'$ such that $C_1'\ \mathcal{R}_{\Sigma',\beta'}\ C_2'$ with $\beta' = \ulcorner C_1', C_2' \urcorner$ and $\Sigma' = \Sigma[\beta' \xrightarrow{C_1.\mathcal{E},C_2.\mathcal{E}} \beta]$
(4) if $C_1 \xrightarrow{\mathsf{ret}(D)} C_1'$ and $\beta \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}_\Sigma \beta'$ then $C_2 \xRightarrow{\mathsf{ret}(D)} C_2'$ and $C_1'[\mathcal{E}_1/\chi]\ \mathcal{R}_{\Sigma@\beta',\beta'}\ C_2'[\mathcal{E}_2/\chi]$.

Similarity ($\lessapprox$) is the largest weak simulation. Relation $\mathcal{R}$ is a *weak bisimulation* when $\mathcal{R}$ and $\mathcal{R}^{-1}$ are weak simulations, where $R^{-1} = \{(C_2, C_1, \Sigma^{-1}, \beta^{-1}) \mid (C_1, C_2, \Sigma, \beta) \in R\}$. Bisimilarity ($\approx$) is the largest weak bisimulation. Expressions $\vdash e_1, e_2 : T$ with $\mathsf{an}(e_i) = \emptyset$ are *PDNF bisimilar*, written $e_1 \approx_{\mathsf{PD}} e_2$, when $(\!|\cdot;\cdot;e_1|\!) \approx_{\Sigma_\diamond,\diamond} (\!|\cdot;\cdot;e_2|\!)$.

Remark 22. The definition above assumes that Σ is well-defined, i.e. it satisfies the conditions of Thm. 15 and tuples satisfy the compatibility conditions of Thm. 20. These conditions are preserved by the bisimulation, thus making them merely initial conditions for the construction of the relations. In particular Σ is well-defined when extending Σ (case 3) as β' is well-defined and:

- Reachability and nominal closure are preserved by construction.
- Top follows from the fact that $C_1.\mathcal{E} = \diamond$ iff $\beta = \diamond$ (by definition) and $(C_1.\mathcal{E} = \diamond \wedge C_2.\mathcal{E} \neq \diamond)$ is not possible due to compatibility.
- For divergence, given that C_1 does not diverge, it suffices to check the conditions for $j = 2$. For the first one, we need to verify that we cannot have $C_2' = (\!|\bot|\!) \neq C_2$, which is indeed the case as C_1 can make the move $\underline{\mathsf{call}}(i, D[\vec{\alpha}])$ and C_1, C_2 are compatible. For the second condition, if $\beta.2 = \bot$ then by compatibility we have $C_2 = (\!|\bot|\!)$, and therefore $C_2.\mathcal{E} = \bot$.

On the other hand, by restricting Σ (case 4) we get a smaller graph with all entry points reachable from β', and its validity follows from the validity of Σ.
Furthermore, compatibility is preserved in the target tuple in each case. For each j, if $C_j = (\!|\bot|\!)$ then $C_j' = (\!|\bot|\!)$ (this is vacuously true for $j = 1$). Moreover, if $\beta = \diamond$ then:

- if C_j an opponent configuration then $C_j.\mathcal{E} = \diamond$ and $\underline{\mathsf{ret}}(D[\vec{\alpha}])$ is not possible;
- if C_j is proponent then $\mathsf{an}(C_j.e) = \emptyset$ and $\mathsf{call}(\alpha, D)$ is not possible;

thus, in either case, β is replaced by β' and the validity of Σ implies validity of the resulting tuple involving C_1', C_2'. Finally, if case 4 takes place and $C_j'[\mathcal{E}_j/\chi].\mathcal{E} = \diamond$ then $\mathcal{E}_j = \diamond$ and therefore $\beta' = \diamond$.

5.3 Decidability of PDNF Bisimulation We previously mentioned that PDNF bisimulation can be used to decide equivalence of context-free terms (Thm. 7). The first step in proving this is to show bounded PDNF bisimilarity decidable.

Given $k \in \mathbb{N}$ and edge $\beta' \xrightarrow{\mathcal{E}_1,\mathcal{E}_2} \beta$ (of Σ), we say that the edge is *k-bounded* if:

$$\max(\|\beta'\|, \|\mathcal{E}_1\|, \|\mathcal{E}_2\|, \|\beta\|) \leq k, \text{ where } \|\ulcorner C_1, C_2 \urcorner\| = \max(\|C_1\|, \|C_2\|) \text{ and } \|\diamond\| = 1.$$

Accordingly, tuple $(C_1, C_2, \Sigma, \beta)$ is k-bounded if all elements of Σ are k-bounded and $\max(\|C_1\|, \|C_2\|, \|\beta\|) \leq k$. Finally, candidate weak bisimulation relation $\mathcal{R}$ is k-bounded if all its elements are. Note below that $\pi \cdot \mathcal{R} = \{\pi \cdot x \mid x \in \mathcal{R}\}$.

Lemma 23. *If $\mathcal{R}$ is k-bounded and equivariant then it is orbit-finite.*

Proof. Note first that, for each $k \in \mathbb{N}$, the set of configurations with size at most k is orbit-finite. Similarly for the set of continuations with size at most k. Accordingly, since (in nominal sets [24]) orbit-finiteness is closed under cartesian products and equivariant subsets [13], the set of all k-bounded edges is also orbit-finite. As orbit-finiteness is also closed under equivariant powerset, the set of all k-bounded Σ's is also orbit-finite. Since orbit-finiteness is closed under cartesian products, disjoint union and equivariant subsets, any k-bounded equivariant candidate weak bisimulation is orbit-finite. □

Theorem 24. *Given e_1, e_2 context-free according to Thm. 7 with bound k, $e_1 \approx_{\mathsf{PD}} e_2$ iff $(\!|\cdot;\cdot;e_1|\!)\mathcal{R}_{\Sigma_\diamond,\diamond}(\!|\cdot;\cdot;e_2|\!)$ for some k-bounded equivariant weak bisimulation $\mathcal{R}$. Therefore, $\approx_{\mathsf{PD}}$ is decidable for context-free expressions.*

Proof. We observe that, since e_1, e_2 are context-free with bound k, in the bisimulation game starting from $((\!|\cdot;\cdot;e_1|\!), (\!|\cdot;\cdot;e_2|\!), \Sigma_\diamond, \diamond)$ we only reach k-bounded tuples $(C_1, C_2, \Sigma, \beta)$. This is due to the fact that C_1, C_2 , all configurations contained in β and in the vertices of Σ, and all continuations found in edges of Σ are all sourced from components of (stacked) configurations found in the set:

$$\{C \mid \exists i, t.\ \langle \cdot;\cdot;\cdot;e_i \rangle \xrightarrow{t} C\}$$

which, by assumption, have size at most k. Thus, if $e_1 \approx_{\mathsf{PD}} e_2$ then $(\!|\cdot;\cdot;e_1|\!)\mathcal{R}_{\Sigma_\diamond,\diamond}(\!|\cdot;\cdot;e_2|\!)$ with $\mathcal{R}$ being the restriction of $\approx_{\mathsf{PD}}$ to k-bounded elements.
Now, given context-free expressions e_1, e_2 with $\vdash e_1, e_2 : T$ and $\mathsf{an}(e_1, e_2) = \emptyset$, to decide whether $e_1 \approx_{\mathsf{PD}} e_2$ we pick a bound $k \in \mathbb{N}$ and try the weak bisimulation conditions on all k-bounded equivariant candidate weak bisimulation relations $\mathcal{R}$. The examination of all such relations is possible due to orbit-finiteness. If in our examination we are led to a tuple $(C_1, C_2, \Sigma, \beta)$ that is not k-bounded, we restart with $k = k + 1$. If a weak bisimulation is found, we Accept. If no weak bisimulation is found, we Reject. □

Example revisited We now show how PDNF bisimulation applies to the example from Sec. 2. To simplify presentation, and relying on determinacy and a simple up-to beta reduction technique (see Sec. F), we shall restrict our attention to bisimulation challenges of the form:

$$C_1 \xrightarrow{\tau}{}^* \xrightarrow{\eta} C_1'$$

where $\eta \neq \tau$, and only present the part of the bisimulation containing the corresponding configurations at the beginning and end of such transition sequences.

Example 25. Recall again the equivalent terms M and N from Thm. 2, as well as their reducts $V_l, V', e_{l,\alpha}, e'_\alpha$ from Thm. 14. We construct a PDNF bisimulation $\mathcal{R}$ by including:

$$\mathcal{R}_0 = \{(\!|\cdot\,;\cdot\,;M|\!), (\!|\cdot\,;\cdot\,;N|\!), \Sigma_\diamond, \diamond)\} \cup \mathsf{orb}(\{((\!|\Gamma_l\,;s_l\,;\diamond|\!), (\!|\Gamma'\,;\cdot\,;\diamond|\!), \Sigma_\diamond, \diamond)\})$$
$$\mathcal{R}_1 = \mathsf{orb}(\{((\!|\Gamma_l\,;s_l\,;e_{l,\alpha}|\!), (\!|\Gamma'\,;\cdot\,;e'_\alpha|\!), \Sigma_1, \beta_{l,\alpha}), ((\!|\Gamma_l\,;s_l\,;E_l|\!), (\!|\Gamma'\,;\cdot\,;E'|\!), \Sigma_1, \beta_{l,\alpha})\})$$
$$\mathcal{R}_2 = \mathsf{orb}(\{((\!|\Gamma_l\,;s_l\,;e_{l,\alpha'}|\!), (\!|\Gamma'\,;\cdot\,;e'_{\alpha'}|\!), \Sigma_2, \beta_{l,\alpha'}), ((\!|\Gamma_l\,;s_l\,;E_l|\!), (\!|\Gamma'\,;\cdot\,;E'|\!), \Sigma_2, \beta_{l,\alpha'})\})$$

for some $l \in \mathsf{Loc}$ and $\alpha \neq \alpha' \in \mathsf{ANam}$, with $s_l = \{l \mapsto 0\}$, $\beta_{l,\alpha} = \ulcorner(\Gamma_l, s_l, e_{l,\alpha}), (\Gamma', \cdot, e'_\alpha)\urcorner$ and:

$$\Gamma_l = {}^1V_l \quad E_l = [\cdot]; !l \quad \Sigma_1 = \Sigma_\diamond[\beta_{l,\alpha} \xrightarrow{\diamond,\diamond} \diamond] \qquad \begin{pmatrix} V_l = \lambda f.\ f(); !l & e_{l,\alpha} = \alpha(); !l \\ V' = \lambda f. f(); 0 & e'_\alpha = \alpha(); 0 \end{pmatrix}$$
$$\Gamma' = {}^1V' \quad E' = [\cdot]; 0 \quad \Sigma_2 = \Sigma_1[\beta_{l,\alpha'} \xrightarrow{E_l,E'} \beta_{l,\alpha}]$$

and taking $\mathcal{R} = \mathcal{R}_1 \cup \mathcal{R}_2 \cup \mathcal{R}_2$. Hence, $M \approx_{\mathsf{PD}} N$.

Note that in the example above the relation we build is infinite, due to the accumulation of edges e.g. in Σ_2:

$$\cdots \beta_{l,\alpha_i} \xrightarrow{E_l,E'} \cdots \beta_{l,\alpha_3} \xrightarrow{E_l,E'} \beta_{l,\alpha_2} \xrightarrow{E_l,E'} \beta_{l,\alpha_1} \xrightarrow{\diamond,\diamond} \diamond \quad \circlearrowleft \diamond,\diamond$$

Nonetheless, Σ_2 is orbit-finite, as it is the closure under permutation of this finite graph:

$$\beta_{l,\alpha_2} \xrightarrow{E_l,E'} \beta_{l,\alpha_1} \xrightarrow{\diamond,\diamond} \diamond \quad \circlearrowleft \diamond,\diamond$$

In fact, as seen by its definition, the bisimulation that we built above is also orbit-finite.

5.4 Soundness We next show that PDNF bisimulation is sound with respected to (standard) NF bisimulation. We will prove that if stackless configurations C_1, C_2 are related by a weak bisimulation $\mathcal{R}$ then we can construct a weak bisimulation $\tilde{\mathcal{R}}$ on stacked configurations containing C_1, C_2 with appropriate stacks attached.

Recall in Thm. 6 the stacked LTS. For notational convenience, in this and the next section, we shall denote the configurations of the stacked LTS by $\tilde{C}$ and variants (and C and variants is reserved for stackless). We move from stackless to stacked configurations by adding compatible stack components.

Definition 26. Given a configuration C and stack K, we set:

$$\widetilde{(C,K)} = \begin{cases} \langle \Gamma\,;K\,;s\,;e\rangle & \text{if } C = (\!|\Gamma\,;s\,;e|\!) \wedge K \neq \bot \\ \langle \Gamma\,;K\,;s\,;\mathcal{E}\rangle & \text{if } C = (\!|\Gamma\,;s\,;\mathcal{E}|\!) \wedge K \neq \bot \wedge (\mathcal{E} = \diamond \iff K = \cdot) \\ \langle \bot\rangle & \text{if } C = (\!|\bot|\!) \\ \text{undefined} & \text{otherwise} \end{cases}$$

Lemma 27. *For any $\beta \xrightarrow{K_1,K_2}{}^*_\Sigma \diamond$ and C_1, C_2, if $\beta = \ulcorner C_1, C_2\urcorner$ or $(C_1, C_2, \Sigma, \beta)$ compatible then $\widetilde{(C_1, K_1)}, \widetilde{(C_2, K_2)}$ are defined.* □

Soundness can be shown by the following result (cf. Sec. C).

Lemma 28 (Soundness)**.** *If $\mathcal{R}$ is a weak simulation then so is:*

$$\tilde{\mathcal{R}} = \{(\widetilde{(C_1, K_1)}, \widetilde{(C_2, K_2)}) \mid \exists \Sigma, \beta.\ C_1\ \mathcal{R}_{\Sigma,\beta}\ C_2 \wedge \beta \xrightarrow{K_1,K_2}{}^*_\Sigma \diamond\}$$

Moreover, if $\mathcal{R}$ is a weak bisimulation then so is $\tilde{\mathcal{R}}$. □

5.5 Completeness In order to derive a pushdown bisimulation $\mathcal{R}$ from a (standard) bisimulation $\tilde{\mathcal{R}}$, we shall define an LTS that follows the bisimulation game in the stackless LTS but is faithful to the stack discipline.

Definition 29. The *saturated simulation LTS* contains transitions of the form:

$$(C_1, C_2) \xrightarrow[\mathsf{SAT}]{\eta} (C'_1, C'_2)$$

where C_1, C_2 compatible and η is either ε or a pair $(\mathcal{E}_1, \mathcal{E}_2)$. The rules for $\xrightarrow[\mathsf{SAT}]{}$ are given in Fig. 4 (note use of stackless LTS). We call a continuation graph Σ *sat-connected* if whenever $\ulcorner C'_1, C'_2\urcorner \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}_\Sigma \ulcorner C_1, C_2\urcorner$ then $(C_1, C_2) \xrightarrow[\mathsf{SAT}]{\varepsilon} \cdot \xrightarrow[\mathsf{SAT}]{(\mathcal{E}_1,\mathcal{E}_2)} (C'_1, C'_2)$.

$$\frac{C_i \overset{\tau}{\mapsto} C_i' \qquad C_{3-i} = C_{3-i}'}{(C_1, C_2) \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1', C_2')}\ \text{Tau} \qquad \frac{C_1 \overset{\mathsf{call}(\alpha,D)}{\longmapsto} C_1' \qquad C_2 \xrightarrow{\mathsf{call}(\alpha,D)} C_2'}{(C_1, C_2) \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1', C_2')}\ \text{PropCall}$$

$$\frac{C_1 \overset{\mathsf{ret}(D[\vec{\alpha}])}{\longmapsto} C_1' \quad C_2 \xrightarrow{\mathsf{ret}(D[\vec{\alpha}])} C_2'}{(C_1, C_2) \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1', C_2')}\ \text{OpRet} \qquad \frac{C_1 \overset{\mathsf{call}(j,D[\vec{\alpha}])}{\longmapsto} C_i' \quad C_2 \xrightarrow{\mathsf{call}(j,D[\vec{\alpha}])} C_2'}{(C_1, C_2) \xrightarrow[\mathsf{SAT}]{(C_1.\mathcal{E}, C_2.\mathcal{E})} (C_1', C_2')}\ \text{OpCall}$$

$$\frac{(C_1, C_2) \xrightarrow[\mathsf{SAT}]{(\mathcal{E}_1,\mathcal{E}_2)} (C_1', C_2') \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1'', C_2'') \qquad C_1'' \overset{\mathsf{ret}(D)}{\longmapsto} C_1''' \qquad C_2'' \xrightarrow{\mathsf{ret}(D)} C_2'''}{(C_1, C_2) \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1'''[\mathcal{E}_1/\chi], C_2'''[\mathcal{E}_2/\chi])}\ \text{PropRet}$$

$$\frac{}{(C_1, C_2) \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1, C_2)}\ \text{Refl} \qquad \frac{(C_1, C_2) \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1', C_2') \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1'', C_2'')}{(C_1, C_2) \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1'', C_2'')}\ \text{Trans}$$

Figure 4. The Saturated Simulation Labelled Transition System.

Remark 30. The LTS defined above is an adaptation of the saturation procedure for pushdown systems presented in [23]. The intended meaning of $(C_1, C_2) \xrightarrow[\mathsf{SAT}]{\eta} (C_1', C_2')$ is that, using synchronisation on visible moves and the stacked LTS, $(\widetilde{(C_1, \cdot)}, \widetilde{(C_2, \cdot)})$ reduces to $(\widetilde{(C_1', K_1)}, \widetilde{(C_2', K_2)})$ following the simulation game and:

- if $\eta = \varepsilon$ then $K_1 = K_2 = \cdot$;
- if $\eta = (\mathcal{E}_1, \mathcal{E}_2)$ then $(K_1, K_2) = (\mathcal{E}_1, \mathcal{E}_2)$.

Standard (stacked) similarity is closed under transitions in the saturated LTS.

Lemma 31. *Given* $\widetilde{(C_1, K_1)} \sqsubseteq\!\!\!\approx \widetilde{(C_2, K_2)}$ *and* $(C_1, C_2) \xrightarrow[\mathsf{SAT}]{\eta} (C_1', C_2')$*:*

- *if* $\eta = \varepsilon$ *then* $\widetilde{(C_1', K_1)} \sqsubseteq\!\!\!\approx \widetilde{(C_2', K_2)}$,
- *if* $\eta = (\mathcal{E}_1, \mathcal{E}_2)$ *then* $(C_1', \widetilde{(\mathcal{E}_1, K_1)}) \sqsubseteq\!\!\!\approx (C_2', \widetilde{(\mathcal{E}_2, K_2)})$.

Proof. We use rule induction. The case of Refl is trivial, while that of Trans follows directly from induction hypothesis. For rule Tau we use Thm. 12. For PropCall we use determinacy of the stacked LTS. For OpCall, OpRet, suppose $C_i \xrightarrow{\eta} C_i'$ for $i = 1, 2$ and η having abstract names $\vec{\alpha} \# C_1, C_2$. Then, by hypothesis and determinacy, $\tilde{C}_1' \sqsubseteq\!\!\!\approx \tilde{C}_2'$. For PropRet, the induction hypothesis gives us $(C_1'', \widetilde{(\mathcal{E}_1, K_1)}) \sqsubseteq\!\!\!\approx (C_2'', \widetilde{(\mathcal{E}_2, K_2)})$. Combining this with hypotheses $C_1'' \overset{\mathsf{ret}(D)}{\longmapsto} C_1'''$ and $C_2'' \xrightarrow{\mathsf{ret}(D)} C_2'''$ (and using the fact that C_2'' has no other transitions), we obtain $(C_1'''[\widetilde{\mathcal{E}_1/\chi}], K_1) \sqsubseteq\!\!\!\approx (C_2'''[\widetilde{\mathcal{E}_2/\chi}], K_2)$. □

The main result is the following (cf. Sec. D). Note $\sqsubseteq\!\!\!\approx$ is standard (stacked) similarity.

Lemma 32. *The following is a weak (pushdown) simulation:*

$$\mathcal{R} = \{\ (C_1, C_2, \Sigma, \beta) \mid \Sigma \textit{ sat-connected} \wedge (C_1, C_2, \Sigma, \beta) \textit{ compatible}$$

(A) $$\wedge\, \forall K_i.\ \beta \xrightarrow{K_1,K_2}{}^{*}_{\Sigma} \diamond \implies \widetilde{(C_1, K_1)} \sqsubseteq\!\!\!\approx \widetilde{(C_2, K_2)}$$

(A*) $$\wedge\, \forall C_i', K_i.\ \ulcorner C_1', C_2' \urcorner \xrightarrow{K_1,K_2}{}^{*}_{\Sigma} \diamond \implies \widetilde{(C_1', K_1)} \sqsubseteq\!\!\!\approx \widetilde{(C_2', K_2)}$$

(B) $$\wedge\, \beta \neq \diamond \implies \exists C_1', C_2'.\ \beta = \ulcorner C_1', C_2' \urcorner \wedge (C_1', C_2') \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1, C_2)\ \}$$ □

We can now prove full abstraction. Recall $\Sigma_\diamond = \{(\diamond, \diamond, \diamond, \diamond)\}$.

Theorem 33. $\langle \cdot ; \cdot ; \cdot ; e_1 \rangle \sqsubseteq\!\!\!\approx \langle \cdot ; \cdot ; \cdot ; e_2 \rangle$ *iff* $(\!| \cdot ; \cdot ; e_1 |\!) \sqsubseteq\!\!\!\approx_{\Sigma_\diamond, \diamond} (\!| \cdot ; \cdot ; e_2 |\!)$.

Proof. We let $C_i = (\!| \cdot ; \cdot ; e_i |\!)$ and $\tilde{C}_i = \langle \cdot ; \cdot ; \cdot ; e_i \rangle$, for $i = 1, 2$. Note first that $\widetilde{(C_i, \cdot)} = \tilde{C}_i$. The right-to-left direction follows from Thm. 28. For the converse, suppose $\tilde{C}_1 \sqsubseteq\!\!\!\approx \tilde{C}_2$. By Thm. 32 there is weak simulation $\mathcal{R}$ defined as in the lemma. We claim that $C_1\ \mathcal{R}_{\Sigma_\diamond, \diamond}\ C_2$. We have that $\Sigma_\diamond$ is sat-connected, $(C_1, C_2, \Sigma_\diamond, \diamond)$ is compatible, conditions (A*) and (B) are vacuously true, while (A) is simplified to $\tilde{C}_1 \sqsubseteq\!\!\!\approx \tilde{C}_2$. Thus, $C_1 \sqsubseteq\!\!\!\approx_{\Sigma_\diamond, \diamond} C_2$. □

Corollary 34. *Contextual equivalence is decidable for context-free expressions.*

Proof. Follows from Thm.(s) 24 and 33. □

6 Up-to Techniques

PDNF bisimulation supports the standard techniques: up to identity, up to garbage collection, up to beta reductions and up to name permutations (see Appendix F). Here we present an *up to name reuse*, which is important for finitising examples such as those in Sec. 2, and a redesigned *up to separation* technique from [41], which is effective in finitising the bisimulation game of many examples. We develop our up-to techniques using the theory of bisimulation enhancements from [58, 57], which is based on *weak progression*, summarised below.

Definition 35. We write $\mathbf{wp}(R)$ for the monotone functional derived from Thm. 21.

Definition 36 (Progressions ($\rightsquigarrow$))**.**

- $\mathcal{R}$ *weakly progresses* to $\mathcal{S}$, and we write $\mathcal{R} \overset{\mathbf{wp}}{\rightsquigarrow} \mathcal{S}$ when $\mathcal{R} \subseteq \mathbf{wp}(\mathcal{S})$.
- For monotone functions f, g we write $f \overset{\mathbf{wp}}{\rightsquigarrow} g$ when $f \circ \mathbf{wp} \sqsubseteq \mathbf{wp} \circ g$.

Lemma 37. *$\mathcal{R}$ is a weak simulation when $\mathcal{R} \overset{\mathbf{wp}}{\rightsquigarrow} \mathcal{R}$. Also, $(\underline{\sqsubseteq}) = (\mathsf{gfp}(\mathbf{wp}))$.* □

The following gives the definition of an up-to technique, what it means to be sound, and the stronger notion of compatibility.

Definition 38.

- *Simulation up-to:* $\mathcal{R}$ is a *weak simulation up to* f when $\mathcal{R} \overset{\mathbf{wp}}{\rightsquigarrow} f(\mathcal{R})$.
- *Sound up-to technique:* Function f is $\mathbf{wp}$-*sound* when $\mathsf{gfp}(\mathbf{wp} \circ f) \subseteq \mathsf{gfp}(\mathbf{wp})$.

Lemma 39 ([58], Thm. 6.3.9)**.** *If $f \overset{\mathbf{wp}}{\rightsquigarrow} f$ then it is $\mathbf{wp}$-sound.* □

6.1 Up to Name Reuse In this section we define an up-to technique that allows us to reuse a single abstract function name in all opponent calls to a function in the knowledge environment, provided that the function does not contain any higher-order references. In such cases, it is guaranteed that the function being called will not contain names from past calls. We first define a name substitution that is only defined under these conditions.

Definition 40 (nr-Substitution)**.** The partial substitution operation $(\cdot)[\alpha'/\alpha]_{\mathsf{nr}}$ is defined only for terms that do not contain α, or have no higher-order references and no occurrences of α'. Note below Φ ranges over $\mathcal{E}$ and e.

$$\Phi[\alpha'/\alpha]_{\mathsf{nr}} \overset{\text{def}}{=} \begin{cases} \Phi[\alpha'/\alpha] & \text{if } \alpha' \# \Phi \text{ and } \Phi \text{ contains no HO references} \\ \Phi & \text{if } \alpha \# \Phi \end{cases}$$

$$\Gamma[\alpha'/\alpha]_{\mathsf{nr}}(i) \overset{\text{def}}{=} (\Gamma(i))[\alpha'/\alpha]_{\mathsf{nr}}$$

$$C[\alpha'/\alpha]_{\mathsf{nr}} \overset{\text{def}}{=} (\!|\Gamma[\alpha'/\alpha]_{\mathsf{nr}} ; s ; \Phi[\alpha'/\alpha]_{\mathsf{nr}}|\!) \qquad \text{if } C = (\!|\Gamma ; s ; \Phi|\!)$$

$$\beta[\alpha'/\alpha]_{\mathsf{nr}} \overset{\text{def}}{=} \ulcorner C_1[\alpha'/\alpha]_{\mathsf{nr}}, C_2[\alpha'/\alpha]_{\mathsf{nr}} \urcorner$$

$$\beta[\alpha'/\alpha]_{\mathsf{nr}} \xrightarrow{\mathcal{E}_1[\alpha'/\alpha]_{\mathsf{nr}}, \mathcal{E}_2[\alpha'/\alpha]_{\mathsf{nr}}}_{\Sigma[\alpha'/\alpha]_{\mathsf{nr}}} \beta'[\alpha'/\alpha]_{\mathsf{nr}} \qquad \text{if } \beta \xrightarrow{\mathcal{E}_1, \mathcal{E}_2}_{\Sigma} \beta'$$

Definition 41 (Up to Name Reuse)**.** The function nr on relations is defined as:

$$C_1 \ \mathsf{nr}(\mathcal{R})_{\Sigma,\beta} \ C_2 \quad \text{when} \quad \exists \alpha, \alpha'. \, C[\alpha'/\alpha]_{\mathsf{nr}} \ \mathcal{R}_{\Sigma[\alpha'/\alpha]_{\mathsf{nr}} @ \beta[\alpha'/\alpha]_{\mathsf{nr}},\ \beta[\alpha'/\alpha]_{\mathsf{nr}}} \ C_2[\alpha'/\alpha]_{\mathsf{nr}}$$

This technique is useful when opponent applies the same higher-order function more than once, e.g. to names α_1, α_2, α_3, etc. Immediately after the calls with arguments α_i, $i > 1$, we can apply the substitution $[\alpha_1/\alpha_i]_{\mathsf{nr}}$ and prove (bi)simulation of the resulting configurations, effectively using the same opponent name on all calls to the same function. The following lemma shows that (bi)simulation shown after applying such a substitution implies (bi)simulation of the configurations before applying the substitution.

Lemma 42. *Function nr is a sound up-to technique.*

Proof. We prove this by showing that $\mathsf{nr} \overset{\mathbf{wp}}{\rightsquigarrow} \mathsf{nr}$, that is $\mathsf{nr} \circ \mathbf{wp}(\mathcal{R}) \subseteq \mathbf{wp} \circ \mathsf{nr}(\mathcal{R})$ unfolding the definition of $\mathbf{wp}$. Note that we only need to prove this for a single substitution $[\alpha'/\alpha]_{\mathsf{nr}}$. Proponent calls and returns may extend the knowledge environments with function containing the substitution,

resulting in configurations captured by $\mathsf{nr}(\mathcal{R})$. Proponent returns in particular involves showing that graph reachability $\Sigma@\beta$ is invariant to name substitution. Opponent returns from configurations in $\mathsf{nr} \circ \mathbf{wp}(\mathcal{R})$ will produce the same configurations as the same transitions from $\mathbf{wp}(\mathcal{R})$, modulo the single substitution $[\alpha'/\alpha]_{\mathsf{nr}}$. Opponent calls are a bit more involved as they extend the call graph with all permutations of the new edge of the call graph; this however is captured by $\Sigma[\alpha'/\alpha]_{\mathsf{nr}}@\beta[\alpha'/\alpha]_{\mathsf{nr}}$ in the above definition. Reductions preserve the conditions of the substitution and termination and compatibility are unaffected by it. □

The lemma below, proven similarly to Thm. 42, shows that up to name reuse is a complete technique. Namely, if after applying a name substitution the (bi)simulation conditions are broken then the configurations before the substitution are inequivalent.

Lemma 43. *The function* nr^{-1} *defined below is a sound up-to technique.*

$$C[\alpha'/\alpha]_{\mathsf{nr}}\ \mathsf{nr}^{-1}(\mathcal{R})_{\Sigma[\alpha'/\alpha]_{\mathsf{nr}}@\beta[\alpha'/\alpha]_{\mathsf{nr}},\beta[\alpha'/\alpha]_{\mathsf{nr}}}\ C_2[\alpha'/\alpha]_{\mathsf{nr}} \quad \textit{when} \quad C_1\ \mathcal{R}_{\Sigma,\beta}\ C_2$$

□

6.2 Up to Separation We next develop an adaptation of up to separation from [41]. This is an effective technique for reducing the state-space of the bisimulation exploration in our verification tool. The intuition of this technique is that if different functions operate on disjoint parts of the store, they can be explored by bisimulation independently, removing interleaving of their calls. In cases where a function does not contain free locations, the effect of this technique is to allow bisimulation to apply it only once, as two copies of the function will not interfere with each other, even if the create new locations when run.

To define up to separation we need a separating conjunction for configurations.

Definition 44 (Separating Conjuction)**.** We define the partial function $\oplus$ on stores and knowledge environments as:

$$s_1 \oplus_L s_2 = s_1, s_2 \qquad \Gamma_1 \oplus_{I,L} \Gamma_2 = \Gamma_1, \Gamma_2$$

when $\mathsf{dom}(s_1) \cap \mathsf{dom}(s_2) = \emptyset$ and $\mathsf{fl}(s_2) \subseteq \mathsf{dom}(s_2) = L$ and $\mathsf{fl}(s_1) \subseteq \mathsf{dom}(s_1)$, and when $\mathsf{dom}(\Gamma_1) \cap \mathsf{dom}(\Gamma_2) = \emptyset$ and $\mathsf{dom}(\Gamma_2) = I$ and $\mathsf{fl}(\Gamma_2) \subseteq L$ and $\mathsf{fl}(\Gamma_1) \cap L = \emptyset$. Moreover we write $s_1 \oplus s_2$ and $\Gamma_1 \oplus \Gamma_2$ when there exist I, L such that $s_1 \oplus_L s_2$ and $\Gamma_1 \oplus_{I,L} \Gamma_2$, respectively.

$$\begin{aligned}
(\!|\Gamma_1 \oplus_{I,L} \Gamma\,;s_1 \oplus s\,;\mathcal{E}|\!) \quad \oplus^0_I\ (\!|\Gamma_2 \oplus_{I,L} \Gamma\,;s_2 \oplus_L s\,;\mathcal{E}|\!) &= (\!|\Gamma_1 \oplus \Gamma_2 \oplus \Gamma\,;s_1 \oplus s_2 \oplus s\,;\mathcal{E}|\!)\\
(\!|\Gamma_1 \oplus_{I,L} \Gamma\,;s_1 \oplus_L s\,;e|\!) \quad \oplus^0_I\ (\!|\Gamma_2 \oplus_{I,L} \Gamma\,;s_2 \oplus_L s\,;e|\!) &= (\!|\Gamma_1 \oplus \Gamma_2 \oplus \Gamma\,;s_1 \oplus s_2 \oplus s\,;e|\!)\\
(\!|\Gamma_1 \oplus_{I,L} \Gamma\,;s_1 \oplus_L s\,;\mathcal{E}_1|\!) \oplus^j_I\ (\!|\Gamma_2 \oplus_{I,L} \Gamma\,;s_2 \oplus_L s\,;\mathcal{E}_2|\!) &= (\!|\Gamma_1 \oplus \Gamma_2 \oplus \Gamma\,;s_1 \oplus s_2 \oplus s\,;\mathcal{E}_j|\!)\\
(\!|\Gamma_1 \oplus_{I,L} \Gamma\,;s_1 \oplus_L s\,;e_1|\!) \oplus^1_I\ (\!|\Gamma_2 \oplus_{I,L} \Gamma\,;s_2 \oplus_L s\,;\mathcal{E}_2|\!) &= (\!|\Gamma_1 \oplus \Gamma_2 \oplus \Gamma\,;s_1 \oplus s_2 \oplus s\,;e_1|\!)\\
(\!|\Gamma_1 \oplus_{I,L} \Gamma\,;s_1 \oplus_L s\,;\mathcal{E}_1|\!) \oplus^2_I\ (\!|\Gamma_2 \oplus_{I,L} \Gamma\,;s_2 \oplus_L s\,;e_2|\!) &= (\!|\Gamma_1 \oplus \Gamma_2 \oplus \Gamma\,;s_1 \oplus s_2 \oplus s\,;e_2|\!)\\
C_1 \oplus^0_I C_2 = (\!|\bot|\!) \text{ when } C_1 = C_2 &= (\!|\bot|\!)\\
C_1 \oplus^j_I C_2 = (\!|\bot|\!) \text{ when } i \in \{1,2\} \text{ and } C_1 = (\!|\bot|\!) \text{ or } C_2 &= (\!|\bot|\!)
\end{aligned}$$

Where $j \in \{1,2\}$, $\mathsf{fl}(e,\mathcal{E}) \subseteq \mathsf{dom}(s) = L$, $\mathsf{fl}(e_i,\mathcal{E}_i) \subseteq \mathsf{dom}(s_i)$ $(i \in \{1,2\})$,

The intuition here is that instead of exploring bisimulation with the composite configuration, we can instead explore it only with the smaller, constituent configurations. Note that these configurations are allowed to contain a common store s and knowledge environment Γ. This makes the technique possible in intermediate configurations where some state has already been allocated and functions in Γ can access it.

The definition of up to separation shall use a product construction on continuation graphs and a dual merging operation. The key intuition is that (bi-)simulation is preserved by these operations for graphs and relations (cf. Sec. G).

Definition 45 (Pair Entry Points and Pair Continuation Graphs)**.**

$$\mathsf{PEPoint} \ni \beth ::= (\beta_1, \beta_2, k) \qquad (k \in \{0,1,2\})$$

$$\mathsf{PCGrph} \ni \mathbb{\Sigma} \subseteq^{\neq\emptyset}_{\text{fin}} \mathsf{PEPoint} \times \mathsf{Kont} \times \mathsf{Kont} \times \mathsf{PEPoint}$$

and each $\mathbb{\Sigma}$ must satisfy the conditions:

- *Reachability.* For all $\beth \in \mathsf{dom}(\mathbb{\Sigma})$ there are stacks K_1, K_2 such that $\beth \xrightarrow{K_1,K_2}{}^{*}_{\mathbb{\Sigma}} \diamond$

- *Top and Divergence.* For all $\mathbb{\beta}'^{k'} \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}_{\mathbb{\Sigma}} \mathbb{\beta}^k$ and $j \in \{1,2\}$:

  $$(\diamond,\diamond,0) \in \mathsf{dom}(\mathbb{\Sigma}) \wedge (\mathbb{\beta}' = (\diamond,\diamond,0) \implies \mathbb{\beta} = (\diamond,\diamond,0)) \wedge (\mathbb{\beta} = (\diamond,\diamond,0) \iff \mathcal{E}_j = \diamond)$$
  $$\wedge\, (\mathbb{\beta}'.k.j = \bot \iff \mathcal{E}_j = \bot) \wedge (\mathbb{\beta}.k.j = \bot \implies \mathcal{E}_j = \bot)$$

  where, if $\mathbb{\beta} = (\beta_1, \beta_2, k)$ and $i \in \{1,2\}$, we write $\mathbb{\beta}.i.1 = \bot$ when $\beta_i = \ulcorner (\!|\bot|\!), C \urcorner$ and $\mathbb{\beta}.i.2 = \bot$ when $\beta_i = \ulcorner C, (\!|\bot|\!) \urcorner$.

- *Nominal closure.* For all $\mathbb{\beta}' \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}_{\mathbb{\Sigma}} \mathbb{\beta}$ and permutations π, $\pi \cdot \mathbb{\beta}' \xrightarrow{\pi\cdot\mathcal{E}_1,\pi\cdot\mathcal{E}_2}_{\mathbb{\Sigma}} \pi \cdot \mathbb{\beta}$.

Finally, we lift Thm. 19 to pair graphs obtaining continuation graph extension $\mathbb{\Sigma}[\mathbb{\beta}' \mapsto (\mathcal{E}_1, \mathcal{E}_2, \mathbb{\beta}'')]$ and restriction $\mathbb{\Sigma}@\mathbb{\beta}$. We will write $\mathbb{\beta}.i$ to mean β_i ($i \in \{1,2\}$), and $\mathbb{\beta}^m$ to mean $k = m$, when $\mathbb{\beta} = (\beta_1, \beta_2, k)$.

Definition 46. Given two continuation graphs Σ_1, Σ_2 we construct the product graph:

$$\frac{}{(\diamond,\diamond,0) \xrightarrow{\diamond,\diamond}_{\Sigma_1\otimes\Sigma_2} (\diamond,\diamond,0)}$$

$$\frac{(\beta_1,\beta_2,k) \in \mathsf{dom}(\Sigma_1 \otimes \Sigma_2) \quad \forall i \in \{1,2\}.\ \beta_i' \xrightarrow{\mathcal{E},\mathcal{E}'}_{\Sigma_i} \beta_i \quad \forall j \in \{1,2\}.\ \beta_1'.j.e = \beta_2'.j.e}{(\beta_1',\beta_2',0) \xrightarrow{\mathcal{E},\mathcal{E}'}_{\Sigma_1\otimes\Sigma_2} (\beta_1,\beta_2,k)}$$

$$\frac{(\beta_1,\beta_2,k) \in \mathsf{dom}(\Sigma_1 \otimes \Sigma_2) \qquad \beta_i' \xrightarrow{\mathcal{E},\mathcal{E}'}_{\Sigma_i} \beta_i \qquad \beta_{\widehat{i}} = \beta'_{\widehat{i}} \qquad i \in \{1,2\}}{(\beta_1',\beta_2',i) \xrightarrow{\mathcal{E},\mathcal{E}'}_{\Sigma_1\otimes\Sigma_2} (\beta_1,\beta_2,k)}$$

Lemma 47. *Suppose Σ_1, Σ_2 well-formed continuation graphs; then $\Sigma_1 \otimes \Sigma_2$ is a well-formed pair continuation graph.* □

Definition 48 (Merging)**.** Suppose

$$\beta_1 = ((\Gamma_1 \oplus_{I,L} \Gamma_3), (\Gamma_1' \oplus_{I,L'} \Gamma_3'), (s_1 \oplus_L s_3), (s_1' \oplus_{L'} s_3'), e_1)$$
$$\beta_2 = ((\Gamma_2 \oplus_{I,L} \Gamma_3), (\Gamma_2' \oplus_{I,L'} \Gamma_3'), (s_2 \oplus_L s_3), (s_2' \oplus_{L'} s_3'), e_2)$$

We define the partial merging function $[\![\cdot]\!]$ for pair nodes as:

$$[\![(\beta_1,\ \beta_2,\ k)]\!] \stackrel{\text{def}}{=} (\Gamma_1 \oplus \Gamma_2 \oplus \Gamma_3, \Gamma_1' \oplus \Gamma_2' \oplus \Gamma_3', s_1 \oplus s_2 \oplus s_3, s_1' \oplus s_2' \oplus s_3', e)$$

provided that when $k = 0$ then $e_1 = e_2 = e$ and when $k \in \{1,2\}$ then $e_i = e$. We extend merging to well-formed pair continuation graphs $\mathbb{\Sigma}$:

$$[\![\mathbb{\beta}']\!] \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}_{[\![\mathbb{\Sigma}]\!]} [\![\mathbb{\beta}]\!] \quad \text{when} \quad \mathbb{\beta}' \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}_{\mathbb{\Sigma}} \mathbb{\beta}$$

Definition 49 (Up to Separation)**.** The partial function sep provides the up to separation technique:

$$C_1 \oplus_I^k C_2\ \mathsf{sep}(\mathcal{R})_{\Sigma,[\![\mathbb{\beta}]\!]}\ C_1' \oplus_I^k C_2'$$

when $C_i\ \mathcal{R}_{\Sigma_i,\beta_i}\ C_i'$ ($i \in \{1,2\}$) and $\Sigma = [\![(\Sigma_1 \otimes \Sigma_2)@\mathbb{\beta}]\!]$ and $\mathbb{\beta} = (\beta_1, \beta_2, k)$.

Lemma 50. *Function sep is a sound up-to technique.* □

This technique is also complete, which is important for our tool as it allows us to use it without backtracking (in contrast to up to weakening and garbage collection).

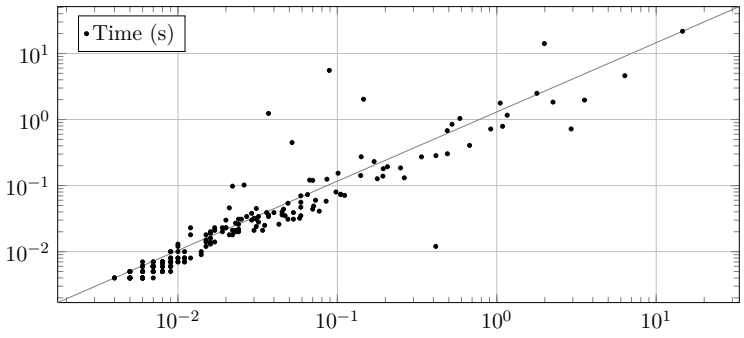
Lemma 51. *Suppose that $C_1 \oplus_I^0 C_2 \sqsubseteq_{\approx\Sigma,\beta} C_1' \oplus_I^0 C_2'$. Then, there exist Σ' and β' such that $C_i \sqsubseteq_{\approx\Sigma',\beta'} C_i'$, for $i \in \{1,2\}$.* □

7 Implementation and Evaluation

We implemented PDNF bisimulation in a prototype tool called PDNF-Bisim that checks programs written in an ML-like syntax for λ^{imp}. The tool was developed by replacing the LTS and bisimulation definition in Hobbit[41] with an implementation of our Stackless LTS (Fig. 5) and a Bounded Symbolic Execution of our PDNF bisimulation (Def. 21). As such, the tools share the same front-end, enhancement techniques, reduction semantics, and symbolic execution routine (calling Z3 to resolve constraints). They otherwise differ in the implementation of the LTS and bisimulation game, as well as in Hobbit's up to reentry, which the PDNF-Bisim cannot use as it lacks a stack.

As a symbolic execution tool, PDNF-Bisim is *sound* (reports only true positives and negatives) and *bounded-complete* (exhaustively and precisely explores all paths up to a bound). The bound used here is different than the bound in the decidability result in Sec. 5.3, and it is intended to

	Hobbit tests: 129 eq's and 78 ineq's			PDNF-Bisim tests: 12 eq's	
	PDNF	Hobbit	H+reentry	PDNF	Hobbit
Eq. Proven	72	62	67	11	0
Ineq. Proven	77	78	78	N/A	N/A



(X) Hobbit vs. (Y) PDNF-Bisim over Hobbit's test suite

Table 1. Summary of experiments comparing PDNF-Bisim to Hobbit

be used as a more straightforward timeout. In PDNF-Bisim, we bound the number of proponent calls and both opponent calls and returns along an execution path, whereas Hobbit bounds only calls. This is done because the saturation procedure in PDNF bisimulation may lead to cycles in the continuation graph, which when explored by PDNF-Bisim lead to unbounded returns without the same number of corresponding calls. We accumulate SAT/SMT constraints by extending the LTS with a *symbolic environment* σ for *symbolic constants* κ and reductions involving any κ; we branch on symbolic conditions as is standard of symbolic execution. The exploration is performed over *configuration pairs* $\langle C_1, C_2, \Sigma, \beta, \sigma, k_{\mathsf{call}}, k_{\mathsf{ret}}, k_{\mathsf{int}} \rangle$ of related term configurations C_1 and C_2, continuation graph Σ, current call entry point β, symbolic environment σ and given bounds k_{call} for calls, k_{ret} for returns and k_{int} for internal reductions. As with Hobbit, we make use of enhancements that help finitise the bisimulation exploration in some examples: explore-set memoisation to discover cycles; normalisation; store garbage collection; σ garbage collection and simplification; up to separation (Sec. 6.2); and up to name reuse (Sec. 6.1). In addition, a normalisation procedure is implemented to ensure Σ is effectively closed under permutation by capturing the complete orbit of every edge in Σ via a canonical representation of the abstract names in said edge.

7.0.1 Evaluation We evaluate here our tool against Hobbit as a reference implementation of the standard (stacked) bisimulation and because of its favourable comparison to other tools in the higher-order program equivalence landscape [41, Sec. 9]. Both tools were executed over two test suites: (1) Hobbit's suite of 129 equivalences and 78 inequivalences; and (2) our own suite of 12 equivalences (11 inspired by Event Handlers in Android [5, 1], JavaScript [4], Java Swing [6], jQuery [3], and the DOM Framework [2]; and 1 based on a simplification of a CDMA-WLAN handoff protocol[40]). Combined, the test suites total 6701LoC — viz. 6182LoC in (1) with 3802LoC in equivalences and 2380LoC in inequivalences, and 519LoC in (2). For this comparison, we are interested in three scenarios: the performance of both approaches as fully automatic techniques, for which all invariant, reentry and synchronisation annotations were removed from the Hobbit testsuite; PDNF-Bisim against Hobbit assuming the reentry annotations have been placed correctly, to measure how PDNF bisimulation fares in comparison to NF bisimulation with up to reentry to finitise reentrant calls; and PDNF-Bisim against Hobbit on our own test suite, which aims to showcase the difficulty of dealing with reentrant functions in the presence of changing state. The tools were evaluated on an Intel Core i7 1.90GHz machine with 32GB RAM running OCaml 4.10.0 and Z3 4.8.10 on Ubuntu 23.04. We record the results of our comparison in Table 1. Execution of each example was capped to a 150-second timeout.

Firstly, PDNF-Bisim verified 72 equivalences, which contain all 62 equivalences that Hobbit verified; without up to reentry Hobbit did not prove any examples that PDNF-Bisim could not prove. Execution times were also not significantly different ($r = 0.74$). We can thus conclude that for equivalences PDNF-Bisim supersedes Hobbit at fully-automatic verification by proving 9 additional examples on Hobbit's own test suite (without manual annotations) with minimal difference in performance. Note, however, that Hobbit is more mature as a semi-automatic tool, and (from testing) is able to prove up

to 95 examples when invariant, reentry and synchronisation annotations are appropriately used (albeit requiring significant effort and experience in formalising equivalence annotations). Additionally, one inequivalence is not proven by PDNF-Bisim. This example (c.f. invariants-4) is particularly difficult as no up-to techniques apply, memoisation is unable to finitise the path exploration, and the failing trace exhibits sequences of sub-traces that nest deeply. Both PDNF-Bisim and Hobbit are able to solve this example on small parameters and both encounter an exponentially growing number of configurations, but Hobbit's more elementary transition system leads to a faster exploration. Bar implementation concerns, the slower analysis may be explained by a higher branching factor due to graph-based returns, which are additionally able to expand more deeply than in Hobbit as returns can occur without the same number of corresponding calls.

Secondly, we observe in Table 1 that, Hobbit was able to prove an additional 5 examples by turning on the up to reentry technique (but not the rest of the manual up-to techniques) and carefully adding reentry annotations in the right functions, leaving 5 examples from Hobbit's testsuite that can be exclusively proven by PDNF-Bisim. We can thus conclude that for our second scenario on equivalences, PDNF-Bisim still supersedes Hobbit with semi-automatic reentry annotations.

Finally, on our own test suite, PDNF-Bisim is clearly superior to Hobbit on higher-order stateful programs that feature reentrant calls with changing state as is common in, but not limited to, higher-order data structures, event-driven programming, and various protocols. In these, the stackless approach was able to quickly saturate the graph and prove equivalence, whereas Hobbit was unable to finitise the bisimulation game and eventually timed out. Lastly, one of the examples in our test suite is provable by neither PDNF-Bisim nor Hobbit. We include this example to illustrate a current limitation of our technique: it cannot finitise exploration concerning infinite state. To achieve this would require adapting Hobbit's invariant annotations technique to our framework.

8 Related and Future Work

Theorems of closed instantiation of uses (CIU theorems) were amongst the first operational techniques that reduced the contexts considered by contextual equivalence in languages with state [51, 21, 31]. Applicative bisimulation [7] was the first application of bisimulation to a (pure) higher-order programming language, which reduces contexts further by considering applying top-level term functions to identical closed arguments. Logical relations [38, 10, 34] can be viewed as similarly reducing the examined contexts applying functions to related arguments. Environmental bisimulation [63, 64, 43, 60] introduces stratification of bisimulations based on state and opponent knowledge, providing an effective proof technique due to being amenable to up-to techniques [12, 43, 60, 58], while applying functions to closed arguments derived by the congruence of the bisimulation. Game semantics [9, 35, 56], provides fully abstract denotational semantics for a range of higher-order languages, and in particular languages with higher-order state [8, 45, 55]. Algorithmic interpretations thereof give rise to decision procedures for contextual equivalence for restricted language fragments [26, 32, 17, 53]. The SyTeCi tool [37] combines notions from game semantics and logical relations, and manages to overcome some of the language restrictions of game-semantics tools. Normal form bisimulation, discussed in the introduction, treats context-generated code symbolically, entirely removing quantification over context-generated code and leading to sound but not complete techniques, with the notable exception of the case of higher-order languages with: sequential control and state [62], state-only [12, 41], and no effects [42]. It has been shown [49, 50, 41] that NF bisimulation relates to operational game semantics models where opponent-generated terms are also represented by names [45, 27, 36]. The closest work to ours is [41], which combines game semantics and techniques from environmental bisimulations and up-to techniques to produce a fully abstract LTS suitable for NF bisimulation.

Unlike prior approaches, our treatment of the stack stems from model checking pushdown systems [15, 23, 61] and exact-stack control-flow analyses of higher-order functional languages [28, 65, 19, 33], and allows us to eliminate the need for a term/context call stack without loss of precision. Our approach is related to [28], where the use of a continuation graph is proposed (called *continuation store*). The reachability analysis of procedural code using pushdown systems and saturation techniques was first considered in [20, 61]. Saturation typically relies on the fact that the underlying control state space is finite, which is not the case in our NF bisimulation games. We therefore follow an on-the-fly forward saturation procedure which over-approximates the saturation procedure devised in [23]. While this over-approximation is generally unsound (cf. Thm. 2), it is sound for reachability.

In conclusion, in this work we created a novel fully abstract technique for contextual equivalence and implement a bounded-complete prototype verification tool. Our tool is able to verify equivalence in a number of examples which were out of reach in previous work. In the future we believe that our work can lead to useful verification tools, for example for regression verification [29, 30, 22, 44] in higher-order languages with state, relational verification of assertion reachability in code, or even (single-program) contextual model checking in settings such as blockchain smart contracts.

For the purpose of Open Access, the author/s has/have applied a CC BY public copyright licence to any Author Accepted Manuscript version arising from this submission.

This appendix is provided for the benefit of the reviewers, and will not appear in a final version of this paper.

A The Stackless LTS

$$
\begin{array}{ll}
\text{PropCall} : (\!|\Gamma\,;s\,;E[\alpha\,v]|\!) \xrightarrow{\mathsf{call}(\alpha,D)} (\!|\Gamma,\Gamma'\,;s\,;E|\!) & \text{if}\quad (D,\Gamma') \in \mathsf{ulpatt}(v) \\
\text{PropRet} : (\!|\Gamma\,;s\,;v|\!) \xrightarrow{\mathsf{ret}(D)} (\!|\Gamma,\Gamma'\,;s\,;\chi|\!) & \text{if}\quad (D,\Gamma') \in \mathsf{ulpatt}(v) \\
\text{OpCall} : (\!|\Gamma\,;s\,;\mathcal{E}|\!) \xrightarrow{\mathsf{call}(i,D[\vec{\alpha}])} (\!|\Gamma\,;s\,;e|\!) & \text{if}\quad \vec{\alpha}\,\#\,\Gamma,s,\mathcal{E} \wedge (D,\vec{\alpha}) \in \mathsf{ulpatt}(T) \\
 & \quad \wedge\, \Sigma_s \vdash \Gamma(i) : T \to T' \wedge \Gamma(i)\,D[\vec{\alpha}] \succ e \\
\text{OpRet} : (\!|\Gamma\,;s\,;E[\cdot]_T|\!) \xrightarrow{\mathsf{ret}(D[\vec{\alpha}])} (\!|\Gamma\,;s\,;E[D[\vec{\alpha}]]|\!) & \text{if}\quad \vec{\alpha}\,\#\,\Gamma,s,\mathcal{E} \wedge (D,\vec{\alpha}) \in \mathsf{ulpatt}(T) \\
\text{Tau} : (\!|\Gamma\,;s\,;e|\!) \xrightarrow{\tau} (\!|\Gamma\,;s'\,;e'|\!) & \text{if}\quad \langle s\,;e\rangle \to \langle s'\,;e'\rangle \\
\text{Response} : C \xrightarrow{\eta} (\!|\bot|\!) & \text{if}\quad \eta \neq \tau \text{ and } C \overset{\eta}{\not\Rightarrow} \text{ from other rules}
\end{array}
$$

Figure 5. The Stackless Labelled Transition System. Relation $\succ$ is defined as in Fig. 3.

B Typing rules of λ^{IMP}

$$
\frac{\alpha \in \mathsf{ANam}_{T\to T'}}{\Delta;\Sigma \vdash \alpha : T \to T'} \qquad \frac{c \text{ cons. of type } T}{\Delta;\Sigma \vdash c : T} \qquad \frac{(x:T)\in\Delta}{\Delta;\Sigma\vdash x:T} \qquad \frac{\Delta;\Sigma\vdash e_1:T_1 \quad \dots \quad \Delta;\Sigma\vdash e_n:T_n}{\Delta;\Sigma\vdash (e_1,\dots,e_n):T_1 * \cdots * T_n}
$$

$$
\frac{op : \vec{T} \to T \quad \Delta;\Sigma\vdash(\vec{e}):\vec{T}}{\Delta;\Sigma\vdash op(\vec{e}):T} \qquad \frac{\Delta;\Sigma\vdash e:\mathsf{bool} \qquad \Delta;\Sigma\vdash(e_1,e_2):T*T}{\Delta;\Sigma\vdash \mathsf{if}\,e\,\mathsf{then}\,e_1\,\mathsf{else}\,e_2:T}
$$

$$
\frac{\Delta;\Sigma\vdash v:T \quad \Delta;\Sigma,l:T\vdash e:T'}{\Delta;\Sigma\vdash \mathsf{ref}\,l=v\,\mathsf{in}\,e:T'} \qquad \frac{(l:T)\in\Sigma}{\Delta;\Sigma\vdash !l:T} \qquad \frac{(l:T)\in\Sigma \quad \Delta;\Sigma\vdash e:T}{\Delta;\Sigma\vdash l:=e:\mathsf{unit}}
$$

$$
\frac{\Delta;\Sigma\vdash e:T\to T' \quad \Delta;\Sigma\vdash e':T}{\Delta;\Sigma\vdash ee':T'} \qquad \frac{\Delta,f:T\to T',x:T;\Sigma\vdash e:T'}{\Delta;\Sigma\vdash \mathsf{fix}f(x).e:T\to T'}
$$

$$
\frac{\Delta,x_1:T_1,\dots,x_n:T_n;\Sigma\vdash e:T \quad \Delta;\Sigma\vdash e':\vec{T}}{\Delta;\Sigma\vdash \mathsf{let}(\vec{x})=e'\,\mathsf{in}\,e:T}
$$

C Simple lemmas

Thm. 18 For any $\beta \xrightarrow{K_1,K_2}{}^*_\Sigma \diamond$, we have that K_1, K_2 are defined and:

- $\beta = \diamond$ and $K_1 = K_2 = \cdot$, or
- $\beta.1, \beta.2 \neq \bot$ and $|K_1| = |K_2|$, or
- $\exists j \in \{1,2\}.\ \beta.j = \bot \wedge K_j = \bot \neq K_{3-j}$

where $|K|$ is the length of K (if $K \neq \bot$).

Proof. By rule induction. The base case is clear. Suppose now

$$
\frac{\beta' \xrightarrow{K_1,K_2}{}^*_\Sigma \diamond \qquad \beta \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}_\Sigma \beta'}{\beta \xrightarrow{(\mathcal{E}_1,K_1),(\mathcal{E}_2,K_2)}{}^*_\Sigma \diamond}
$$

By induction hypothesis, either $\beta'.1, \beta'.2 \neq \bot$ and $|K_1| = |K_2|$, or (WLOG) $\beta'.1 = \bot$ and $K_1 = \bot \neq K_2$. Suppose the former is the case. If $\beta' \neq \diamond$ then $\mathcal{E}_j \neq \diamond$ and $\mathcal{E}_j, K_j$ is defined (for $j = 1, 2$), hence $\beta.1, \beta.2 \neq \bot$ and $|\mathcal{E}_1, K_1| = |\mathcal{E}_2, K_2|$, or (by divergence) there is exactly one j such that $\mathcal{E}_j, K_j = \bot$ and $\beta.j = \bot$. If $\beta' = \diamond$ then by definition $K_1 = K_2 = \cdot$ and $\mathcal{E}_1 = \mathcal{E}_2 = \diamond$ and the claim follows. It remains to check the case $K_1 = \bot \neq K_2$ and $\beta'.1 = \bot$. By Thm. 15 (divergence) we have that $\mathcal{E}_1 = \beta.1 = \bot$. It remains to check that $\mathcal{E}_2, K_2$ is defined and not $\bot$. The former is clear as $\mathcal{E}_2 \neq \diamond$ (as $\beta' \neq \diamond$) and $K_2 \neq \bot$. For the latter, observe that $\mathcal{E}_1 = \bot \neq \mathcal{E}_2$. □

Thm. 27 For any $\beta \models_\Sigma (K_1, K_2)$ and C_1, C_2, if $\beta = \ulcorner C_1, C_2 \urcorner$ or $(C_1, C_2, \Sigma, \beta)$ compatible then $\widetilde{(C_1, K_1)}, \widetilde{(C_2, K_2)}$ are defined.

Proof. By symmetry, it suffices to show that $\widetilde{(C_1, K_1)}$ is defined. WLOG assume $C_1 \neq (\!|\bot|\!)$, so $\beta.1 \neq \bot$. Then, by Thm. 18 we have that $K_1 \neq \bot$. Finally, we need to check that if C_1 is an O-configuration and $(C_1, C_2, \Sigma, \beta)$ compatible then $C_1.\mathcal{E} = \diamond \iff K_1 = \cdot$. By Thm. 18, $K_1 = \cdot$ iff $\beta = \diamond$. By compatibility, $\beta = \diamond$ iff $C_1.\mathcal{E} = \diamond$. Thus, $\widetilde{(C_1, K_1)}$ is defined. □

Thm. 28 If $\mathcal{R}$ is a weak simulation then so is:

$$\tilde{\mathcal{R}} = \{(\widetilde{(C_1, K_1)}, \widetilde{(C_2, K_2)}) \mid \exists \Sigma, \beta.\ C_1\ \mathcal{R}_{\Sigma,\beta}\ C_2 \wedge \beta \xrightarrow{K_1,K_2}{}^*_\Sigma \diamond\}$$

Moreover, if $\mathcal{R}$ is a weak bisimulation then so is $\tilde{\mathcal{R}}$.

Proof. Let $\Sigma, \beta, C_1, C_2, K_1, K_2$ be such that $\beta \xrightarrow{K_1,K_2}{}^*_\Sigma \diamond$ and $C_1\ \mathcal{R}_{\Sigma,\beta}\ C_2$, and $\tilde{C}_i = \widetilde{(C_i, K_i)}$. We assume WLOG that $\tilde{C}_1 \neq \langle \bot \rangle$ as otherwise the simulation conditions are vacuously true. If $\tilde{C}_1 \downarrow$ then $K_1 = \cdot$ and $C_1.\mathcal{E} = \diamond$, and hence $C_1 \downarrow$ and by $C_1\ \mathcal{R}_{\Sigma,\beta}\ C_2$ we obtain $C_2 \downarrow$. Since $\beta \xrightarrow{K_1,K_2}{}^*_\Sigma \diamond$ by Thm. 18 we have $K_2 = \cdot$ or $K_2 = \beta.2 = \bot$. The latter case is excluded by $C_2 \neq (\!|\bot|\!)$, hence $\tilde{C}_2 \downarrow$. Suppose now $C_i = (\!|\Gamma_i\,;s_i\,;\Phi_i|\!)$, for $i = 1, 2$, so $\tilde{C}_i = \langle \Gamma_i\,;K_i\,;s_i\,;\Phi_i\rangle$. Cases:

- $\tilde{C}_1 \xmapsto{\mathsf{call}(i,D[\vec{\alpha}])} \tilde{C}'_1$ with $\vec{\alpha} \# \tilde{C}_2$. By Thm. 12 and equivariance of the LTS, we can assume WLOG that $\vec{\alpha} \# \beta$, so $\vec{\alpha} \# C_2, \beta$. Then, $\Phi_i = \mathcal{E}_i$ $(i = 1, 2)$ and $C_1 \xmapsto{\mathsf{call}(i,D[\vec{\alpha}])} C'_1$ with $\tilde{C}'_1 = \widetilde{(C'_1, \widetilde{(\mathcal{E}_1, K_1)})}$. Thus, $C_2 \xmapsto{\mathsf{call}(i,D[\vec{\alpha}])} C'_2$ and $C'_1\ \mathcal{R}_{\Sigma',\beta'}\ C'_2$ with $\beta' = \ulcorner C'_1, C'_2 \urcorner$ and $\Sigma' = \Sigma[\beta' \xrightarrow{\mathcal{E}_1,\mathcal{E}_2} \beta]$. Hence, $\tilde{C}_2 \xmapsto{\mathsf{call}(i,D[\vec{\alpha}])} \tilde{C}'_2$ with $\tilde{C}'_2 = \widetilde{(C'_2, \widetilde{(E_2, K_2)})}$. The claim follows from the fact that $\beta' \xrightarrow{(\mathcal{E}_1,K_1),(\mathcal{E}_2,K_2)}{}^*_{\Sigma'} \diamond$.
- $\tilde{C}_1 \xmapsto{\mathsf{ret}(D)} \tilde{C}'_1$. Then, $\Phi_i = v_i$ $(i = 1, 2)$ and, assuming $K_i = \mathcal{E}_i, K'_i$ $(i = 1, 2)$ and $\tilde{C}'_1 = \langle \Gamma'_1\,;K'_1\,;s_1\,;\mathcal{E}_1\rangle$, we have $C_1 \xmapsto{\mathsf{ret}(D)} C'_1$ with $C'_1 = (\!|\Gamma'_1\,;s_1\,;\chi|\!)$. Now, if $\mathcal{E}_1, \mathcal{E}_2 \neq \diamond$ then $\beta \xrightarrow{K_1,K_2}{}^*_\Sigma \diamond$ must be due to some $\beta' \xrightarrow{K'_1,K'_2}{}^*_\Sigma \diamond$ and $\beta \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}{}_\Sigma \beta'$. Otherwise, $\mathcal{E}_1 = \mathcal{E}_2 = \diamond$ and $K_i = K'_i = \cdot$ $(i = 1, 2)$. In either case, there is β' such that $\beta \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}{}_\Sigma \beta'$ and $\beta' \xrightarrow{K'_1,K'_2}{}^*_{\Sigma'} \diamond$, where $\Sigma' = \Sigma@\beta'$. For that β', by bisimilarity conditions, we have $C_2 \xRightarrow{\mathsf{ret}(D)} C'_2$ and $C'_1[\mathcal{E}_1/\chi]\ \mathcal{R}_{\Sigma',\beta'}\ C'_2[\mathcal{E}_2/\chi]$, and hence $\tilde{C}_2 \xRightarrow{\mathsf{ret}(D)} \tilde{C}'_2$ with $\tilde{C}'_2 = \langle \Gamma'_2\,;K'_2\,;s_2\,;\mathcal{E}_2\rangle$ and $C'_2 = (\!|\Gamma'_2\,;s_2\,;\chi|\!)$ for some Γ'_2 and s_2. Observing that $\tilde{C}'_i = \widetilde{(C'_i[\mathcal{E}_i/\chi], K'_i)}$ we obtain $\tilde{C}'_1\ \tilde{\mathcal{R}}\ \tilde{C}'_2$.
- $\tilde{C}_1 \xmapsto{\tau} \tilde{C}'_1$. Then, $\Phi_i = e_i$ $(i = 1, 2)$ and, assuming $\tilde{C}'_1 = \langle \Gamma_1\,;K_1\,;s'_1\,;e'_1\rangle$, we have $C_1 \xmapsto{\tau} C'_1$ with $C'_1 = (\!|\Gamma_1\,;s'_1\,;e'_1|\!)$. Thus, by $C_1\ \mathcal{R}_{\Sigma,\beta}\ C_2$, we have $C_2 \xRightarrow{\tau} C'_2$ with $C'_1\ \mathcal{R}_{\Sigma,\beta}\ C'_2$ for some $C'_2 = (\!|\Gamma_2\,;s'_2\,;e'_2|\!)$. But then $\tilde{C}_2 \xRightarrow{\tau} \tilde{C}'_2 = \langle \Gamma_2\,;K_2\,;s'_2\,;e'_2\rangle$ and, by definition, $\tilde{C}'_1\ \tilde{\mathcal{R}}\ \tilde{C}'_2$.
- $\tilde{C}_1 \xmapsto{\mathsf{call}(\alpha,D)} \tilde{C}'_1$ or $\tilde{C}_1 \xmapsto{\mathsf{ret}(D[\vec{\alpha}])} \tilde{C}'_1$. Similar to the previous cases.

For the case where $C_2 = (\!|\bot|\!)$ we can replay the same arguments as in the cases above.
Finally, if $\mathcal{R}$ is a weak bisimulation then $\mathcal{R}, \mathcal{R}^{-1}$ are weak simulations. Moreover:

$$\begin{aligned}
\tilde{\mathcal{R}}^{-1} &= \{(\widetilde{(C_2, K_2)}, \widetilde{(C_1, K_1)}) \mid \exists \Sigma, \beta.\ \beta \xrightarrow{K_1,K_2}{}^*_\Sigma \diamond \wedge C_1\ \mathcal{R}_{\Sigma,\beta}\ C_2\} \\
&= \{(\widetilde{(C_2, K_2)}, \widetilde{(C_1, K_1)}) \mid \exists \Sigma, \beta.\ \beta^{-1} \xrightarrow{K_2,K_1}{}^*_{\Sigma^{-1}} \diamond \wedge C_2\ \mathcal{R}^{-1}_{\Sigma^{-1},\beta^{-1}}\ C_1\} \\
&= \{(\widetilde{(C_1, K_1)}, \widetilde{(C_2, K_2)}) \mid \exists \Sigma, \beta.\ \beta \xrightarrow{K_1,K_2}{}^*_\Sigma \diamond \wedge C_1\ \mathcal{R}^{-1}_{\Sigma,\beta}\ C_2\}
\end{aligned}$$

and, hence, both $\tilde{\mathcal{R}}, \tilde{\mathcal{R}}^{-1}$ are weak simulations. □

D Proof of completeness

Thm. 32 The following is a weak (pushdown) simulation:

$$\begin{aligned}
\mathcal{R} = \{\ & (C_1, C_2, \Sigma, \beta) \mid\ \Sigma \text{ sat-connected} \wedge (C_1, C_2, \Sigma, \beta) \text{ compatible} \\
& \wedge \forall K_i.\ \beta \xrightarrow{K_1,K_2}{}^*_\Sigma \diamond \implies (\widetilde{C_1, K_1}) \mathrel{\widetilde{\sqsubseteq}} (\widetilde{C_2, K_2}) && \text{(A)} \\
& \wedge \forall C'_i, K_i.\ \ulcorner C'_1, C'_2 \urcorner \xrightarrow{K_1,K_2}{}^*_\Sigma \diamond \implies (\widetilde{C'_1, K_1}) \mathrel{\widetilde{\sqsubseteq}} (\widetilde{C'_2, K_2}) && \text{(A*)} \\
& \wedge \beta \neq \diamond \implies \exists C'_1, C'_2.\ \beta = \ulcorner C'_1, C'_2 \urcorner \wedge (C'_1, C'_2) \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1, C_2)\ \} && \text{(B)}
\end{aligned}$$

Proof. Let $C_1\ \mathcal{R}_{\Sigma,\beta}\ C_2$, for $C_1 \neq (\!|\bot|\!)$, and pick some $\beta \xrightarrow{K_1,K_2}{}^*_\Sigma \diamond$. By (A) we have $\tilde{C}_1 \sqsubseteq\!\!\!\approx \tilde{C}_2$. If $C_1\downarrow$ then $C_1.\mathcal{E} = \diamond$ and, by compatibility, $\beta = \diamond$. But then $K_1 = \cdot$ and therefore $\tilde{C}_1\downarrow$, so $\tilde{C}_2\downarrow$. Again by compatibility we get $C_2.\mathcal{E} = \diamond$ hence $C_2\downarrow$. Suppose now $C_1 \overset{\eta}{\mapsto} C_1'$. Cases:

- $\eta = \tau$. Then, $\tilde{C}_1 \overset{\tau}{\mapsto} \tilde{C}_1' = \widetilde{(C_1', K_1)}$ and hence, by τ-closure, $\tilde{C}_1' \sqsubseteq\!\!\!\approx \tilde{C}_2$. It suffices to show that $C_1'\ \mathcal{R}_{\Sigma,\beta}\ C_2$. Condition (A*) holds as Σ has not changed, while (B) follows from the hypothesis and Thm. 29. Finally, for (A) we use Thm. 31.
- $\eta = \mathsf{call}(\alpha, D)$. Then, $\tilde{C}_1 \xmapsto{\mathsf{call}(\alpha,D)} \tilde{C}_1' = \widetilde{(C_1', K_1)}$ and hence $\tilde{C}_2 \xRightarrow{\mathsf{call}(\alpha,D)} \tilde{C}_2'$ with $\tilde{C}_1' \sqsubseteq\!\!\!\approx \tilde{C}_2'$. We obtain $C_2 \xRightarrow{\mathsf{call}(\alpha,D)} C_2'$, with $\tilde{C}_2' = \widetilde{(C_2', K_2)}$, and it suffices to show conditions (A) to (B). These are shown as in the previous case above.
- $\eta = \underline{\mathsf{ret}}(D[\vec{\alpha}])$ with $\vec{\alpha} \# C_2$. Let $\vec{\alpha}' \# K_1, K_2, \vec{\alpha}$ be of the same length as $\vec{\alpha}$. Then, $\tilde{C}_1 \xmapsto{\underline{\mathsf{ret}}(D[\vec{\alpha}'])} \tilde{C}_1'$ and hence $\tilde{C}_2 \xrightarrow{\underline{\mathsf{ret}}(D[\vec{\alpha}'])} \tilde{C}_2'$ with $\tilde{C}_1' \sqsubseteq\!\!\!\approx \tilde{C}_2'$. We obtain $C_2 \xrightarrow{\underline{\mathsf{ret}}(D[\vec{\alpha}])} C_2'$, where $\tilde{C}_i' = \widetilde{(\pi \cdot C_i', K_i)}$ and $\pi = (\vec{\alpha}\ \vec{\alpha}')$ (permute component-wise $\vec{\alpha}$ with $\vec{\alpha}'$). We need to show that $C_1'\ \mathcal{R}_{\Sigma,\beta}\ C_2'$, and in particular show conditions (A) to (B). Condition (A*) still holds as Σ has not changed. For (A), we use Thm. 31. For (B), assuming $\beta \neq \diamond$, by hypothesis there are C_1'', C_2'' such that $\beta = \ulcorner C_1'', C_2''\urcorner$ and $(C_1'', C_2'') \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1, C_2)$. By Thm. 29 we have $(C_1, C_2) \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1', C_2')$, and we then use rule Trans.
- $\eta = \underline{\mathsf{call}}(i, D[\vec{\alpha}])$ with $\vec{\alpha} \# C_2$. Let $\vec{\alpha}' \# K_1, K_2, \vec{\alpha}$ be of the same length as $\vec{\alpha}$. Then, setting $\mathcal{E}_i = C_i.\mathcal{E}$, $\tilde{C}_1 \xmapsto{\underline{\mathsf{call}}(i,D[\vec{\alpha}'])} \tilde{C}_1'$ and, hence, $\tilde{C}_2 \xrightarrow{\underline{\mathsf{call}}(i,D[\vec{\alpha}'])} \tilde{C}_2'$ with $\tilde{C}_1' \sqsubseteq\!\!\!\approx \tilde{C}_2'$. We obtain $C_2 \xrightarrow{\underline{\mathsf{call}}(i,D[\vec{\alpha}])} C_2'$ with $\tilde{C}_i' = \widetilde{((\vec{\alpha}\ \vec{\alpha}') \cdot C_i', (\mathcal{E}_i, K_i))}$. We set $\beta' = \ulcorner C_1', C_2'\urcorner$ and $\Sigma' = \Sigma[\beta' \xrightarrow{\mathcal{E}_1,\mathcal{E}_2} \beta]$. To verify that Σ' is sat-connected, by equivariance of Σ (and of the sat-LTS) it suffices to show that the edge $\beta' \xrightarrow{\mathcal{E}_1,\mathcal{E}_2} \beta$ preserves it. By condition (B) (on Σ) we have $\beta = \ulcorner C_{10}, C_{20}\urcorner$ and $(C_{10}, C_{20}) \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1, C_2)$ and, by Thm. 29, $(C_1, C_2) \xrightarrow[\mathsf{SAT}]{(\mathcal{E}_1,\mathcal{E}_2)} (C_1', C_2')$, so $(C_{10}, C_{20}) \xrightarrow[\mathsf{SAT}]{\varepsilon} \cdot \xrightarrow[\mathsf{SAT}]{(\mathcal{E}_1,\mathcal{E}_2)} (C_1', C_2')$ as required.

It remains to show $C_1'\ \mathcal{R}_{\Sigma',\beta'}\ C_2'$, and in particular that conditions (A) to (B) hold. For (B), we have that $\beta' = \ulcorner C_1', C_2'\urcorner$ so we require that $(C_1', C_2') \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1', C_2')$, which is trivial. We also note that (A) follows from (A*) since $\beta' = \ulcorner C_1', C_2'\urcorner$. For (A*), let β'', C_i'', K_i'' be such that $\beta'' = \ulcorner C_1'', C_2''\urcorner \xrightarrow{K_1'',K_2''}{}^*_{\Sigma'} \diamond$. We show that $\widetilde{(C_1'', K_1'')} \sqsubseteq\!\!\!\approx \widetilde{(C_2'', K_2'')}$ using rule induction on $\beta'' \xrightarrow{K_1'',K_2''}{}^*_{\Sigma'} \diamond$. Since $\beta'' \neq \diamond$, we must have:

$$\frac{\hat{\beta} \xrightarrow{\hat{K}_1,\hat{K}_2}{}^*_{\Sigma'} \diamond \qquad \beta'' \xrightarrow{\hat{\mathcal{E}}_1,\hat{\mathcal{E}}_2}{}_{\Sigma'} \hat{\beta}}{\beta'' \xrightarrow{K_1'',K_2''}{}^*_{\Sigma'} \diamond}$$

for some $\hat{\beta}, \hat{K}_i, \hat{\mathcal{E}}_i$ with $K_i'' = \hat{\mathcal{E}}_i, \hat{K}_i$. In the base case, $\hat{\beta} = \hat{\mathcal{E}}_i = \diamond$ and $\hat{K}_i = K_i'' = \cdot$. If $\beta'' \xrightarrow{\hat{\mathcal{E}}_1,\hat{\mathcal{E}}_2}{}_\Sigma \hat{\beta}$ then $\beta'' \xrightarrow{K_1'',K_2''}{}^*_\Sigma \diamond$ and the claim follows by (A*) applied on Σ (as $C_1\ \mathcal{R}_{\Sigma,\beta}\ C_2$). Otherwise, we must have $\beta'' = \pi \cdot \beta'$ (for some π) and $\beta = \mathcal{E}_i = \diamond$, and so the claim follows from $\tilde{C}_1' \sqsubseteq\!\!\!\approx \tilde{C}_2'$ and closure under π (as $C_i' = C_i''$). Suppose now $\hat{\beta} \neq \diamond$. By sat-connectedness, $\hat{\beta} = \ulcorner \hat{C}_1, \hat{C}_2\urcorner$ for some $\hat{C}_i$ such that $(\hat{C}_1, \hat{C}_2) \xrightarrow[\mathsf{SAT}]{\varepsilon} \cdot \xrightarrow[\mathsf{SAT}]{(\hat{\mathcal{E}}_1,\hat{\mathcal{E}}_2)} (C_1'', C_2'')$. By induction hypothesis we have $\widetilde{(\hat{C}_1, \hat{K}_1)} \sqsubseteq\!\!\!\approx \widetilde{(\hat{C}_2, \hat{K}_2)}$, so by Thm. 31 we have $\widetilde{(C_1'', K_1'')} \sqsubseteq\!\!\!\approx \widetilde{(C_2'', K_2'')}$.

- $\eta = \mathsf{ret}(D)$ and $\beta \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}{}_\Sigma \beta'$. Note first that, for any K_i' such that $\beta' \xrightarrow{K_1',K_2'}{}^*_\Sigma \diamond$, we get $A \cup \tilde{C}_1 \xrightarrow{\mathsf{ret}(D)} \tilde{C}_1'$ and hence $A \cup \tilde{C}_2 \xRightarrow{\mathsf{ret}(D)} \tilde{C}_2'$ with $\tilde{C}_1' \sqsubseteq\!\!\!\approx \tilde{C}_2'$, where $\tilde{C}_i = \widetilde{(C_i, (\mathcal{E}_i, K_i'))}$ and $\tilde{C}_1' = A \cup \widetilde{(C_1'[\mathcal{E}_1/\chi], K_1')}$ (by compatibility, these are all defined). Picking now any such K_1', K_2', we obtain $C_2 \xRightarrow{\mathsf{ret}(D[\vec{\alpha}])} C_2'$ for C_2' such that $\tilde{C}_2' = A \cup \widetilde{(C_2'[\mathcal{E}_2/\chi], K_2')}$, so it remains to show that $C_1'[\mathcal{E}_1/\chi]\ \mathcal{R}_{\Sigma',\beta'}\ C_2'[\mathcal{E}_2/\chi]$ for $\Sigma' = \Sigma@\beta'$. Condition (A*) still holds as Σ' is a subgraph of Σ; while we saw above that condition (A) holds ($\tilde{C}_1' \sqsubseteq\!\!\!\approx \tilde{C}_2'$ is true for any valid choice of K_i'). For (B), if $\beta' \neq \diamond$ then $\beta \neq \diamond$. Hence, condition (B) on $C_1\ \mathcal{R}_{\Sigma,\beta}\ C_2$ implies $(C_{10}, C_{20}) \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1, C_2)$ for $\beta = \ulcorner C_{10}, C_{20}\urcorner$. Since Σ is

sat-connected, assuming $\beta' = \ulcorner C_1'', C_2'' \urcorner$, we get

$$(C_1'', C_2'') \xrightarrow[\mathsf{SAT}]{\varepsilon} \cdot \xrightarrow[\mathsf{SAT}]{(\mathcal{E}_1,\mathcal{E}_2)} (C_{10}, C_{20})$$

and by Thm. 29 we obtain $(C_1'', C_2'') \xrightarrow[\mathsf{SAT}]{\varepsilon} (C_1'[\mathcal{E}_1/\chi], C_2'[\mathcal{E}_2/\chi])$. □

E Theory of Enhancements

Here we present additional definitions and results from [58, 57] omitted from Sec. 6. The main result of this section is a set of proof obligations with which we can proof an up-to technique sound, shown in Thm. 59.

Definition 52. Consider monotone functions $f, g : \mathcal{P}(X) \to \mathcal{P}(X)$ on some set X. We write $f \circ g$ for the composition of f and g, and $f \sqcup g$ for the function $\mathcal{S} \mapsto f(\mathcal{S}) \sqcup g(\mathcal{S})$. For any set F of functions, we write $\bigsqcup F$ for the function $\mathcal{S} \mapsto \bigcup_{f \in F} f(\mathcal{S})$. We also write c_X to be the constant function with range $\{X\}$. We let $f^0 \stackrel{\text{def}}{=} \mathsf{id}$ and $f^{n+1} \stackrel{\text{def}}{=} f \circ f^n$. Moreover, we write f^ω to mean $\bigsqcup_{k<\omega} f^k$. We write $f \sqsubseteq g$ when, for all $\mathcal{S} \in \mathcal{P}(X)$, $f(\mathcal{S}) \subseteq g(\mathcal{S})$.

Lemma 53 ([58], Lem. 6.3.12)**.** *$f \overset{\mathbf{wp}}{\rightsquigarrow} g$ if and only if for all $\mathcal{R} \overset{\mathbf{wp}}{\rightsquigarrow} \mathcal{S}$ we have $f \circ \mathbf{wp}(\mathcal{R}) \subseteq \mathbf{wp} \circ g(\mathcal{S})$.* □

Lemma 54 ([58], Prop. 6.3.11 and 6.3.12)**.** *The following functions are* **wp***-compatible:*

- *the reflexive $\mathsf{c}_{\mathsf{refl}}$ and identity id functions;*
- *$f \circ g$, for any* **wp***-compatible monotone functions f, g;*
- *$\bigsqcup F$, for any set F of* **wp***-compatible monotone functions.* □

Pous [57] extends the theory of enhancements with the notion of companion of **wp**, the largest **wp**-compatible function.

Definition 55 (Companion)**.** $\mathbf{t_{wp}} \stackrel{\text{def}}{=} \bigsqcup\{f : \mathcal{P}(X) \to \mathcal{P}(X) \mid f \overset{\mathbf{wp}}{\rightsquigarrow} f\}$.

Lemma 56 ([57])**.**

(1) $\mathbf{t_{wp}}$ *is* **wp***-compatible:* $\mathbf{t_{wp}} \rightsquigarrow \mathbf{t_{wp}}$*;*
(2) **wp** *is* **wp***-compatible:* $\mathbf{wp} \sqsubseteq \mathbf{t_{wp}}$*;*
(3) $\mathbf{t_{wp}}$ *is idempotent:* $\mathsf{id} \sqsubseteq \mathbf{t_{wp}}$ *and* $\mathbf{t_{wp}} \circ \mathbf{t_{wp}} \sqsubseteq \mathbf{t_{wp}}$*;*
(4) $\mathbf{t_{wp}}$ *is* **wp***-sound:* $\mathsf{gfp}(\mathbf{wp} \circ \mathbf{t_{wp}}) \subseteq \mathsf{gfp}(\mathbf{wp})$. □

This gives rise a proof technique for proving up-to techniques sound.

Lemma 57. *Let $f \sqsubseteq \mathbf{t_{wp}}$. Then f is* **wp***-sound.*

Proof. By showing that $f \cup \mathbf{t_{wp}} \overset{\mathbf{wp}}{\rightsquigarrow} f \cup \mathbf{t_{wp}}$ and using Thm. 39. □

Lemma 58 (Function Composition Laws)**.** *Consider monotone functions $f, g, h : \mathcal{P}(X) \to \mathcal{P}(X)$ and set $\mathcal{S} \in \mathcal{P}(X)$. We have*

(1) $\mathsf{c}_{\mathcal{S}} \circ f = \mathsf{c}_{\mathcal{S}}$
(2) $(f \sqcup g) \circ h = (f \circ h) \sqcup (g \circ h)$
(3) $h \circ (f \sqcup g) = (h \circ f) \sqcup (h \circ g)$
(4) $(f \sqcup g) \sqsubseteq (f \sqcup h)$ *and* $(f \circ g) \sqsubseteq (f \circ h)$ *and* $(g \circ f) \sqsubseteq (h \circ f)$*, when $g \sqsubseteq h$.*
(5) $f \sqsubseteq f^\omega$ *and* $f \circ f^\omega = f^\omega \circ f \sqsubseteq f^\omega \circ f^\omega \sqsubseteq f^\omega$.
(6) $f^\omega \circ g = \bigsqcup_{i<\omega}(f^i \circ g)$. □

We distil this up-to technique to the following three proof obligations, each sufficient for proving the soundness of up-to techniques.

Lemma 59 (POs for Up-To Soundness)**.** *Let f be a monotone function and $\mathcal{R}$ be a weak simulation; f is* **wp***-sound when one of the following holds:*

(1) $f \overset{\mathbf{wp}}{\rightsquigarrow} f$*; or*
(2) $f \overset{\mathbf{wp}}{\rightsquigarrow} f^\omega$*; or*
(3) $f \overset{\mathbf{wp}}{\rightsquigarrow} (f \circ g)$*, for some $g \sqsubseteq \mathbf{t_{wp}}$; or*
(4) $f = \bigsqcup_{f_i \in F} f_i \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})}$*, where F is a set of monotone functions and, for all $f_i \in F$, there exists $g_i \sqsubseteq \mathbf{t_{wp}}$ such that $f_i \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} \overset{\mathbf{wp}}{\rightsquigarrow} (f \sqcup g_i)^\omega \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})}$.*

Proof.

(1) By Thm. 39.

(2) By Thm. 57, it suffices to show $f \sqsubseteq \mathbf{t_{wp}}$. Because $f \sqsubseteq f^\omega$, it suffices to show that $f^\omega \overset{\mathbf{wp}}{\rightsquigarrow} f^\omega$. This is proven by showing that for all k,

$$(P(k)) \qquad f^k \overset{\mathbf{wp}}{\rightsquigarrow} f^\omega.$$

We proceed by induction on k. The base case is straightforward:

$$f^0 \circ \mathbf{wp} = \mathsf{id} \circ \mathbf{wp} = \mathbf{wp} \circ \mathsf{id} = \mathbf{wp} \circ f^0 \sqsubseteq \mathbf{wp} \circ f^\omega$$

In the inductive case we assume $P(k)$ and prove $P(k+1)$ as follows:

$$\begin{aligned} f^{k+1} \circ \mathbf{wp} &= f \circ f^k \circ \mathbf{wp} \\ &\sqsubseteq f \circ \mathbf{wp} \circ f^\omega && (P(k)) \\ &\sqsubseteq \mathbf{wp} \circ f^\omega \circ f^\omega && (f \overset{\mathbf{wp}}{\rightsquigarrow} f^\omega) \\ &= \mathbf{wp} \circ f^\omega \end{aligned}$$

(3) By Thm. 57, it suffices to show $f \sqsubseteq \mathbf{t_{wp}}$. Because $f \sqsubseteq f \circ (\mathsf{id} \sqcup \mathbf{t_{wp}}) \sqsubseteq f \circ \mathbf{t_{wp}}$, it suffices to show $f \circ \mathbf{t_{wp}} \overset{\mathbf{wp}}{\rightsquigarrow} f \circ \mathbf{t_{wp}}$ by unfolding definitions and the premise:

$$f \circ \mathbf{t_{wp}} \circ \mathbf{wp} \sqsubseteq f \circ \mathbf{wp} \circ \mathbf{t_{wp}} \sqsubseteq \mathbf{wp} \circ f \circ g \circ \mathbf{t_{wp}} \sqsubseteq \mathbf{wp} \circ f \circ \mathbf{t_{wp}}.$$

(4) Let $g = \bigsqcup_{f_i \in F} g_i$. By Thm. 57, it suffices to show $f \sqsubseteq \mathbf{t_{wp}}$. Because

$$\begin{aligned} f_i \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} &= f_i \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} \sqsubseteq f \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} \sqsubseteq (f \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})}) \sqcup (\mathbf{t_{wp}} \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})}) \\ &= (f \sqcup \mathbf{t_{wp}}) \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} \sqsubseteq (f \sqcup \mathbf{t_{wp}})^\omega \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} \end{aligned}$$

it suffices to show that $(f \sqcup \mathbf{t_{wp}})^\omega \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} \overset{\mathbf{wp}}{\rightsquigarrow} (f \sqcup \mathbf{t_{wp}})^\omega \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})}$. This is proven by showing that for all k,

$$(P(k)) \qquad (f \sqcup \mathbf{t_{wp}})^k \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} \overset{\mathbf{wp}}{\rightsquigarrow} (f \sqcup \mathbf{t_{wp}})^\omega \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})}.$$

We proceed by induction on k. The base case is straightforward:

$$\mathsf{id} \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} \circ \mathbf{wp} = \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} = \mathbf{wp} \circ \mathsf{id} \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} \sqsubseteq \mathbf{wp} \circ (f \sqcup \mathbf{t_{wp}})^\omega \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})}$$

In the inductive case we assume $P(k)$ and prove $P(k+1)$ as follows:

$$\begin{aligned} &(f \sqcup \mathbf{t_{wp}})^{k+1} \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} \circ \mathbf{wp} \\ &= (f \sqcup \mathbf{t_{wp}}) \circ (f \sqcup \mathbf{t_{wp}})^k \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} \circ \mathbf{wp} \\ &\sqsubseteq (f \sqcup \mathbf{t_{wp}}) \circ \mathbf{wp} \circ h && (P(k),\ h = (f \sqcup \mathbf{t_{wp}})^\omega \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})}) \\ &\sqsubseteq (f \circ \mathbf{wp} \circ h) \sqcup (\mathbf{t_{wp}} \circ \mathbf{wp} \circ h) && (\text{Thm. } 58) \\ &= \left(\bigsqcup_{f_i \in F} (f_i \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} \circ \mathbf{wp} \circ h) \right) \sqcup (\mathbf{t_{wp}} \circ \mathbf{wp} \circ h) && (\text{definition of } f \text{ and Thm. } 58) \\ &\sqsubseteq \left(\bigsqcup_{f_i \in F} (\mathbf{wp} \circ (f \sqcup g_i)^\omega \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} \circ h) \right) \sqcup (\mathbf{t_{wp}} \circ \mathbf{wp} \circ h) && (\text{premise}) \\ &\sqsubseteq \left(\bigcup_{f_i \in F} (\mathbf{wp} \circ (f \sqcup \mathbf{t_{wp}})^\omega \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})}) \right) \sqcup (\mathbf{t_{wp}} \circ \mathbf{wp} \circ h) && (\text{Thm. } 58 \text{ and premise on } g_i) \\ &\sqsubseteq (\mathbf{wp} \circ (f \sqcup \mathbf{t_{wp}})^\omega \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})}) \sqcup (\mathbf{wp} \circ \mathbf{t_{wp}} \circ h) && (\text{Thm. } 56\ (1)) \\ &\sqsubseteq (\mathbf{wp} \circ \mathsf{id} \circ h) \sqcup (\mathbf{wp} \circ \mathbf{t_{wp}} \circ h) && (\text{definition of } h) \\ &= \mathbf{wp} \circ (\mathsf{id} \sqcup \mathbf{t_{wp}}) \circ h && (\text{Thm. } 58) \\ &= \mathbf{wp} \circ \mathbf{t_{wp}} \circ h && (\text{Thm. } 56\ (3)) \\ &\sqsubseteq \mathbf{wp} \circ (f \sqcup \mathbf{t_{wp}}) \circ h && (\text{Thm. } 58) \\ &\sqsubseteq \mathbf{wp} \circ (f \sqcup \mathbf{t_{wp}})^\omega \circ \mathsf{c}_{\mathsf{gfp}(\mathbf{wp})} && (\text{Thm. } 58 \text{ and definition of h }) \end{aligned}$$

$$\text{UpToPerm}\quad \frac{C_1\ \mathcal{R}_{\Sigma,\beta}\ C_2 \qquad \pi_1 = \pi_{l_1}\pi_\alpha\pi_i \qquad \pi_2 = \pi_{l_2}\pi_\alpha\pi_i}{\pi_1 \cdot C_1\ \mathsf{perm}(\mathcal{R})_{(\pi_1,\pi_2)\cdot\Sigma,(\pi_1,\pi_2)\cdot\beta}\ \pi_2 \cdot C_2}$$

Figure 6. Up to Permutations.

□

As we are only interested in weak progression, in the following we drop the **wp** annotation from progressions, compatibility and companion.

F Simple Up-To Techniques

We develop our up-to techniques using the theory of bisimulation enhancements from [58, 57] (see Appendix E). We start by presenting three straightforward up-to techniques which are needed to reduce the configurations considered by bisimulation, achieving finite LTSs in many examples. These techniques are *up to permutations*, *beta reductions*, *garbage collection*, and *weakening of knowledge environments*. To present these techniques we first need the following definitions.

F.1 Up to Permutations

Definition 60 (Permutations)**.** We consider permutations of store locations, π_l, abstract names, π_α and environment indices, π_i, respectively. We use juxtaposition to denote permutation composition.

When applying a permutation π_i to an environment Γ, it only acts on its domain; other types of permutations only act on the codomain of Γ. When applying a permutation π_l to a store s, the former acts on both the domain and range of the latter; a permutation π_α acts on the codomain of s, and a permutation π_i leaves s unaffected. When $\pi_1 = \pi_{l_1}\pi_\alpha\pi_i$ and $\pi_2 = \pi_{l_2}\pi_\alpha\pi_i$ and $\beta = \ulcorner C_1, C_2 \urcorner$, we define

$$(\pi_1,\pi_2)\cdot\beta = \ulcorner \pi_1 C_1, \pi_2 C_2 \urcorner$$
$$(\pi_1,\pi_2)\cdot\Sigma = \{((\pi_1,\pi_2)\cdot\beta', \pi_1\cdot\mathcal{E}_1, \pi_2\cdot\mathcal{E}_2, (\pi_1,\pi_2)\cdot\beta) \mid (\beta',\mathcal{E}_1,\mathcal{E}_2,\beta)\in\Sigma\}$$

Moreover if $\pi_1 = \pi_2 = \pi$ we write $\pi\cdot\beta$ and $\pi\cdot\Sigma$ to mean $(\pi,\pi)\cdot\beta$ and $(\pi,\pi)\cdot\Sigma$, respectively.

Note that call graphs Σ are closed under π_α permutations, and thus are unaffected by such. However they are affected by π_i and π_l permutations.

Lemma 61 (Permutation Invariance for Reductions)**.** *Let π_l, π_α, π_i be permutations on locations, abstract names and indices respectively, and $\pi = \pi_l\pi_\alpha\pi_i$. If $\langle s\,;e\rangle \hookrightarrow \langle s'\,;e'\rangle$ then $\pi\cdot\langle s\,;e\rangle \hookrightarrow \pi\cdot\langle s'\,;e'\rangle$. Moreover, if $\langle s\,;e\rangle \to \langle s'\,;e'\rangle$ then $\pi\cdot\langle s\,;e\rangle \to \pi\cdot\langle s'\,;e'\rangle$.*

Proof. By nominal sets reasoning (all reduction rules are closed under permutation). □

Lemma 62. *Let π_l, π_α, and π_i be permutations on locations, abstract names, and indices, respectively, and $\pi = \pi_l\pi_\alpha\pi_i$. If $C \xrightarrow{\eta} C'$ then $\pi\cdot C \xrightarrow{\pi_\alpha\pi_i\cdot\eta} \pi\cdot C'$.*

Proof. By nominal sets reasoning (all transition rules are closed under permutation). □

Lemma 63. *Let $C \xrightarrow{\eta} C'$; then for all finite L_0, A_0, I_0 there exists π such that $C \xrightarrow{\pi\cdot\eta} \pi\cdot C'$ and*

$$(\mathsf{an}(\pi\cdot C') \setminus \mathsf{an}(C)) \cap A_0 = (\mathsf{dom}(\pi\cdot C'.s) \setminus \mathsf{dom}(C.s)) \cap L_0$$
$$= (\mathsf{dom}(\pi\cdot C'.\Gamma) \setminus \mathsf{dom}(C.\Gamma)) \cap I_0 = \emptyset$$

Proof. By Thm. 62, picking permutations π that rename new names in C' that do not exist in C to fresh ones, and observing that $\pi\cdot C = C$. □

Corollary 64. *Let $C \xRightarrow{\eta} C'$; then for all finite L_0, A_0, I_0 there exists π such that $C \xrightarrow{\pi\cdot\eta} \pi\cdot C'$ and*

$$(\mathsf{an}(\pi\cdot C') \setminus \mathsf{an}(C)) \cap A_0 = (\mathsf{dom}(\pi\cdot C'.s) \setminus \mathsf{dom}(C.s)) \cap L_0$$
$$= (\mathsf{dom}(\pi\cdot C'.\Gamma) \setminus \mathsf{dom}(C.\Gamma)) \cap I_0 = \emptyset$$

Proof. By induction on the length of the transition from C, using Thm. 63. □

Lemma 65. *Function* perm *is a sound up-to technique.*

$$\text{UpToBeta}\quad \frac{C_1' \ \mathcal{R}_{\Sigma,\beta}\ C_2' \qquad C_1 \overset{\tau}{\mapsto}_{\beta} C_1' \text{ or } C_1 = C_1' \qquad C_2 \overset{\tau}{\mapsto}_{\beta} C_2' \text{ or } C_2 = C_2'}{C_1 \ \mathsf{beta}(\mathcal{R})_{\Sigma,\beta}\ C_2}$$

Figure 7. Up to Beta Moves.

Proof. From Thm. 59 (1), it suffices to show that perm is compatible; i.e., $\mathsf{perm}(\mathbf{wp}(\mathcal{R})) \subseteq \mathbf{wp}(\mathsf{perm}(\mathcal{R}))$, for any configuration relation $\mathcal{R}$.

Let $C_1\ \mathbf{wp}(\mathcal{R})_{\Sigma,\beta}\ C_2$ and $\pi_1 \cdot C_1\ \mathsf{perm}(\mathcal{R})_{(\pi_1,\pi_2)\cdot\Sigma,(\pi_1,\pi_2)\cdot\beta}\ \pi_2 \cdot C_2$, where $\pi_1 = \pi_{l1}\pi_\alpha\pi_i$ and $\pi_2 = \pi_{l2}\pi_\alpha\pi_i$. Moreover, let $\pi_1 \cdot C_1 \overset{\eta}{\mapsto} C_1'$. Because of $\pi_1\pi_1 = \mathsf{id}$ and Thm. 62 we get $C_1 \overset{\pi_\alpha\pi_i\cdot\eta}{\longmapsto} C_1'\pi_1$. We proceed by definition of $\mathbf{wp}(\mathcal{R})$, taking cases on η.

When $\eta \in \{\tau, \mathsf{call}(\alpha, D), \underline{\mathsf{ret}}(D[\vec{\alpha}]) \mid \vec{\alpha} \ \#\ C_2\}$, there exists C_2' such that $C_2 \overset{\pi_\alpha\pi_i\cdot\eta}{\Longrightarrow} C_2'$ and $\pi_1 \cdot C_1'\ \mathcal{R}_{\Sigma,\beta}\ C_2'$. By Thm. 62, $\pi_2 \cdot C_2 \overset{\eta}{\Rightarrow} \pi_2 \cdot C_2'$, and by definition of $\mathsf{perm}(\mathcal{R})$: $C_1'\ \mathsf{perm}(\mathcal{R})_{(\pi_1,\pi_2)\cdot\Sigma,(\pi_1,\pi_2)\cdot\beta}\ \pi_2 \cdot C_2'$.

When $\eta = \underline{\mathsf{call}}(i, D[\vec{\alpha}])$, there exists C_2' such that $C_2 \overset{\pi_\alpha\pi_i\cdot\eta}{\Longrightarrow} C_2'$ and $\pi_1 \cdot C_1'\ \mathcal{R}_{\Sigma',\beta'}\ C_2'$ with $\beta' = \ulcorner \pi_1 \cdot C_1', C_2' \urcorner$ and $\Sigma' = \Sigma[\beta' \mapsto (\pi_1 \cdot C_1.\mathcal{E}, C_2.\mathcal{E}, \beta)]$. By Thm. 62, $\pi_2 \cdot C_2 \overset{\eta}{\Rightarrow} \pi_2 \cdot C_2'$ and by definition of $\mathsf{perm}(\mathcal{R})$: $C_1'\ \mathcal{R}_{(\pi_1,\pi_2)\cdot\Sigma',(\pi_1,\pi_2)\cdot\beta'}\ \pi_2 \cdot C_2'$.

When $\eta = \mathsf{ret}(D)$, there exists $(\mathcal{E}_1, \mathcal{E}_2, \beta') \in \Sigma(\beta)$ and exists C_2' such that $C_2 \overset{\pi_\alpha\pi_i\cdot\eta}{\Longrightarrow} C_2'$ and $\pi_1 \cdot C_1'[\mathcal{E}_1/\chi]\ \mathcal{R}_{\Sigma',\beta'}\ C_2'[\mathcal{E}_2/\chi]$ with $\Sigma' = \Sigma @ \beta'$. By Thm. 62, $\pi_2 \cdot C_2 \overset{\eta}{\Rightarrow} \pi_2 \cdot C_2'$, and by definition of $\mathsf{perm}(\mathcal{R})$:

$$C_1'[\pi_1 \cdot \mathcal{E}_1/\chi] = \pi_1 \cdot (\pi_1 \cdot C_1'[\mathcal{E}_1/\chi])\ \mathsf{perm}(\mathcal{R})_{(\pi_1,\pi_2)\cdot\Sigma',(\pi_1,\pi_2)\cdot\beta'}\ \pi_2 \cdot (C_2'[\mathcal{E}_2/\chi]) = \pi_2 \cdot C_2'[\pi_2 \cdot \mathcal{E}_2/\chi]$$

with $(\pi_1, \pi_2) \cdot \beta' = \ulcorner C_1', \pi_2 \cdot C_2' \urcorner$ and

$$(\pi_1, \pi_2) \cdot \Sigma' = (\pi_1, \pi_2) \cdot \Sigma[(\pi_1, \pi_2) \cdot \beta' \mapsto (\pi_1 \cdot C_1.\mathcal{E}, \pi_2 \cdot C_2.\mathcal{E}, (\pi_1, \pi_2) \cdot \beta)]$$

□

F.2 Up to Beta Moves

Lemma 66 (beta-move)**.** *Any τ-transition $C \overset{\tau}{\mapsto} C'$ is called a* beta-move*, and we write $C \overset{\tau}{\mapsto}_{\beta} C'$, because for all transitions $C \overset{\eta}{\mapsto} C''$, we have $\eta = \tau$ and $C' = C''\pi_l$, for some location permutations π_l on the locations in $\mathsf{fl}(C', C'') \setminus \mathsf{fl}(C)$.*

Proof. By the deterministic nature of the reduction semantics. □

Lemma 67. *Function* beta *is a sound up-to technique.*

Proof. From Thm. 59 (1), it suffices to show that $\mathsf{beta} \overset{\mathbf{wp}}{\rightsquigarrow} \mathsf{beta}$. Let $C_1\ \mathsf{beta}(\mathbf{wp}(\mathcal{R}))_{\Sigma,\beta}\ C_2$ and $C_1'\ \mathbf{wp}(\mathcal{R})_{\Sigma,\beta}\ C_2'$ and $C_1 \overset{\tau}{\mapsto}_{\beta} C_1'$ or $C_1 = C_1'$ and $C_2 \overset{\tau}{\mapsto}_{\beta} C_2'$ or $C_2 = C_2'$.

When $C_1 \overset{\tau}{\mapsto}_{\beta} C_1'$, then by Thm. 66 C_1 can only take this real transition. This can be matched with the transition $C_2 \overset{\tau}{\Rightarrow} C_2'$. Moreover, $C_1'\ \mathsf{beta}(\mathbf{wp}(\mathcal{R}))_{\Sigma,\beta}\ C_2'$ by definition.

When $C_1 = C_1'$, let $C_1 \overset{\eta}{\mapsto} C_1''$, thus $C_1' \overset{\eta}{\mapsto} C_1''$. By definition of $\mathbf{wp}(\mathcal{R})$, there exists C_4,Σ',β',$\mathcal{E}_1$,$\mathcal{E}_2$ such that $C_2' \overset{\eta}{\Rightarrow} C_2''$ and $C_1''\ \mathcal{R}_{\Sigma',\beta'}\ C_2''$ or $C_1''[\mathcal{E}_1/\chi]\ \mathcal{R}_{\Sigma',\beta'}\ C_2''[\mathcal{E}_2/\chi]$. Moreover, $C_2 \overset{\tau}{\Rightarrow} C_2' \overset{\eta}{\Rightarrow} C_2''$. and $C_1''\ \mathsf{beta}(\mathcal{R})_{\Sigma',\beta'}\ C_2''$ or $C_1''[\mathcal{E}_1/\chi]\ \mathsf{beta}(\mathcal{R})_{\Sigma',\beta'}\ C_2''[\mathcal{E}_2/\chi]$ by definition. □

F.3 Up to Garbage Collection

Definition 68 (Garbage Collection)**.** Given a set of location names S, we define the following total operation on configurations:

$$(\!|\Gamma\,;s,s_g\,;\Phi|\!) \succ^{\mathsf{gc}}_{S} \begin{cases} (\!|\Gamma\,;s\,;\Phi|\!) & \text{if } S = \mathsf{dom}(s_g),\ \mathsf{dom}(s_g) \cap \mathsf{fl}(\Gamma, s, \mathcal{E}) = \emptyset \\ (\!|\Gamma\,;s,s_g\,;\Phi|\!) & \text{otherwise} \end{cases}$$

$$\text{UpToGC}\quad \frac{C_1 \succ_{S_1}^{\mathsf{gc}} \mathcal{R}_{\Sigma^{gc}_{S_1,S_2},\ulcorner C_3',C_4'\urcorner} \prec_{S_2}^{\mathsf{gc}} C_2 \qquad C_3' \prec_{S_1}^{\mathsf{gc}} C_3 \qquad C_4' \prec_{S_2}^{\mathsf{gc}} C_4}{C_1 \ \mathsf{gc}(\mathcal{R})_{\Sigma,\ulcorner C_3,C_4\urcorner} \ C_2}$$

Figure 8. Up-to garbage collection.

$$\text{UpToWeakening}\quad \frac{C_1 \prec_i^{\mathsf{wk}} \mathcal{R}_{\Sigma,\ulcorner C_3',C_4'\urcorner} \succ_i^{\mathsf{wk}} C_2 \qquad C_3 \prec_i^{\mathsf{wk}} C_3' \qquad C_4 \prec_i^{\mathsf{wk}} C_4'}{C_1 \ \mathsf{weak_i}(\mathcal{R})_{\Sigma_i^{wk},\ulcorner C_3,C_4\urcorner} \ C_2}$$

Figure 9. Up to Weakening of the Opponent Knowledge.

Moreover $\asymp_S^{\mathsf{gc}} = \prec_S^{\mathsf{gc}} \cup \succ_S^{\mathsf{gc}}$. Given S_1, S_2, Σ with $S_i \cap \mathsf{fl}(\Sigma.\mathcal{E}_i) = \emptyset$, we also define $\Sigma^{gc}_{S_1,S_2}$ as follows:

$$\ulcorner C_{11}', C_{21}' \urcorner \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}_{\Sigma^{gc}_{S_1,S_2}} \ulcorner C_{12}', C_{22}' \urcorner$$

when $\ulcorner C_{11}, C_{21} \urcorner \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}_{\Sigma} \ulcorner C_{12}, C_{22} \urcorner$ and, for $i, j \in \{1,2\}$, we have $C_{ij}' \prec_{S_i}^{\mathsf{gc}} C_{ij}$.

Lemma 69. *If $C \asymp_S^{\mathsf{gc}} C'$ then $\pi \cdot C \asymp_{\pi\cdot S}^{\mathsf{gc}} \pi \cdot C'$.* □

Lemma 70. *Given $C_1 \asymp_S^{\mathsf{gc}} C_2$ and $(\mathsf{fl}(C_1') \setminus \mathsf{fl}(C_1)) \cap \mathsf{fl}(S) = \emptyset$:*

- *if $C_1 \xrightarrow{\eta} C_1'$ then $C_2 \xrightarrow{\eta} C_2'$ and $C_1' \asymp_S^{\mathsf{gc}} C_2'$*
- *if $C_1 \stackrel{\eta}{\Rightarrow} C_1'$ then $C_2 \stackrel{\eta}{\Rightarrow} C_2'$ and $C_1' \asymp_S^{\mathsf{gc}} C_2'$.*

Proof. First part by induction on the derivation of $C_1 \asymp_S^{\mathsf{gc}} C_2$ and case analysis on the transition from C_1. Second part by induction on the length of the transition from C_1 and using first part. □

Lemma 71. *Function* gc *is a sound up-to technique.*

Proof. By showing $\mathsf{gc} \rightsquigarrow \mathsf{gc} \circ \mathsf{perm}$ and Thm. 59 (3) and Thm.(s) 65 and 70. □

F.4 Up to Opponent Knowledge Weakening

Lemma 72. *Let C_1 and C_2 be well formed configurations with $C_2 = C_1, {}^i v$, meaning that C_2 is identical to C_1 except it contains an additional value v indexed by i in $C_2.\Gamma$. Then the following hold:*

(1) If $C_1 \xrightarrow{\eta} C_1'$, where $\eta \notin \{\mathsf{call}(\alpha, i), \mathsf{ret}(i) \mid \text{any } \alpha\}$, then $C_2 \xrightarrow{\eta} C_1', {}^i v$.
(2) If $C_2 \xrightarrow{\eta} C_2', {}^i v$, where $\eta \notin \{\underline{\mathsf{call}}(i, \alpha) \mid \text{any } \alpha\}$, then $C_1 \xrightarrow{\eta} C_2'$.

Proof. By case analysis on the transitions. □

Definition 73 (Opponent Knowledge Weakening)**.** We let $(\prec^{\mathsf{wk}})$ as follows:

- $(\!|\Gamma\,;s\,;\mathcal{E}|\!) \prec_i^{\mathsf{wk}} (\!|\Gamma, {}^i v_1\,;s\,;\mathcal{E}|\!)$
- $(\!|\Gamma\,;s\,;e|\!) \prec_i^{\mathsf{wk}} (\!|\Gamma, {}^i v_1\,;s\,;e|\!)$

  We also define Σ_i^{wk}:

$$(\mathcal{E}_1, \mathcal{E}_2, \ulcorner C_{12}', C_{22}' \urcorner) \in \Sigma_i^{wk}(\ulcorner C_{11}', C_{21}' \urcorner)$$

  when
  - $(\mathcal{E}_1, \mathcal{E}_2, \ulcorner C_{12}, C_{22} \urcorner) \in \Sigma(\ulcorner C_{11}, C_{21} \urcorner)$; and
  - for $k, j \in \{1,2\}$, we have $C_{kj}' \prec_i^{\mathsf{wk}} C_{kj}$.

Lemma 74. *Function* $\mathsf{weak_i}$ *is a sound up-to technique.*

Proof. By showing $\mathsf{weak} \overset{\mathbf{wp}}{\rightsquigarrow} \mathsf{weak} \circ \mathsf{perm}$ and Thm. 59 (3), using Thm.(s) 65 and 72. □

G Pair (Bi-)Simulation

In order to define and prove our up to separation technique, we need to extend the stackless LTS to pairs of configurations and define a notion of bisimulation over it.

Definition 75 (Pair Configuration)**.** We define pair configurations $\langle\!\langle C_1 \,;\, C_2 \rangle\!\rangle$ for all stackless LTS configurations C_1, C_2. To enable symmetric reasoning we define $\widehat{1} \stackrel{\text{def}}{=} 2$ and $\widehat{2} \stackrel{\text{def}}{=} 1$. We let $\mathbb{C}$ range over pair configurations, and write $\mathbb{C}.i$ to get the i'th inner configuration. We write $\mathbb{C}\downarrow$ when $\mathbb{C}.1\downarrow$ and $\mathbb{C}.2\downarrow$.

Definition 76 (Pair LTS)**.** We extend the LTS of Fig. 5 to pair configurations as follows:

$$\begin{array}{lll} \langle\!\langle C_1 \,;\, C_2 \rangle\!\rangle \xrightarrow{\eta,i} \langle\!\langle C_1' \,;\, C_2' \rangle\!\rangle & \text{if} & C_i \xrightarrow{\eta} C_i' \text{ and } C_{\widehat{i}} = C_{\widehat{i}}' \text{ an opponent configuration and } i \in \{1,2\} \\ \langle\!\langle C_1 \,;\, C_2 \rangle\!\rangle \xrightarrow{\eta,0} \langle\!\langle C_1' \,;\, C_2' \rangle\!\rangle & \text{if} & C_1 \xrightarrow{\eta} C_1' \text{ and } C_2 \xrightarrow{\eta} C_2' \text{ and } C_1.e = C_2.e \text{ or } C_1.\mathcal{E} = C_2.\mathcal{E} \\ & & \quad \text{and } C_1'.e = C_2'.e \text{ or } C_1'.\mathcal{E} = C_2'.\mathcal{E} \end{array}$$

We also write $\mathbb{C} \stackrel{\eta,k}{\longmapsto} \mathbb{C}'$ when the transition is derived from the above LTS, without using the Response rule of the stackless LTS in Fig. 5.

The following definition lifts Thm. 15 to the pair LTS.

Definition 77 (Pair Entry Points and Pair Continuation Graphs)**.**

$$\mathsf{PEPoint} \ni \beta\!\!\beta ::= (\beta_1, \beta_2, k) \qquad (k \in \{0,1,2\})$$

$$\mathsf{PCGrph} \ni \mathbb{\Sigma} \subseteq^{\neq\emptyset}_{\text{fin}} \mathsf{PEPoint} \times \mathsf{Kont} \times \mathsf{Kont} \times \mathsf{PEPoint}$$

and each $\mathbb{\Sigma}$ must satisfy the conditions:

- *Reachability.* For all $\beta\!\!\beta \in \mathsf{dom}(\mathbb{\Sigma})$ there are evaluation stacks K_1, K_2 such that $\beta\!\!\beta \xrightarrow{K_1,K_2}{}^{*}_{\mathbb{\Sigma}} \diamond$
- *Top and Divergence.* For all $\beta\!\!\beta'^{k'} \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}{}_{\mathbb{\Sigma}} \beta\!\!\beta^{k}$ and $j \in \{1,2\}$:

  $$(\diamond,\diamond,0) \in \mathsf{dom}(\mathbb{\Sigma}) \wedge (\beta\!\!\beta' = (\diamond,\diamond,0) \implies \beta\!\!\beta = (\diamond,\diamond,0)) \wedge (\beta\!\!\beta = (\diamond,\diamond,0) \iff \mathcal{E}_j = \diamond)$$
  $$\wedge\, (\beta\!\!\beta'.k.j = \bot \iff \mathcal{E}_j = \bot) \wedge (\beta\!\!\beta.k.j = \bot \implies \mathcal{E}_j = \bot)$$

  where, if $\beta\!\!\beta = (\beta_1, \beta_2, k)$ and $i \in \{1,2\}$, we write $\beta\!\!\beta.i.1 = \bot$ when $\beta_i = \ulcorner (\!|\bot|\!), C \urcorner$ and $\beta\!\!\beta.i.2 = \bot$ when $\beta_i = \ulcorner C, (\!|\bot|\!) \urcorner$.
- *Nominal closure.* For all $\beta\!\!\beta' \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}{}_{\mathbb{\Sigma}} \beta\!\!\beta$ and permutations π, $\pi \cdot \beta\!\!\beta' \xrightarrow{\pi\cdot\mathcal{E}_1,\pi\cdot\mathcal{E}_2}{}_{\mathbb{\Sigma}} \pi \cdot \beta\!\!\beta$.

Finally, we lift Thm. 19 to pair graphs obtaining continuation graph extension $\mathbb{\Sigma}[\beta\!\!\beta' \mapsto (\mathcal{E}_1, \mathcal{E}_2, \beta\!\!\beta'')]$ and restriction $\mathbb{\Sigma}@\beta\!\!\beta$. We will write $\beta\!\!\beta.i$ to mean β_i $(i \in \{1,2\})$, and $\beta\!\!\beta^m$ to mean $k = m$, when $\beta\!\!\beta = (\beta_1, \beta_2, k)$.

We define simulation on compatible pair configurations.

Definition 78 (Compatible Pair Configurations and Pair (Bi)simulation Tuples)**.** Configurations $\langle\!\langle C_1 \,;\, C_2 \rangle\!\rangle$ and $\langle\!\langle C_1' \,;\, C_2' \rangle\!\rangle$ are compatible, when C_j and C_j' are compatible according to Thm. 20, for $j \in \{1,2\}$.

A tuple $(\langle\!\langle C_1 \,;\, C_2 \rangle\!\rangle, \langle\!\langle C_1' \,;\, C_2' \rangle\!\rangle, \mathbb{\Sigma}, (\beta_1, \beta_2, k))$ is compatible if $\langle\!\langle C_1 \,;\, C_2 \rangle\!\rangle$ and $\langle\!\langle C_1' \,;\, C_2' \rangle\!\rangle$ compatible, $\mathbb{\Sigma}@(\beta_1, \beta_2, k) = \mathbb{\Sigma}$ and for $i, j \in \{1,2\}$:

($\bot$) if $\beta_i.1 = \bot$ then $C_i = (\!|\bot|\!)$; if $\beta_i.2 = \bot$ then $C_i' = (\!|\bot|\!)$;

($\diamond$) if $\beta_i = \diamond$ then $C_i.\mathcal{E} = C_i'.\mathcal{E} = \diamond$ or $\mathsf{an}(C_i.e) = \mathsf{an}(C_i'.e) = \emptyset$; and if $C_i.\mathcal{E} = \diamond$ then $\beta_i = \diamond$.[5]

and moreover:

- if C_i is a proponent and $C_{\widehat{i}}$ an opponent configuration, then $k = i$;
- if C_1 and C_2 are proponent configurations, then $k = 0$;
- if $k = 0$ then $\beta_1.e = \beta_2.e$ and $C_1.e = C_2.e$ or $C_1.\mathcal{E} = C_2.\mathcal{E}$, and the same for C_1', C_2'.

Definition 79 (Weak Pair (Bi)Simulation)**.** A relation $\mathbb{R}$ with elements of the form $(\mathbb{C}_1, \mathbb{C}_2, \mathbb{\Sigma}, \beta\!\!\beta)$, and membership thereof denoted $\mathbb{C}_1\ \mathbb{R}_{\mathbb{\Sigma},\beta\!\!\beta}\ \mathbb{C}_2$, is called *weak pair simulation* when for all $\mathbb{C}_1\ \mathbb{R}_{\mathbb{\Sigma},\beta\!\!\beta^k}\ \mathbb{C}_2$ we have $(\mathbb{C}_1, \mathbb{C}_2, \mathbb{\Sigma}, \beta\!\!\beta^k)$ compatible and:

0. if $\mathbb{C}_1\downarrow$ then $\mathbb{C}_2\downarrow$
1. if $\mathbb{C}_1 \xrightarrow{\underline{\mathsf{ret}}(D[\vec{\alpha}]),k} \mathbb{C}_1'$ with $\vec{\alpha} \# \mathbb{C}_2$ then $\mathbb{C}_2 \xRightarrow{\underline{\mathsf{ret}}(D[\vec{\alpha}]),k} \mathbb{C}_2'$ and $\mathbb{C}_1'\ \mathbb{R}_{\mathbb{\Sigma},\beta\!\!\beta^k}\ \mathbb{C}_2'$
2. if $\mathbb{C}_1 \xrightarrow{\eta,k} \mathbb{C}_1'$ then $\mathbb{C}_2 \xRightarrow{\eta,k} \mathbb{C}_2'$ and $\mathbb{C}_1'\ \mathbb{R}_{\mathbb{\Sigma},\beta\!\!\beta^k}\ \mathbb{C}_2'$, for $\eta \in \{\tau, \mathsf{call}(\alpha, D)\}$

[5] And also $C_i'.\mathcal{E} = \diamond$ by compatibility of C_i, C_i'.

(3) if $\mathbb{C}_1 \xmapsto{\mathsf{call}(i,D[\vec{\alpha}]),j} \mathbb{C}_1'$ with $\vec{\alpha} \# \mathbb{C}_2$ then $\mathbb{C}_2 \xrightarrow{\mathsf{call}(i,D[\vec{\alpha}]),j} \mathbb{C}_2'$ and $\mathbb{C}_1' \; \mathbb{R}_{\mathbb{\Sigma}',\mathbb{\beta}'} \; \mathbb{C}_2'$ and $\mathbb{\Sigma}' = \mathbb{\Sigma}[\mathbb{\beta}' \mapsto (\mathcal{E}_1, \mathcal{E}_2, \mathbb{\beta}^k)]$ with

$$\begin{array}{llll} \mathbb{\beta}' = (\ulcorner \mathbb{C}_1'.1, \mathbb{C}_2'.1 \urcorner, \ulcorner \mathbb{C}_1'.2, \mathbb{C}_2'.2 \urcorner, 0) & \mathcal{E}_1 = \mathbb{C}_1.1.\mathcal{E} = \mathbb{C}_1.2.\mathcal{E}, & \mathcal{E}_2 = \mathbb{C}_2.1.\mathcal{E} = \mathbb{C}_2.2.\mathcal{E} & \text{if } j = 0; \text{ or} \\ \mathbb{\beta}' = (\ulcorner \mathbb{C}_1'.1, \mathbb{C}_2'.1 \urcorner, \mathbb{\beta}.2, 1) & \mathcal{E}_1 = \mathbb{C}_1.1.\mathcal{E} & \mathcal{E}_2 = \mathbb{C}_2.1.\mathcal{E} & \text{if } j = 1; \text{ or} \\ \mathbb{\beta}' = (\mathbb{\beta}.1, \ulcorner \mathbb{C}_1'.2, \mathbb{C}_2'.2 \urcorner, 2) & \mathcal{E}_2 = \mathbb{C}_1.2.\mathcal{E} & \mathcal{E}_2 = \mathbb{C}_2.2.\mathcal{E} & \text{if } j = 2 \end{array}$$

(4) if $\mathbb{C}_1 \xmapsto{\mathsf{ret}(D),k} \mathbb{C}_1'$ and $\mathbb{\beta}^k \xrightarrow{\mathcal{E}_1,\mathcal{E}_2}_{\mathbb{\Sigma}} \mathbb{\beta}'$ then $\mathbb{C}_2 \xRightarrow{\mathsf{ret}(D),k} \mathbb{C}_2'$ and $\mathbb{C}_1'[\mathcal{E}_1/\chi] \; \mathbb{R}_{\mathbb{\Sigma}',\mathbb{\beta}'} \; \mathbb{C}_2'[\mathcal{E}_2/\chi]$ with $\mathbb{\Sigma}' = \mathbb{\Sigma}@\mathbb{\beta}'$.

Pair Similarity ($\widehat{\lesssim}$) is the largest weak simulation. Relation $\mathbb{R}$ is a *weak pair bisimulation* when $\mathbb{R}$ and $\mathbb{R}^{-1}$ are weak pair simulations, where $R^{-1} = \{(\mathbb{C}_1, \mathbb{C}_2, \mathbb{\Sigma}^{-1}, \mathbb{\beta}^{-1}) \mid (\mathbb{C}_2, \mathbb{C}_1, \mathbb{\Sigma}, \mathbb{\beta}) \in \mathbb{R}\}$. Pair bisimilarity ($\hat{\approx}$) is the largest weak bisimulation.

Definition 80. Given two continuation graphs Σ_1, Σ_2 we construct the product graph:

$$\frac{}{((\diamond,\diamond,0),\diamond,\diamond,(\diamond,\diamond,0)) \in \Sigma_1 \otimes \Sigma_2}$$

$$\frac{(\beta_1,\beta_2,k) \in \mathsf{dom}(\Sigma_1 \otimes \Sigma_2) \qquad \forall i \in \{1,2\}.\ (\beta_i',\mathcal{E},\mathcal{E}',\beta_i) \in \Sigma_i \qquad \forall j \in \{1,2\}.\ \beta_1'.j.e = \beta_2'.j.e}{((\beta_1',\beta_2',0),\mathcal{E},\mathcal{E}',(\beta_1,\beta_2,k)) \in \Sigma_1 \otimes \Sigma_2}$$

$$\frac{(\beta_1,\beta_2,k) \in \mathsf{dom}(\Sigma_1 \otimes \Sigma_2) \quad (\beta_i',\mathcal{E},\mathcal{E}',\beta_i) \in \Sigma_i \qquad \beta_{\widehat{i}} = \beta_{\widehat{i}}' \qquad i \in \{1,2\}}{((\beta_1',\beta_2',i),\mathcal{E},\mathcal{E}',(\beta_1,\beta_2,k)) \in \Sigma_1 \otimes \Sigma_2}$$

Lemma 81. *Suppose Σ_1, Σ_2 well-formed continuation graphs; then $\Sigma_1 \otimes \Sigma_2$ is a well-formed pair continuation graph.*

Proof. By induction on the construction of $\Sigma_1 \otimes \Sigma_2$, reasoning about each condition separately and using the induction hypothesis for proving reachability and the second part of closure. Because Σ_i ($i \in \{1,2\}$) are closed under permutations of their tuples, so is the product graph. □

Moreover, given two stackless configuration relations $\mathcal{R}_1$,$\mathcal{R}_2$, we define the pair relation $\mathcal{R}_1 \otimes \mathcal{R}_2$ by induction:

$$\frac{\begin{array}{cc} C_1 \; \mathcal{R}_{1\,\Sigma_1,\beta_1} \; C_1' & C_2 \; \mathcal{R}_{2\,\Sigma_2,\beta_2} \; C_2' \\ C_1.\Phi = C_2.\Phi & C_1'.\Phi = C_2'.\Phi \end{array} \quad \mathbb{\beta} = (\beta_1,\beta_2,0) \in \mathsf{dom}(\Sigma_1 \otimes \Sigma_2)}{\langle\!\langle C_1 ; C_2 \rangle\!\rangle \; (\mathcal{R}_1 \otimes \mathcal{R}_2)_{(\Sigma_1 \otimes \Sigma_2)@\mathbb{\beta},\,\mathbb{\beta}} \; \langle\!\langle C_1' ; C_2' \rangle\!\rangle}$$

$$\frac{\begin{array}{cc} C_1 \; \mathcal{R}_{1\,\Sigma_1,\beta_1} \; C_1' & C_2 \; \mathcal{R}_{2\,\Sigma_2,\beta_2} \; C_2' \end{array} \quad \begin{array}{cc} \mathbb{\beta} = (\beta_1,\beta_2,i) \in \mathsf{dom}(\Sigma_1 \otimes \Sigma_2) & C_{\widehat{i}}, C_{\widehat{i}}' \text{ opponent configurations} \end{array}}{\langle\!\langle C_1 ; C_2 \rangle\!\rangle \; (\mathcal{R}_1 \otimes \mathcal{R}_2)_{(\Sigma_1 \otimes \Sigma_2)@\mathbb{\beta},\,\mathbb{\beta}} \; \langle\!\langle C_1' ; C_2' \rangle\!\rangle}$$

Lemma 82. *Let $\mathcal{R}_1$ and $\mathcal{R}_2$ are stackless simulations; then $\mathcal{R}_1 \otimes \mathcal{R}_2$ is a pair simulation.*

Proof. By showing that (η, i) transitions are matched because $\mathcal{R}_i$ is a simulation (for $i \in \{1,2\}$), and $\eta, 0$ transitions are matched because both $\mathcal{R}_1$, $\mathcal{R}_2$ are simulations and the constituent configurations can perform the same moves. In addition we use simple lemmas to show that the extension and reachability operations of the pair continuation graph is captured by the construction of the product relation (cf. Sec. G.1). □

G.1 Pair Configuration Lemmas

Lemma 83. *Suppose $\Sigma_i = \Sigma_i@\beta_i (i \in \{1,2\})$ are valid call graphs. Then $(\beta_1,\beta_2,k) \in \mathsf{dom}(\Sigma_1 \otimes \Sigma_2)$.* □

Lemma 84. *Suppose*

$$\begin{array}{ll} \Sigma_i' = \Sigma_i[\beta_i' \mapsto (E_a, E_b, \beta_i)] & (i \in \{1,2\}) \\ \Sigma_i = \Sigma_i@\beta_i & (i \in \{1,2\}) \end{array}$$

Then

$$(\beta_1',\beta_2',k') \in \mathsf{dom}(\Sigma_1' \otimes \Sigma_2') \qquad \textit{and}$$

$$(\Sigma_1' \otimes \Sigma_2')@(\beta_1',\beta_2',k') = ((\Sigma_1 \otimes \Sigma_2)@(\beta_1,\beta_2,k))[(\beta_1',\beta_2',k') \mapsto (E_a,E_b,k)]$$

□

Lemma 85. *Suppose*

$$\Sigma_i = \Sigma_i @ \beta_i \qquad (i \in \{1,2\})$$
$$(\beta_i', E_{1a}, E_{2b}, \beta_i) \in \Sigma_i \qquad (i \in \{1,2\})$$

Then

$$((\Sigma_1 \otimes \Sigma_2)@(\beta_1', \beta_2', k'))@(\beta_1, \beta_2, k) = (\Sigma_1 \otimes \Sigma_2)@(\beta_1, \beta_2, k)$$

□

Lemma 86. *Suppose* $\mathbb{\beta} \in \mathsf{dom}(\mathbb{\Sigma})$*; then:*

(1) $[\![\mathbb{\Sigma}]\!]@[\![\mathbb{\beta}]\!] = [\![\mathbb{\Sigma}@\mathbb{\beta}]\!]$

(2) $[\![\mathbb{\Sigma}]\!][[\![\mathbb{\beta}']\!] \mapsto (\mathcal{E}_1, \mathcal{E}_2, [\![\mathbb{\beta}]\!])] = [\![\mathbb{\Sigma}[\mathbb{\beta}' \mapsto (\mathcal{E}_1, \mathcal{E}_2, \mathbb{\beta})]]\!]$ □

H Up to Separation

H.1 Soundness of Up to Separation

Proof. (Thm. 50) Let

$$C_1 \oplus_I^k C_2 \ \mathsf{sep}(\mathcal{R})_{\Sigma, [\![\mathbb{\beta}_{12}]\!]} \ C_1' \oplus_I^k C_2'$$
$$C_1 = (\!|\Gamma_1 \oplus_{I,L} \Gamma \,;\, s_1 \oplus_L s \,;\, e_1|\!) \qquad C_2 = (\!|\Gamma_2 \oplus_{I,L} \Gamma \,;\, s_2 \oplus_L s \,;\, \mathcal{E}_2|\!)$$
$$C_1' = (\!|\Gamma_1' \oplus_{I,L'} \Gamma \,;\, s_1' \oplus_{L'} s' \,;\, e_1'|\!) \qquad C_2' = (\!|\Gamma_2' \oplus_{I,L'} \Gamma \,;\, s_2' \oplus_{L'} s' \,;\, \mathcal{E}_2'|\!)$$

and $C_i \sqsubseteq_{\approx_{\Sigma_i, \beta_i}} C_i'$ $(i \in \{1,2\})$ and $\Sigma = [\![(\Sigma_1 \otimes \Sigma_2)@\mathbb{\beta}_{12}]\!]$ and $\mathbb{\beta}_{12} = (\beta_1, \beta_2, k)$. We show the case where $k = 1$ and the two interesting subcases of **wp**:

• Case $C_1 \oplus_I^1 C_2 \overset{\eta}{\mapsto} C_{32}$ and $\eta = \mathsf{ret}(D)$ and $(\mathcal{E}_3, \mathcal{E}_3', [\![\mathbb{\beta}_{32}]\!]) \in \Sigma([\![\beta_{12}]\!])$:
We have $C_1 \overset{\eta}{\mapsto} C_3$ and

$$C_3 = (\!|\Gamma_1, \Gamma_3 \oplus_{I,L} \Gamma \,;\, s_1 \oplus_L s \,;\, \chi|\!) \qquad C_{32} = C_3 \oplus_I^k C_2$$
$$(\mathcal{E}_3, \mathcal{E}_3', \beta_3) \in \Sigma_1(\beta_1) \qquad \mathbb{\beta}_{32} = (\beta_3, \beta_2, k_{32})$$

We then have

$$\langle\!\langle C_1 \,;\, C_2 \rangle\!\rangle \overset{\eta,1}{\longmapsto} \langle\!\langle C_3 \,;\, C_2 \rangle\!\rangle \qquad (\mathcal{E}_3, \mathcal{E}_3', \mathbb{\beta}_{32}) \in (\Sigma_1 \oplus \Sigma_2)(\mathbb{\beta}_{12})$$

By Thm. 82, and $\langle\!\langle C_1 \,;\, C_2 \rangle\!\rangle \ (\sqsubseteq_{\approx} \otimes \sqsubseteq_{\approx})_{(\Sigma_1 \otimes \Sigma_2)@\mathbb{\beta}_{12},\ \mathbb{\beta}_{12}} \ \langle\!\langle C_1' \,;\, C_2' \rangle\!\rangle$:

$$\langle\!\langle C_1' \,;\, C_2' \rangle\!\rangle \overset{\eta,1}{\Longrightarrow} \langle\!\langle C_3' \,;\, C_2 \rangle\!\rangle \qquad C_3' = (\!|\Gamma_1', \Gamma_3' \oplus_{I,L'} \Gamma' \,;\, s_1' \oplus_L s' \,;\, \chi|\!)$$
$$\langle\!\langle C_3 \,;\, C_2 \rangle\!\rangle[\mathcal{E}_3/\chi] = \langle\!\langle C_3[\mathcal{E}_3/\chi] \,;\, C_2 \rangle\!\rangle \ (\sqsubseteq_{\approx} \otimes \sqsubseteq_{\approx})_{(\Sigma_1 \otimes \Sigma_2)@\mathbb{\beta}_{32},\ \mathbb{\beta}_{32}} \ \langle\!\langle C_3'[\mathcal{E}_3'/\chi] \,;\, C_2 \rangle\!\rangle = \langle\!\langle C_3' \,;\, C_2 \rangle\!\rangle[\mathcal{E}_3'/\chi]$$

Moreover,

$$C_1' \oplus_I^1 C_2' \overset{\eta}{\Rightarrow} C_3' \oplus_I^1 C_2'$$
$$C_3[\mathcal{E}_3/\chi] \oplus_I^{k_{32}} C_2 \ \mathsf{sep}(\sqsubseteq_{\approx})_{[\![(\Sigma_1 \otimes \Sigma_2)@\mathbb{\beta}_{32}]\!],\ [\![\mathbb{\beta}_{32}]\!]} \ C_3'[\mathcal{E}_3/\chi] \oplus_I^{k_{32}} C_2'$$

• Case $C_1 \oplus_I^1 C_2 \overset{\eta}{\mapsto} C_{32}$ and $\eta = \underline{\mathsf{call}}(i, D[\vec{\alpha}])$:
If $i \in \mathsf{dom}(\Gamma_1)$, we have $C_1 \overset{\eta}{\mapsto} C_3$ and

$$C_3 = (\!|\Gamma_1 \oplus_{I,L} \Gamma \,;\, s_1 \oplus_L s \,;\, \mathcal{E}_3|\!) \qquad C_{32} = C_3 \oplus_I^1 C_2$$

We then have $\langle\!\langle C_1 \,;\, C_2 \rangle\!\rangle \overset{\eta,1}{\longmapsto} \langle\!\langle C_3 \,;\, C_2 \rangle\!\rangle$. By Thm. 82, $\langle\!\langle C_1 \,;\, C_2 \rangle\!\rangle \ (\sqsubseteq_{\approx} \otimes \sqsubseteq_{\approx})_{(\Sigma_1 \otimes \Sigma_2)@\mathbb{\beta}_{12},\ \mathbb{\beta}_{12}} \ \langle\!\langle C_1' \,;\, C_2' \rangle\!\rangle$:

$$\langle\!\langle C_1' \,;\, C_2' \rangle\!\rangle \overset{\eta,1}{\longmapsto} \langle\!\langle C_3' \,;\, C_2' \rangle\!\rangle \qquad \beta_e = \ulcorner C_3, C_3' \urcorner \qquad \mathbb{\beta}_{32} = (\beta_3, \beta_2, 1)$$
$$\mathbb{\Sigma}_{32} = (\Sigma_1 \oplus \Sigma_2)[\mathbb{\beta}_{32} \mapsto (\mathcal{E}_3, \mathcal{E}_3', \mathbb{\beta}_{12})] \qquad \langle\!\langle C_3 \,;\, C_2 \rangle\!\rangle \ (\sqsubseteq_{\approx} \otimes \sqsubseteq_{\approx})_{\mathbb{\Sigma}_{32},\ \mathbb{\beta}_{32}} \ \langle\!\langle C_3' \,;\, C_2 \rangle\!\rangle$$

Moreover,

$$C_1' \oplus_I^1 C_2' \overset{\eta}{\to} C_3' \oplus_I^1 C_2'$$
$$C_3 \oplus_I^1 C_2 \ \mathsf{sep}(\sqsubseteq_{\approx})_{[\![\mathbb{\Sigma}_{32}@\mathbb{\beta}_{32}]\!],\ [\![\mathbb{\beta}_{32}]\!]} \ C_3' \oplus_I^1 C_2'$$

If $i \in \Gamma$ the proof is similar, with the exception that we consider transition $\eta, 0$ from the pair configurations. □

H.2 Completeness of Up to Separation *Proof.* (Thm. 51) Let $\mathcal{R}$ be a simulation and $C_1 \oplus^0_I C_2 \sqsubseteq_{\approx_{\Sigma,\beta}} C'_1 \oplus^0_I C'_2$. We will show that $C_1 \sqsubseteq_{\approx_{\Sigma',\beta'}} C'_1$, for some Σ', β' (the proof for C_2, C'_2 is symmetric). We unfold the definition for $C_1 \oplus^0_I C_2$ and $C'_1 \oplus^0_I C'_2$, considering cases. The case where $C_1 \oplus^0_I C_2 = C_1 = \langle \bot \rangle$ is trivial. The remaining two cases are similar and we only show one:

$$C_1 = (\!|\Gamma_1 \oplus_{I,L} \Gamma \,;\, s_1 \oplus s \,;\, \mathcal{E}|\!)$$
$$C_2 = (\!|\Gamma_2 \oplus_{I,L} \Gamma \,;\, s_2 \oplus_L s \,;\, \mathcal{E}|\!)$$
$$C_1 \oplus^0_I C_2 = (\!|\Gamma_1 \oplus \Gamma_2 \oplus \Gamma \,;\, s_1 \oplus s_2 \oplus s \,;\, \mathcal{E}|\!)$$
$$C_1 \oplus^0_I C_2 \succ^{\mathsf{wk}(n)} (\!|\Gamma_1 \oplus \Gamma \,;\, s_1 \oplus s_2 \oplus s \,;\, \mathcal{E}|\!) = C_3$$

where $\succ^{\mathsf{wk}(n)} = \succ^{\mathsf{wk}}_{i_1} \ldots \succ^{\mathsf{wk}}_{i_n}$ and $\{i_1, \ldots, i_n\} = \mathsf{dom}(\Gamma_2)$. Similarly

$$C'_1 = (\!|\Gamma'_1 \oplus_{I,L'} \Gamma' \,;\, s'_1 \oplus s' \,;\, \mathcal{E}'|\!)$$
$$C'_2 = (\!|\Gamma'_2 \oplus_{I,L'} \Gamma' \,;\, s'_2 \oplus_{L'} s' \,;\, \mathcal{E}'|\!)$$
$$C'_1 \oplus^0_I C'_2 = (\!|\Gamma'_1 \oplus \Gamma'_2 \oplus \Gamma' \,;\, s'_1 \oplus s'_2 \oplus s' \,;\, \mathcal{E}'|\!)$$
$$C'_1 \oplus^0_I C'_2 \succ^{\mathsf{wk}(n)} (\!|\Gamma'_1 \oplus \Gamma' \,;\, s'_1 \oplus s'_2 \oplus s' \,;\, \mathcal{E}'|\!) = C'_3$$

By Thm. 74, we have $C_3 \sqsubseteq_{\approx_{\Sigma_3,\beta_3}} C'_3$, for some Σ_3, β_3. Moreover,

$$C_3 \succ^{\mathsf{gc}}_{A_2,\mathsf{dom}(s_2)} (\!|\Gamma_1 \oplus \Gamma \,;\, s_1 \oplus s \,;\, \mathcal{E}|\!) = C_1$$
$$C'_3 \succ^{\mathsf{gc}}_{A'_2,\mathsf{dom}(s'_2)} (\!|\Gamma'_1 \oplus \Gamma' \,;\, s'_1 \oplus s' \,;\, \mathcal{E}'|\!) = C'_1$$

By Thm. 71, we have $C_1 \sqsubseteq_{\approx_{\Sigma_4,\beta_4}} C'_1$, for some Σ_4, β_4. □

References

[1] n.d.. Countdown Timer | Android Developers. https://developer.android.com/reference/android/os/CountDownTimer. Accessed: 2023-10-10.

[2] n.d.. DOM Standard. https://dom.spec.whatwg.org/. Accessed: 2023-10-10.

[3] n.d.. Events | jQuery API Documentation. https://api.jquery.com/category/events/. Accessed: 2023-10-10.

[4] n.d.. EventTarget - Web APIs | MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/EventTarget. Accessed: 2023-10-10.

[5] n.d.. Input events overview | Android Developers. https://developer.android.com/develop/ui/views/touch-and-input/input-events. Accessed: 2023-10-10.

[6] n.d.. Introduction to Event Listeners (The Java™ Tutorials). https://docs.oracle.com/javase/tutorial/uiswing/events/intro.html. Accessed: 2023-10-10.

[7] Samson Abramsky. 1990. The Lazy Lambda Calculus. In *Research Topics in Functional Programming*. Addison-Wesley Longman Publishing Co., Inc., USA, 65–116.

[8] Samson Abramsky, Kohei Honda, and Guy McCusker. 1998. A Fully Abstract Game Semantics for General References. In *Thirteenth Annual IEEE Symposium on Logic in Computer Science, Indianapolis, Indiana, USA, June 21-24, 1998*. IEEE Computer Society, 334–344. https://doi.org/10.1109/LICS.1998.705669

[9] Samson Abramsky, Radha Jagadeesan, and Pasquale Malacaria. 2000. Full Abstraction for PCF. *Inf. Comput.* 163, 2, 409–470. https://doi.org/10.1006/inco.2000.2930

[10] Amal Ahmed, Derek Dreyer, and Andreas Rossberg. 2009. State-dependent representation independence. In *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2009, Savannah, GA, USA, January 21-23, 2009*, Zhong Shao and Benjamin C. Pierce (Eds.). ACM, 340–353. https://doi.org/10.1145/1480881.1480925

[11] Dariusz Biernacki and Sergueï Lenglet. 2012. Normal Form Bisimulations for Delimited-Control Operators. In *Functional and Logic Programming - 11th International Symposium, FLOPS 2012, Kobe, Japan, May 23-25, 2012. Proceedings (Lecture Notes in Computer Science, Vol. 7294)*, Tom Schrijvers and Peter Thiemann (Eds.). Springer, 47–61. https://doi.org/10.1007/978-3-642-29822-6_7

[12] Dariusz Biernacki, Sergueï Lenglet, and Piotr Polesiuk. 2019. A Complete Normal-Form Bisimilarity for State. In *Foundations of Software Science and Computation Structures - 22nd International Conference, FOSSACS 2019, Proceedings (Lecture Notes in Computer Science, Vol. 11425)*, Mikolaj Bojanczyk and Alex Simpson (Eds.). Springer, 98–114. https://doi.org/10.1007/978-3-030-17127-8_6

[13] Mikolaj Bojanczyk, Bartek Klin, and Slawomir Lasota. 2014. Automata theory in nominal sets. *Log. Methods Comput. Sci.* 10, 3 (2014). https://doi.org/10.2168/LMCS-10(3:4)2014

[14] Mikolaj Bojanczyk, Bartek Klin, Slawomir Lasota, and Szymon Torunczyk. 2013. Turing Machines with Atoms. In *28th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2013, New Orleans, LA, USA, June 25-28, 2013*. IEEE Computer Society, 183–192. https://doi.org/10.1109/LICS.2013.24

[15] Ahmed Bouajjani, Javier Esparza, and Oded Maler. 1997. Reachability Analysis of Pushdown Automata: Application to Model-Checking. In *CONCUR '97: Concurrency Theory, 8th International Conference, Warsaw, Poland, July 1-4, 1997, Proceedings (Lecture Notes in Computer Science, Vol. 1243)*, Antoni W. Mazurkiewicz and Józef Winkowski (Eds.). Springer, 135–150. https://doi.org/10.1007/3-540-63141-0_10

[16] Edward Y. C. Cheng and Michael Kaminski. 1998. Context-Free Languages over Infinite Alphabets. *Acta Informatica* 35, 3 (1998), 245–267. https://doi.org/10.1007/s002360050120

[17] Aleksandar S. Dimovski. 2014. Program verification using symbolic game semantics. *Theor. Comput. Sci.* 560 (2014), 364–379. https://doi.org/10.1016/j.tcs.2014.01.016

[18] Derek Dreyer, Georg Neis, and Lars Birkedal. 2010. The impact of higher-order state and control effects on local relational reasoning. In *Proceeding of the 15th ACM SIGPLAN international conference on Functional programming, ICFP 2010, Baltimore, Maryland, USA, September 27-29, 2010*, Paul Hudak and Stephanie Weirich (Eds.). ACM, 143–156. https://doi.org/10.1145/1863543.1863566

[19] Christopher Earl, Matthew Might, and David Van Horn. 2010. Pushdown Control-Flow Analysis of Higher-Order Programs. *CoRR* abs/1007.4268 (2010). arXiv:1007.4268 http://arxiv.org/abs/1007.4268

[20] Javier Esparza, Antonín Kucera, and Stefan Schwoon. 2001. Model-Checking LTL with Regular Valuations for Pushdown Systems. In *Theoretical Aspects of Computer Software, 4th International Symposium, TACS 2001, Sendai, Japan, October 29-31, 2001, Proceedings (Lecture Notes in Computer Science, Vol. 2215)*, Naoki Kobayashi and Benjamin C. Pierce (Eds.). Springer, 316–339. https://doi.org/10.1007/3-540-45500-0_16

[21] Matthias Felleisen. 1987. *The calculi of lambda-nu-cs conversion: a syntactic theory of control and state in imperative higher-order programming languages*. Ph. D. Dissertation. Indiana University.

[22] Dennis Felsing, Sarah Grebing, Vladimir Klebanov, Philipp Rümmer, and Mattias Ulbrich. 2014. Automating regression verification. In *ACM/IEEE International Conference on Automated Software Engineering, ASE '14, Vasteras, Sweden - September 15 - 19, 2014*, Ivica Crnkovic, Marsha Chechik, and Paul Grünbacher (Eds.). ACM, 349–360. https://doi.org/10.1145/2642937.2642987

[23] Alain Finkel, Bernard Willems, and Pierre Wolper. 1997. A direct symbolic approach to model checking pushdown systems. In *Second International Workshop on Verification of Infinite State Systems, Infinity 1997, Bologna, Italy, July 11-12, 1997 (Electronic Notes in Theoretical Computer Science, Vol. 9)*, Faron Moller (Ed.). Elsevier, 27–37. https://doi.org/10.1016/S1571-0661(05)80426-8

[24] Murdoch Gabbay and Andrew M. Pitts. 2002. A New Approach to Abstract Syntax with Variable Binding. *Formal Aspects Comput.* 13, 3-5 (2002), 341–363. https://doi.org/10.1007/s001650200016

[25] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. 1995. *Design patterns: elements of reusable object-oriented software*. Pearson Deutschland GmbH.

[26] Dan R. Ghica and Guy McCusker. 2003. The regular-language semantics of second-order idealized A_{LGOL}. *Theor. Comput. Sci.* 309, 1-3 (2003), 469–502. https://doi.org/10.1016/S0304-3975(03)00315-3

[27] Dan R. Ghica and Nikos Tzevelekos. 2012. A System-Level Game Semantics. In *Proceedings of the 28th Conference on the Mathematical Foundations of Programming Semantics, MFPS 2012, Bath, UK, June 6-9, 2012 (Electronic Notes in Theoretical Computer Science, Vol. 286)*, Ulrich Berger and Michael W. Mislove (Eds.). Elsevier, 191–211. https://doi.org/10.1016/j.entcs.2012.08.013

[28] Thomas Gilray, Steven Lyde, Michael D. Adams, Matthew Might, and David Van Horn. 2016. Pushdown control-flow analysis for free. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20 - 22, 2016*, Rastislav Bodík and Rupak Majumdar (Eds.). ACM, 691–704. https://doi.org/10.1145/2837614.2837631

[29] Benny Godlin and Ofer Strichman. 2008. Inference rules for proving the equivalence of recursive procedures. *Acta Informatica* 45, 6 (2008), 403–439. https://doi.org/10.1007/s00236-008-0075-2

[30] Benny Godlin and Ofer Strichman. 2009. Regression verification. In *Proceedings of the 46th Design Automation Conference, DAC 2009, San Francisco, CA, USA, July 26-31, 2009*. ACM, 466–471. https://doi.org/10.1145/1629911.1630034

[31] Andrew D. Gordon, Paul D. Hankin, and Søren B. Lassen. 1999. Compilation and equivalence of imperative objects. *Journal of Functional Programming* 9, 4 (1999), 373–426. https://doi.org/10.1017/S0956796899003482

[32] David Hopkins, Andrzej S. Murawski, and C.-H. Luke Ong. 2012. Hector: An Equivalence Checker for a Higher-Order Fragment of ML. In *Computer Aided Verification - 24th International Conference, CAV 2012, Berkeley, CA, USA, July 7-13, 2012 Proceedings (Lecture Notes in Computer Science, Vol. 7358)*, P. Madhusudan and Sanjit A. Seshia (Eds.). Springer, 774–780. https://doi.org/10.1007/978-3-642-31424-7_63

[33] David Van Horn and Matthew Might. 2010. Abstracting abstract machines. In *Proceeding of the 15th ACM SIGPLAN international conference on Functional programming, ICFP 2010, Baltimore, Maryland, USA, September 27-29, 2010*, Paul Hudak and Stephanie Weirich (Eds.). ACM, 51–62. https://doi.org/10.1145/1863543.1863553

[34] Chung-Kil Hur, Derek Dreyer, Georg Neis, and Viktor Vafeiadis. 2012. The marriage of bisimulations and Kripke logical relations. In *Proceedings of the 39th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2012, Philadelphia, Pennsylvania, USA, January 22-28, 2012*, John Field and Michael Hicks (Eds.). ACM, 59–72. https://doi.org/10.1145/2103656.2103666

[35] J. M. E. Hyland and C.-H. Luke Ong. 2000. On Full Abstraction for PCF: I, II, and III. *Inf. Comput.* 163, 2 (2000), 285–408. https://doi.org/10.1006/inco.2000.2917

[36] Guilhem Jaber. 2015. Operational Nominal Game Semantics. In *Foundations of Software Science and Computation Structures - 18th International Conference, FoSSaCS 2015, Proceedings (Lecture Notes in Computer Science, Vol. 9034)*, Andrew M. Pitts (Ed.). Springer, 264–278. https://doi.org/10.1007/978-3-662-46678-0_17

[37] Guilhem Jaber. 2020. SyTeCi: automating contextual equivalence for higher-order programs with references. *Proc. ACM Program. Lang.* 4, POPL (2020), 59:1–59:28. https://doi.org/10.1145/3371127

[38] Guilhem Jaber and Nicolas Tabareau. 2015. Kripke Open Bisimulation - A Marriage of Game Semantics and Operational Techniques. In *Programming Languages and Systems - 13th Asian Symposium, APLAS 2015, Pohang, South Korea, November 30 - December 2, 2015, Proceedings (Lecture Notes in Computer Science, Vol. 9458)*, Xinyu Feng and Sungwoo Park (Eds.). Springer, 271–291. https://doi.org/10.1007/978-3-319-26529-2_15

[39] Radha Jagadeesan, Corin Pitcher, and James Riely. 2009. Open Bisimulation for Aspects. *LNCS Trans. Aspect Oriented Softw. Dev.* 5, 72–132. https://doi.org/10.1007/978-3-642-02059-9_3

[40] Jang-Sub Kim, Erchin Serpedin, Dong Ryeol Shin, and Khalid A. Qaraqe. 2008. Handoff Triggering and Network Selection Algorithms for Load-Balancing Handoff in CDMA-WLAN Integrated Networks. *EURASIP J. Wirel. Commun. Netw.* 2008 (2008). https://doi.org/10.1155/2008/136939

[41] Vasileios Koutavas, Yu-Yang Lin, and Nikos Tzevelekos. 2022. From Bounded Checking to Verification of Equivalence via Symbolic Up-to Techniques. In *Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 13244)*, Dana Fisman and Grigore Rosu (Eds.). Springer, 178–195. https://doi.org/10.1007/978-3-030-99527-0_10

[42] Vasileios Koutavas, Yu-Yang Lin, and Nikos Tzevelekos. 2023. Fully Abstract Normal Form Bisimulation for Call-by-Value PCF. In *LICS*. 1–13. https://doi.org/10.1109/LICS56636.2023.10175778

[43] Vasileios Koutavas and Mitchell Wand. 2006. Small bisimulations for reasoning about higher-order imperative programs. In *Proceedings of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2006, Charleston, South Carolina, USA, January 11-13, 2006*, J. Gregory Morrisett and Simon L. Peyton Jones (Eds.). ACM, 141–152. https://doi.org/10.1145/1111037.1111050

[44] Shuvendu K. Lahiri, Chris Hawblitzel, Ming Kawaguchi, and Henrique Rebêlo. 2012. SYMDIFF: A Language-Agnostic Semantic Diff Tool for Imperative Programs. In *Computer Aided Verification - 24th International Conference, CAV 2012, Berkeley, CA, USA, July 7-13, 2012 Proceedings (Lecture Notes in Computer Science, Vol. 7358)*, P. Madhusudan and Sanjit A. Seshia (Eds.). Springer, 712–717. https://doi.org/10.1007/978-3-642-31424-7_54

[45] James Laird. 2007. A Fully Abstract Trace Semantics for General References. In *Automata, Languages and Programming, 34th International Colloquium, ICALP 2007, Wroclaw, Poland, July 9-13, 2007, Proceedings (Lecture Notes in Computer Science, Vol. 4596)*, Lars Arge, Christian Cachin, Tomasz Jurdzinski, and Andrzej Tarlecki (Eds.). Springer, 667–679. https://doi.org/10.1007/978-3-540-73420-8_58

[46] Søren B. Lassen. 1999. Bisimulation in Untyped Lambda Calculus: Böhm Trees and Bisimulation up to Context. In *Fifteenth Conference on Mathematical Foundations of Progamming Semantics, MFPS 1999, Tulane University, New Orleans, LA, USA, April 28 - May 1, 1999 (Electronic Notes in Theoretical Computer Science, Vol. 20)*, Stephen D. Brookes, Achim Jung, Michael W. Mislove, and Andre Scedrov (Eds.). Elsevier, 346–374. https://doi.org/10.1016/S1571-0661(04)80083-5

[47] Søren B. Lassen. 2005. Eager Normal Form Bisimulation. In *20th IEEE Symposium on Logic in Computer Science (LICS 2005), 26-29 June 2005, Chicago, IL, USA, Proceedings*. IEEE Computer Society, 345–354. https://doi.org/10.1109/LICS.2005.15

[48] Søren B. Lassen. 2005. Normal Form Simulation for McCarthy's Amb. In *Proceedings of the 21st Annual Conference on Mathematical Foundations of Programming Semantics, MFPS 2005, Birmingham, UK, May 18-21, 2005 (Electronic Notes in Theoretical Computer Science, Vol. 155)*, Martín Hötzel Escardó, Achim Jung, and Michael W. Mislove (Eds.). Elsevier, 445–465. https://doi.org/10.1016/j.entcs.2005.11.068

[49] Søren B. Lassen and Paul Blain Levy. 2007. Typed Normal Form Bisimulation. In *Computer Science Logic, 21st International Workshop, CSL 2007, 16th Annual Conference of the EACSL, Lausanne, Switzerland, September 11-15, 2007, Proceedings (Lecture Notes in Computer Science, Vol. 4646)*, Jacques Duparc and Thomas A. Henzinger (Eds.). Springer, 283–297. https://doi.org/10.1007/978-3-540-74915-8_23

[50] Søren B. Lassen and Paul Blain Levy. 2008. Typed Normal Form Bisimulation for Parametric Polymorphism. In *Proceedings of the Twenty-Third Annual IEEE Symposium on Logic in Computer Science, LICS 2008, 24-27 June 2008, Pittsburgh, PA, USA*. IEEE Computer Society, 341–352. https://doi.org/10.1109/LICS.2008.26

[51] Ian Mason and Carolyn Talcott. 1991. Equivalence in functional languages with effects. *Journal of Functional Programming* 1, 3 (1991), 287–327. https://doi.org/10.1017/S0956796800000125

[52] J. H. Morris, Jr. 1968. *Lambda Calculus Models of Programming Languages*. Ph. D. Dissertation. MIT, Cambridge, MA.

[53] Andrzej S. Murawski, Steven J. Ramsay, and Nikos Tzevelekos. 2015. A Contextual Equivalence Checker for IMJ $_*$. In *Automated Technology for Verification and Analysis - 13th International Symposium, ATVA 2015, Shanghai, China, October 12-15, 2015, Proceedings (Lecture Notes in Computer Science, Vol. 9364)*, Bernd Finkbeiner, Geguang Pu, and Lijun Zhang (Eds.). Springer, 234–240. https://doi.org/10.1007/978-3-319-24953-7_19

[54] Andrzej S. Murawski, Steven J. Ramsay, and Nikos Tzevelekos. 2017. Reachability in pushdown register automata. *J. Comput. Syst. Sci.* 87 (2017), 58–83. https://doi.org/10.1016/j.jcss.2017.02.008

[55] Andrzej S. Murawski and Nikos Tzevelekos. 2021. Game Semantics for Interface Middleweight Java. *J. ACM* 68, 1 (2021), 4:1–4:51. https://doi.org/10.1145/3428676

[56] Hanno Nickau. 1994. Hereditarily Sequential Functionals. In *Logical Foundations of Computer Science, Third International Symposium, LFCS'94, St. Petersburg, Russia, July 11-14, 1994, Proceedings (Lecture Notes in Computer Science, Vol. 813)*, Anil Nerode and Yuri V. Matiyasevich (Eds.). Springer, 253–264. https://doi.org/10.1007/3-540-58140-5_25

[57] Damien Pous. 2016. Coinduction All the Way Up. In *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science, LICS '16, New York, NY, USA, July 5-8, 2016*, Martin Grohe, Eric Koskinen, and Natarajan Shankar (Eds.). ACM, 307–316. https://doi.org/10.1145/2933575.2934564

[58] Damien Pous and Davide Sangiorgi. 2012. Enhancements of the bisimulation proof method. In *Advanced Topics in Bisimulation and Coinduction*, Davide Sangiorgi and Jan J. M. M. Rutten (Eds.). Cambridge tracts in theoretical computer science, Vol. 52. Cambridge University Press, 233–289.

[59] Davide Sangiorgi. 1994. The Lazy Lambda Calculus in a Concurrency Scenario. *Inf. Comput.* 111, 1 (1994), 120–153. https://doi.org/10.1006/inco.1994.1042

[60] Davide Sangiorgi, Naoki Kobayashi, and Eijiro Sumii. 2011. Environmental bisimulations for higher-order languages. *ACM Trans. Program. Lang. Syst.* 33, 1 (2011), 5:1–5:69. https://doi.org/10.1145/1889997.1890002

[61] Stefan Schwoon. 2002. *Model checking pushdown systems.* Ph. D. Dissertation. Technical University Munich, Germany.

[62] Kristian Støvring and Søren B. Lassen. 2007. A complete, co-inductive syntactic theory of sequential control and state. In *Proceedings of the 34th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2007, Nice, France, January 17-19, 2007*, Martin Hofmann and Matthias Felleisen (Eds.). ACM, 161–172. https://doi.org/10.1145/1190216.1190244

[63] Eijiro Sumii and Benjamin C. Pierce. 2007. A bisimulation for dynamic sealing. *Theor. Comput. Sci.* 375, 1-3 (2007), 169–192. https://doi.org/10.1016/j.tcs.2006.12.032

[64] Eijiro Sumii and Benjamin C. Pierce. 2007. A bisimulation for type abstraction and recursion. *J. ACM* 54, 5 (2007), 26. https://doi.org/10.1145/1284320.1284325

[65] Dimitrios Vardoulakis and Olin Shivers. 2011. CFA2: a Context-Free Approach to Control-Flow Analysis. *Log. Methods Comput. Sci.* 7, 2 (2011). https://doi.org/10.2168/LMCS-7(2:3)2011